National Information Technology Development Agency

Folio Handbook

The complete documentation: architecture, processes, operation and use

National Information Technology Development Agency · Office of the Director-General/CEO · DGO Digital Operations

Document	Folio Handbook
Reference	NITDA-FOLIO-HO-2.1
Version	1.0
Date	20 October 2026
Classification	Internal
Built from	Repository dgoeaa/ECM_DOCS_DEV, commit c6e49d73
Build	v2026.10.20-1 (Workspace and Portal packages)
Register of record	Version 1.12.87: 0 open, 122 closed

Contents

About this handbook 7

Chapters 7

Reading paths 8

How this handbook relates to the other records 8

1 · Go-live walkthrough 9

1.1 Where things stand (2026-10-10) 9

1.2 The order, at a glance 10

Stage A · Prove the deployed Workspace 10

Stage B · Prove each mail and intake path once 11

Stage C · The browser suite against the live site (MANUAL-3) 13

Stage D · Retire what the new paths replace 14

Stage E · Decisions for the owner 14

Stage F · Clean, record, declare 15

Stage G · Hypercare (first two weeks) 16

2 · Architecture 17

2.1 The shape of the system 17

2.2 The surfaces 18

2.3 Identity: sign-in by e-mailed code 18

2.4 Authorisation: roles, personas and data scope 20

2.5 Configuration and delivery 20

2.6 The standard reply envelope 21

2.7 Observability 21

2.8 Limits that shape the design 22

2.9 Related records 22

3 · Components 23

3.1 Folio Workspace: how the code is layered 23

3.2 Core services 23

3.3 Screens (31 routes, 13 workspaces) 25

3.4 The two deliberate exceptions to "everything through the data client" 26
3.5 NITDA Document Portal (document-portal/) 26
3.6 DGO Correspondence Mailer (mailer/) 27
3.7 Platform-team tools 27
3.8 Power Automate flows 27
3.9 DYNAMIC_ACTIONS operations 28
4 · Data model 29
4.1 Sites 29
4.2 Workspace lists 29
4.3 Portal lists 31
4.4 Operational lists (all flows) 32
4.5 Identifiers 32
4.6 The Workspace's working copy (client state) 33
4.7 Roles, personas and statuses 33
4.8 Status vocabularies 34
5 · Processes 36
5.1 The correspondence lifecycle 36
5.2 Telling the public submitter 38
5.3 Signing in 38
5.4 Giving someone access 38
5.5 Support 38
5.6 Scheduled work 39
5.7 Where the processes are specified in full 39
6 · Service contracts 40
6.1 A request 40
6.2 The client's contracts 40
6.3 Status codes a service returns 42
6.4 Portal and Mailer services 42
6.5 Where to look next 42

7 · Developer guide 44

7.1 Ground rules 44

7.2 Set up 44

7.3 Run it 45

7.4 Check it 45

7.5 Change a screen 45

7.6 Change a flow 46

7.7 Build and deliver the Workspace 47

7.8 Add a contract key (a new service) 47

7.9 Generated files 48

7.10 Debugging a live problem 48

8 · Administrator guide 50

8.1 Who administers what 50

8.2 People and access 50

8.3 Server-side settings: Flow Configuration 51

8.4 Monitoring 52

8.5 Deploying a new Workspace build 52

8.6 Security operations 53

8.7 Test data 53

8.8 Related procedures 54

9 · User guide 55

9.1 Signing in 55

9.2 Finding your way 55

9.3 Tasks by role 56

9.4 Confirmations and messages 58

9.5 The public Document Portal 58

9.6 Getting help 59

10 · Troubleshooting 60

10.1 Cannot sign in 60

10.2 Data does not load after sign-in 60
10.3 "NITDA Folio could not start" 62
10.4 401s in run history that nobody caused 62
10.5 A flow change is refused 62
10.6 Limits and design notes at commissioning 62

11 · Commissioning report 64

1. Purpose 64
2. Declaration 64
3. Scope 65
4. The system as commissioned 65
5. Verification of each application and tool 66
6. Tests against the deployed applications 67
7. Flows and packages 67
8. Notifications 68
9. Readiness registers 69
10. Defects found during commissioning and corrected 69
11. Security 70
12. Test records removed 70
13. Actions for the owner after handover 70
14. Operating notes 71
15. Related documents 71

Appendix A · Commissioning runbook 72

Phase 0 — the internal proof ledger (ITEM-64) 72
Phase 1 — the four migration decisions (ITEM-63) — settled 73
Phase 2 — paste the 11 packages (G-04) 74
Phase 3 — verification, and the write path (ITEM-2) 79
Appendix A · The one key the mapping does not name 80
Appendix B · Confirming a flow before you import 80
Appendix C · If something is wrong 81

Appendix D · What this does not do 81

Part 2 · DGO ECM — Implementation Execution Guide 82

Part 3 · Portal tenant runbook — the console steps for portal and endpoint commissioning 112

Part 4 · Notification tenant runbook — the steps for notification remediation 133

Part 5 · Governance tenant runbook — actions outside the repository 136

Part 6 · SharePoint and Power Automate — the flow build walkthrough 164

Part 7 · Minimal pilot — the short path 183

Part 8 · Commissioning from a phone — Termux and the harvest bookmark 191

Appendix B · The platform at a glance 203

1 · What this is 203

2 · How a request travels 204

3 · Identity 204

4 · The wired estate 204

5 · What it stores 206

6 · How this documentation is kept true 206

7 · Where the work stands 206

About this handbook

The consolidated documentation for NITDA Folio: what it is, how it is built, how its data and processes work, how to take it live, and how to develop, administer and use it.

Every statement here was taken from the code, configuration and registers in this repository, or read from the live tenant on the date given, on **2026-10-10** (register version 1.12.41), and brought up to the commissioning of **2026-10-19** and the live set read on **2026-10-20** by the commissioning report. Where a fact comes from a register or a generated record, the chapter links to it. **If this handbook and a register disagree, the register wins**; tell the platform team so the handbook is corrected.

Chapters

#	Chapter	For	What it answers
1	Go-live walkthrough	Owner, platform team	Every step from today to a declared go-live: who, what, expected result, evidence, item closed
2	Architecture	Everyone technical	The shape of the system, sign-in, authorisation, configuration, limits
3	Components	Developers, administrators	Every screen, core service, surface, tool and flow, and what each does
4	Data model	Developers, administrators	Every SharePoint list, its key columns, identifiers, roles, status vocabularies
5	Processes	Everyone	The correspondence lifecycle end to end, sign-in, access, support, scheduled work
6	Service contracts	Developers	Request and reply formats, headers, timeouts, retries, status codes
7	Developer guide	Developers	Set up, run, test, change a screen or a flow, build and deliver, debug
8	Administrator guide	Administrators	People and roles, settings, monitoring, deployment, security operations
9	User guide	Staff	Signing in, finding your way, tasks by role, messages, getting help
10	Troubleshooting	Administrators, platform team	Diagnosing sign-in and data problems from evidence; known gaps at go-live
11	Commissioning report	Owner, everyone	What was commissioned on 2026-10-19, the proof for each part, and the declaration

Reading paths

You are	Read
The owner preparing go-live	1, then 10.6
New to the platform team	2, 3, 4, 7
An administrator	8, 9, 10
A member of staff	9

How this handbook relates to the other records

Record	Kind	Kept true by
`docs/deployment/PRODUCTION_READINESS_REGISTER.json`	The register of record for open and closed work	`npm run outstanding`; register revisions
`docs/PLATFORM.md`	Generated estate description	`npm run platform` (see the stale points in chapter 10.6)
`docs/deployment/RUNBOOK.md`, `PLAN.md`	Cutover and standing procedures; the plan	hand-maintained, checked by tests
`docs/process/`	The full process catalogue	`npm run process:docs`
`docs/folio-Master-Flows-and-Endpoints/`	**Every** flow and endpoint record: definitions, packages, captures, evidence, registers	its generators and `npm run test:masterflows`
`docs/reference/identification/`	The identification system	`npm run test:node`
This handbook	The consolidated, readable view	maintained by hand; `npm run test:links` checks its links

Flow and endpoint records are kept only in the master folder (owner rule, 2026-10-03). This handbook describes and links to them; it does not copy them.

1 · Go-live walkthrough

> **Completed 2026-10-19.** Every step below has been carried out and every item it names is closed (`npm run outstanding`: 0 open, 122 closed). The results, with their evidence, are in the commissioning report. The steps stay here as the record of how go-live was done, and for re-commissioning.
>
> **Status:** handbook chapter, written 2026-10-10. It carries steps, and owns none: each step's procedure is owned by the readiness register item it names and by `docs/deployment/RUNBOOK.md`.

The path from where Folio stands today to a declared go-live, as an ordered list of steps. Each step names who acts, what they do, what they should see, what is recorded, and which register item it closes.

> **Source of truth.** The open items come from `npm run outstanding`, which reads `docs/deployment/PRODUCTION_READINESS_REGISTER.json` (version 1.12.41 when this chapter was written: 17 open, 105 closed). If this chapter and the register ever disagree, the register wins. Re-run `npm run outstanding` before you start.

1.1 Where things stand (2026-10-10)

Done and verified	Evidence
The Workspace is deployed to `https://folio-platform.page.gd/` (build of 2026-10-09 21:58 UTC: 15 signed addresses, sign-in enforced, no AI addresses).	Six key files compared byte for byte with the delivered zip; ledger `registers/config-build-ledger.json`.
Sign-in by e-mailed code works.	OTP_VERIFY runs at 22:41, 23:28, 23:55 and 00:02 UTC, all succeeded with a `systemadmin` proof for dgsregistry@nitda.gov.ng.
Data loads into the Workspace.	FETCH_ALL runs at 23:55:44 and 00:02:39 UTC from the owner's browser, carrying the proof: 200, about 6.4 MB. The Command Center showed 475 open references, 500 active tasks and 483 overdue.
Browsers fetch new files after every deployment.	The host answers `cache-control: no-cache, must-revalidate` for HTML, JS and CSS, set by the root `.htaccess` shipped in the build.
A false "NITDA Folio could not start" no longer appears while someone types their code.	Browser test `tests/deeplinks-signin.spec.js` ("waiting longer than the boot watchdog..."); commit `5b9f7ed`.

Done and verified	Evidence
The Portal was then served from `https://internal-platform.rf.gd/CG/`; since 2026-10-19 it is `https://folio-portal.page.gd/` (owner).	Both answer 200.

1.2 The order, at a glance

```
Stage A  Prove the deployed Workspace, one role at a time ......... owner (A1–A2)
Stage B  Prove each mail and intake path once, with a test record .. owner (B1–B4)
Stage C  Run the browser suite against the live site ............... platform team (C1)
Stage D  Retire what the new paths replace ......................... platform team (D1–D2)
Stage E  Take the go-live decisions ................................ owner (E1–E7)
Stage F  Clean the test records, record the counter, declare ........ platform team +
owner (F1–F3)
Stage G  Hypercare (first two weeks) ............................... operational owner
```

Stages A to C can run on the same day. Stage D waits for B3. Stage F waits for every test in A to D, because each test writes records that F removes. Stage E can be decided at any time before F3.

Blocks go-live below means "Folio cannot be declared live without it". **After go-live** means the item is real work but does not stop staff using Folio for incoming correspondence. The register's own category is shown for each item. Where the go-live classification is a recommendation rather than a recorded decision, it is marked *(recommended)*.

Stage A · Prove the deployed Workspace

A1 · Sign-in and data, as an administrator (done 2026-10-10)

Recorded above. Nothing to do.

A2 · Who sees what: the viewer test

Who	Owner
Why	Role scoping (ITEM-90, closed) was proven with a test account. This proves it on the deployed site with a real viewer.
Before	You need the e-mail address of an **active viewer** in `DGO_UserDirectory` you can receive mail for.
Blocks go-live	Yes *(recommended)*

Steps:

1. Open a **private window** and go to `https://folio-platform.page.gd/`.
2. Enter the viewer's address and press **Email me a code**. Enter the code from the e-mail.
3. Wait about 20 seconds, then press **Continue to Command Center** if the welcome screen shows.
4. Check:
   - the counts show only **that viewer's own tasks** (a viewer with no tasks sees zeros);
   - the menu offers only Command Center, SLA & Tracking, My Work, Insight and Lookup (the viewer row in `config/rbac.config.js`, shown as the workspaces whose tabs it opens);
   - in Lookup, other people's e-mail addresses are blank.
5. Report the result to the platform team.

Expected: as above. If the viewer sees everyone's records, stop and report it: that is a scoping defect.

A3 · User Administration: add and change one test person

Who	Owner (signed in as a System Administrator)
Why	The User Administration screen writes through the new `USER_ADMIN` service (proven by script on 2026-10-08). This proves the deployed screen.
Blocks go-live	Yes *(recommended)*

Steps:

1. Signed in as dgsregistry@nitda.gov.ng, open **Administration › User Administration**.
2. Add one person with a recognisable test name (for example `ZTEST Folio Viewer`), role **Read-only Viewer**, status **active**.
3. Change their role to **Operator** and save.
4. Disable them.
5. Tell the platform team the name and e-mail used, so F1 removes the row.

Expected: each save reports success; the row in `DGO_UserDirectory` shows the latest values. Granting **System Administrator** is refused unless you are a System Administrator yourself.

Stage B · Prove each mail and intake path once

B1 · One assigned task, end to end (ITEM-87, ITEM-50, ITEM-52)

Who	Owner
Closes	ITEM-87 (BLOCKING), ITEM-50 (CORRECTNESS), ITEM-52 (CORRECTNESS)
Blocks go-live	Yes

Steps:

1. In **Intake & Assignment**, pick one test document and assign it to an officer whose mailbox you can read (a test mailbox is fine).
2. In that mailbox, open the assignment e-mail.
3. Press **Acknowledge Receipt** in the e-mail.

Expected:

- the e-mail arrives at the **assigned officer**, and no copy goes to a fixed mailbox (ITEM-50);
- no audit or diagnostics mail is sent alongside it (ITEM-52);
- the button opens `https://folio-platform.page.gd/` on the **Acknowledgment Queue**, on that task, after sign-in (ITEM-87); **Support Access** opens the Assistant.

Record: the time, the task reference and the recipient. Send them to the platform team; they file the evidence and close the three items.

B2 · One real scan (ITEM-80)

Who	Owner (registry desk)
Closes	ITEM-80 (ARCHITECTURE), and with D1, ITEM-67
Blocks go-live	Yes *(recommended: scanning is one of the four intake sources)*

Steps:

1. Scan one real or test document at the registry station so it lands in `/E-DOCUMENT STATION`, the folder the scanner already uses (owner decision 2026-10-16).
2. Wait a few minutes. The flow checks the folder once a minute.
3. Tell the platform team the time and file name.

Expected: the new flow `Folio_Incoming_Correspondence_Scan_Processing_Flow` (197527c8) picks the file up and a record appears in Folio. The platform team confirms from the flow's run history.

B3 · The sweep's failure alert (ITEM-70)

Who	Operational owner, with the platform team
Closes	ITEM-70 (BLOCKING)
Blocks go-live	Yes (register category BLOCKING)

Steps:

1. Agree a 15-minute window with the platform team.
2. The platform team sets one value the sweep reads in `Flow Configuration` to a wrong value, so the next run of `AUTO_SCHEDULED_SWEEP` (8c19b45c) fails.
3. Confirm the alert e-mail reaches the named operator address.

4. The platform team restores the value at once and confirms the next run succeeds.

B4 · Delivery proof for sent mail (ITEM-66)

Who	Owner (or anyone holding an Exchange admin role)
Closes	ITEM-66 (CORRECTNESS)
Blocks go-live	No *(recommended: receipts are already written; this adds delivery evidence)*

Choose one:

- **(a)** Run one message trace yourself: Exchange admin centre › **Mail flow › Message trace**, sender `dgsregistry@nitda.gov.ng`, the last 7 days. Export the CSV and send it to the platform team.
- **(b)** Give the registry account the read-only **Global Reader** role, and the platform team runs the traces with `Get-MessageTraceV2`.

Message trace needs one of: Exchange Administrator, Global Reader, Security Reader or Administrator, Compliance Administrator, or the Exchange *Message Tracking* role.

Stage C · The browser suite against the live site (MANUAL-3)

Who	Platform team, with the owner's go-ahead
Closes	MANUAL-3 (MANUAL_GATE)
Blocks go-live	Yes

The suite normally starts its own local server. Pointed at the deployed site, it tests what users get:

```
DGO_BASE_URL=https://folio-platform.page.gd \
PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH=/opt/pw-browsers/chromium \
npx playwright test
```

Notes:

- Specs that need the dev server's fixed sign-in code (`000000`) start their own server and cannot run against production. The platform team records which specs ran against the live host and which ran locally, and why.
- The host shows a JavaScript challenge on the first HTTPS request; a real browser passes it.

Done 2026-10-19: 159 passed, 0 failed against folio-platform.page.gd and folio-portal.page.gd (evidence); MANUAL-3 RESOLVED.

Record: the pass and fail counts and the commit tested, in the register's MANUAL-3 entry. The register's criterion says "all ten spec files"; the suite has since grown to 17 files (192 tests on 2026-

10-10). Run all of them, and record the count.

Stage D · Retire what the new paths replace

D1 · Turn off the legacy scan flow (ITEM-80, ITEM-67)

After B2 succeeds, the platform team turns off `DGSO INCOMING AI PROCESSING` (b6d92bae). That flow is the only one still sending mail as `dgs@nitda.gov.ng`, so turning it off also closes ITEM-67 (BLOCKING): every remaining mail connection then signs in as `dgsregistry@nitda.gov.ng`.

D2 · Close the AI items on the delivered build (ITEM-88, ITEM-75)

The owner decided (2026-10-08) that the go-live Workspace carries no AI address. The deployed build (2026-10-09 21:58 UTC) carries none: `AI_CHAT`, `AI_DOC_ANALYSIS` and `AI_EMAIL_ANALYSIS` are empty in its `config/config.local.js`. ITEM-88's own resolution criterion is "the go-live build delivered without the AI addresses by decision", and ITEM-75 closes with ITEM-88. The platform team records this evidence in a register revision. In the Workspace, AI buttons say the service is unavailable and change nothing.

Stage E · Decisions for the owner

Each decision is the owner's alone. The platform team acts on the answer.

#	Item	The question	Options	Recommendation	Blocks go-live
E1	ITEM-25 (DECISION_REQUIRED)	Do staff still route activities to **SIWES** or **NYSC**, or archive activities, from Folio? These three operations still answer 501.	(a) Needed: build them. (b) Not needed: retire them from the client and close the item.	Answer per operation, from a needs assessment.	No
E2	ITEM-74 (APPLIED_UNVERIFIED)	Raising a task from an e-mail needs an e-mail source in Folio, and the go-live build has none.	(a) Build an e-mail read. (b) Defer e-mail-to-task past go-live.	(b) for go-live.	No, if (b)
E3	ITEM-91 (SPECIFIED_NOT_APPLIED)	Directors see only their own items until one task-list index is freed.	(a) Drop one of CorrelationId, AssigneeKey or RoutedOrgKey. (b) Keep the current behaviour.	(b) now; (a) when the first director is added.	No (no director in the directory today)

#	Item	The question	Options	Recommendation	Blocks go-live
E4	ITEM-71 (BLOCKED)	Notification release gates: baseline, cutover and hypercare records in the tenant-execution pack. Agency approvals are recorded as not applicable (2026-10-07).	Sign off each record.	Sign off with Stage F.	No *(recommended)*
E5	ITEM-51 (SPECIFIED_NOT_APPLIED)	Who is copied on officer notices: the primary assignee only, or supporting and copy-to parties too?	Primary only, or all parties.	Decide with E4; the sends are built after.	No
E6	ITEM-68 (READY)	Accept the e-mail template library after a person has looked at it in Outlook, Gmail, iOS Mail, dark mode, with images off and with a screen reader.	Accept, or list changes.	Accept after B1, using the real e-mails B1 produced.	No
E7	Hosting (2026-10-09)	Chrome flagged `rf.gd`; the owner moved the Workspace and Portal to folio-platform.page.gd and folio-portal.page.gd (2026-10-19). Hosting is the owner's, outside the commissioning scope.	—	Decided by the owner.	No

---

Stage F · Clean, record, declare

F1 · Remove the test records (MANUAL-2)

Who	Platform team (cleaning approved by the owner, 2026-10-08)

When	After every test in A to D
Closes	MANUAL-2 (READY)

The platform team lists every row created by commissioning (test documents, tasks, the A3 test person, test proofs and telemetry), removes them, and records each removal. It then records the **sequence counter's starting value** (`Portal Sequence Counters`), so the first real reference number is known.

F2 · Re-check the register

`npm run outstanding` must show no open item that blocks go-live. Items left open must be the ones the owner accepted as after-go-live in Stage E.

F3 · Declare go-live

The owner tells staff:

- the address, `https://folio-platform.page.gd/`;
- that they sign in with their work e-mail and the code it receives;
- whom to contact for access: a User Administrator adds people in User Administration;
- the User guide.

Stage G · Hypercare (first two weeks)

Daily, the operational owner or platform team checks:

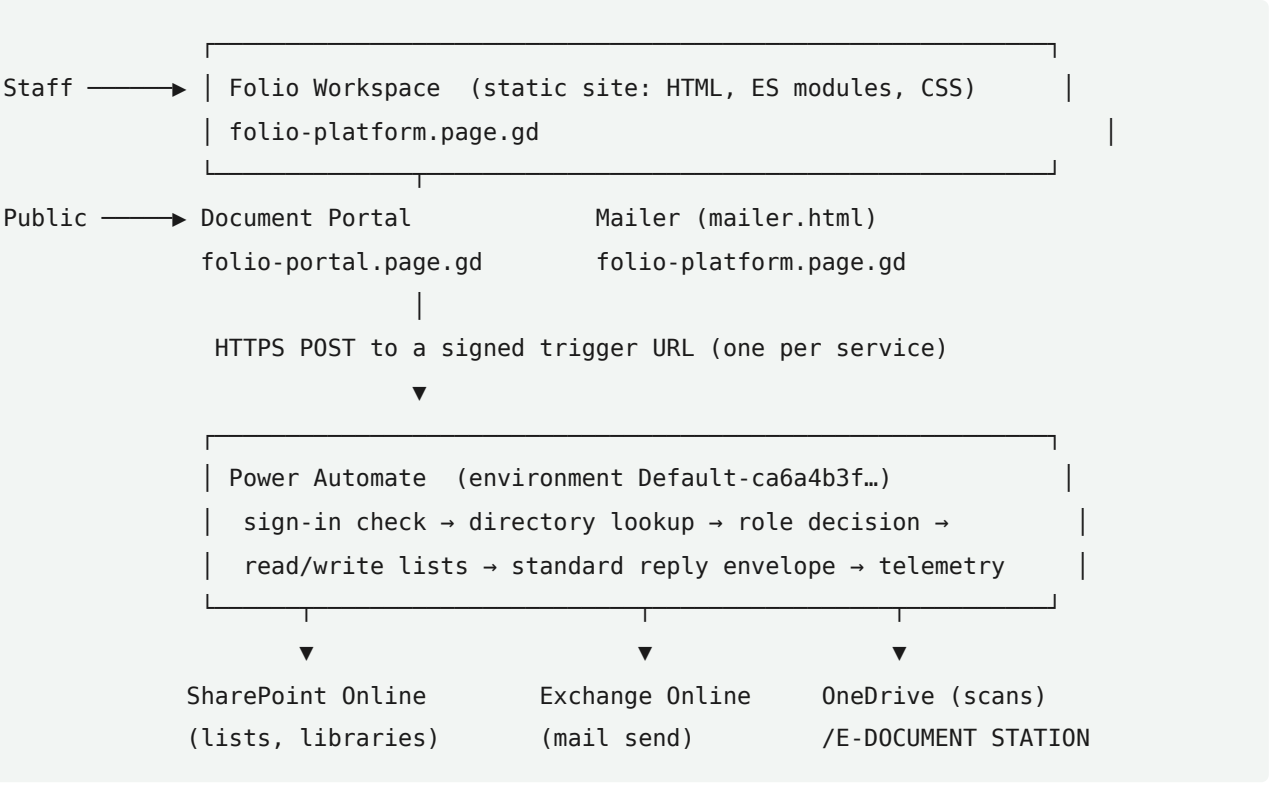
Check	Where	Act when
Failed runs	Power Automate › My flows › each Folio flow › Run history	Any `Failed` run: open it and read the failed action.
401 replies	FETCH_ALL and DYNAMIC_ACTIONS run history	A 401 on a real user's request (not the System Health probe): the person's proof expired after 4 hours or they are not active in the directory.
Sweep	`AUTO_SCHEDULED_SWEEP` run history	Not run for longer than its schedule, or failed.
Telemetry	SharePoint `Portal Flow Telemetry`	Rows with an error status.
Access requests	User Administration	New staff waiting for a role.

Rollback. The Workspace is static files. To roll back, upload the previous zip. The flows are untouched by a Workspace rollback. A flow change is rolled back by re-applying the previous released package (`npm run flows:update`), which the platform team holds in git.

2 · Architecture

2.1 The shape of the system

Folio is a set of **static browser applications** that call **Power Automate flows** directly. The flows read and write **SharePoint lists**, and send mail through **Exchange Online**. There is no application server, no database server and no API gateway of Folio's own.



Three decisions explain most of the design:

1. **The browser is only a client.** Every rule that matters (who the caller is, what their role allows, what data they may see) is decided **inside the flow**. Route guards in the browser only shape what people see; they do not protect data.
2. **A trigger URL is a credential.** Possession of a signed URL lets anyone call that flow. The flow therefore never trusts the request body for identity; it takes the caller from the sign-in proof (§2.3).
3. **Zero build, zero runtime dependencies.** The Workspace runs as written: native ES modules, no bundler, no framework. What the repository holds is what the browser runs, apart from one generated file, `config/config.local.js`, which carries the signed addresses (§2.5).

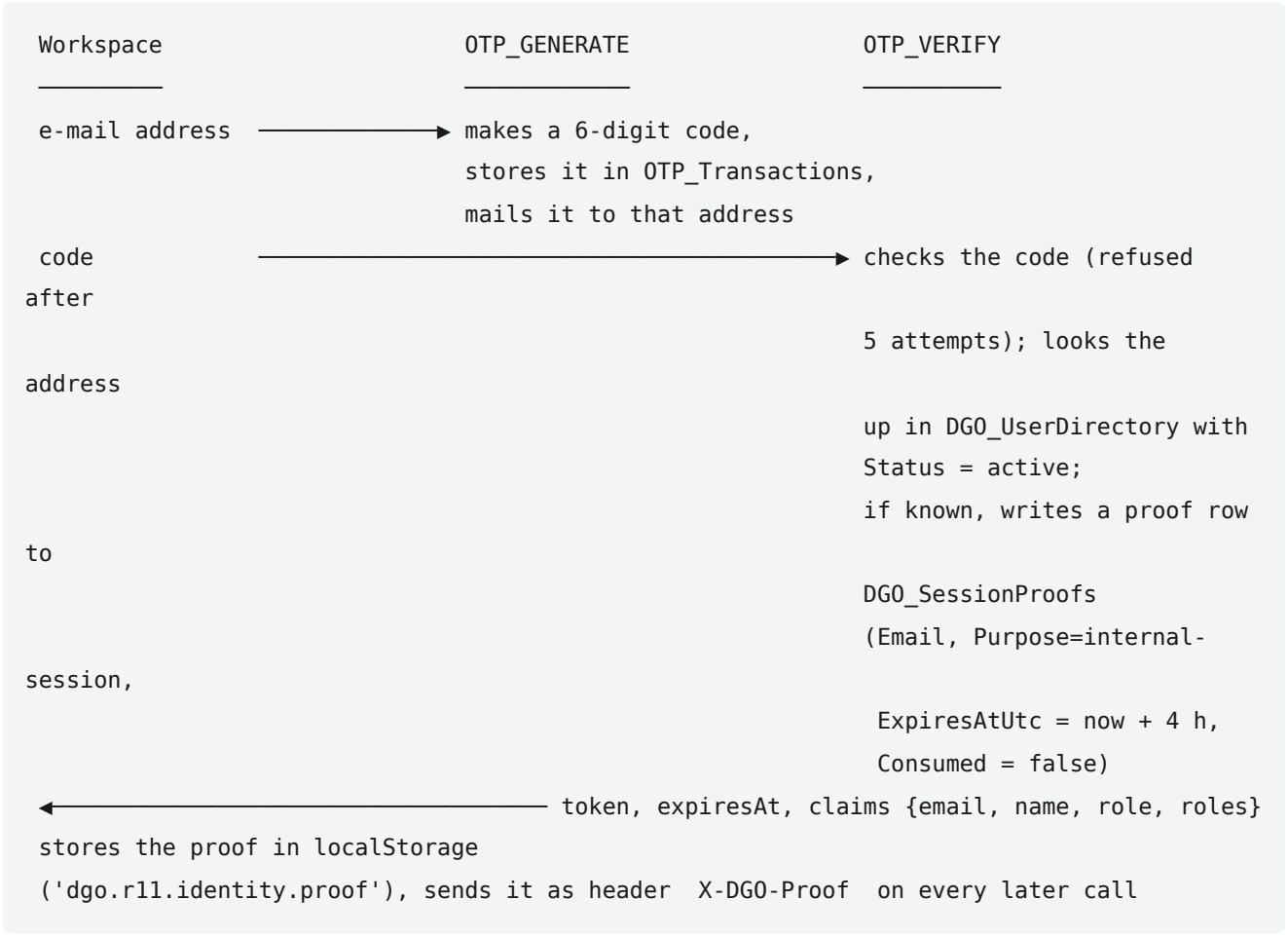
2.2 The surfaces

Surface	Users	Hosted at	Source in this repository
Folio Workspace	Staff: registry, officers, directors, executives, administrators	`https://folio-platform.page.gd/` (primary host, owner decision 2026-10-08)	repository root: `index.html`, `config/`, `core/`, `modules/`, `shared/`, `styles/`, `assets/`
NITDA Document Portal	The public: submit correspondence, track it, request support	`https://folio-portal.page.gd/` (owner, 2026-10-19)	`document-portal/`
DGO Correspondence Mailer	DGO staff sending official letters	`https://folio-platform.page.gd/mailer.html`, one file beside the Workspace (owner, 2026-10-19)	`mailer/`
Folio Estate Console	Platform team	opened from a file	`tools/admin-console.html`
Folio Flow Workbench	Platform team	local only (127.0.0.1)	`tools/flow-workbench/`
Folio Vault	Platform team	local, optional	`tools/flow-capsule/`
Folio Central Administration Centre	Platform team	local only	`tools/admin-centre/`

Each runs on its own port during development: `npm run platforms` lists them (see the Developer guide).

2.3 Identity: sign-in by e-mailed code

There is no external identity provider in the request path. Identity is a **proof** issued after a one-time code:



Every other service does the same check first (as built in FETCH_ALL, for example):

1. Read `IDENTITY_PROOF_REQUIRED` from `Flow Configuration` (it is `true` in production).
2. Take the token from `X-DGO-Proof` (or `Authorization: Bearer …`).
3. Look it up in `DGO_SessionProofs`: same token, `Consumed eq 0`, `Purpose eq 'internal-session'`, and `ExpiresAtUtc` later than now.
4. The caller is the proof row's `Email`, never anything in the request body.
5. Look the caller up in `DGO_UserDirectory` (`Status eq 'active'`) to get their **Role** and **Persona**. Not found or not active: **401**.

Facts that follow from this (read from the live flows on 2026-10-10):

- OTP_GENERATE mails a code only to an **active** `DGO_UserDirectory` member (2026-10-10). Its reply is the same for any address, so the sign-in box does not reveal who is a user. OTP_VERIFY checks the directory again before it issues a proof.
- A proof lasts **4 hours**. After that, the next call answers 401 and the Workspace asks for a new code.
- The proof lives in `localStorage` (owner, 2026-10-20), so every tab of the Workspace in the same browser shares one sign-in, and it survives a reload and a browser restart until its 4 hours run out. **Sign out** (the account menu) ends it in every tab at once; a sign-in or sign-out in one tab reaches the others through the browser's `storage` event.

- Revoking access is immediate: set the person's `Status` to anything but `active`, and their next call answers 401.
- The authentication contract, with its numbered obligations, is in `docs/architecture/AUTHENTICATION_CONTRACT.md`.

2.4 Authorisation: roles, personas and data scope

Three layers, decided in this order:

Layer	Decided by	Effect
Active directory entry	the flow, from `DGO_UserDirectory.Status`	Not active: 401 for everything.
Data scope	the flow (FETCH_ALL `Compose_FetchAll_Scope`)	**all**: systemAdmin, userAdmin, executive, operator, or persona admin, executive or registry. **directorate**: director (today their own items only, ITEM-91). **own**: everyone else, who receives only their own tasks, those tasks' documents and comments, and only their own e-mail address in the directory.
Screens	the browser (`config/rbac.config.js` `RoleRouteAccess`)	Which menu entries a role sees. This is presentation; the flow has already decided the data.

The six roles, their personas and permissions are listed in the Data model.

2.5 Configuration and delivery

What	Where	Who changes it
Signed service addresses	`config/config.local.js` in the deployed site (generated, never committed)	`npm run config:rebuild`, which reads them live from the tenant
Digest of what was last delivered	`docs/folio-Master-Flows-and-Endpoints/registers/config-build-ledger.json` (digests only, no URL)	written by `config:rebuild`; compared by `config:drift`
Sign-in enforcement	`window.DGO_CONFIG.auth = { enabled: true, roleSource: "verified" }` in `config.local.js`	the build
Browser caching	`.htaccess` at the site root: `no-cache` for HTML, JS, CSS and JSON	the build (`scripts/package.mjs` includes it)

What	Where	Who changes it
Server-side switches	SharePoint `Flow Configuration` list (`Title` = key, `ConfigValue` = value), for example `IDENTITY_PROOF_REQUIRED` , `ALLOWED_ORIGIN…` , `DATASET_…_SITE_URL` and `…_LIST_ID` , `ENVIRONMENT_CODE`	administrators with list access
Flow logic	released packages in `docs/folio-Master-Flows-and-Endpoints/packages/` , applied with `npm run flows:update`	platform team, through a release (`npm run idrelease`)

A delivered Workspace build is one zip, uploaded to the site root. Because the addresses are inside it, **the zip is private**.

2.6 The standard reply envelope

Every Folio service answers in the same shape:

```
{
  "ok": true,
  "status":  { "http": 200, "code": "OK", "message": "Success" },
  "request": { "requestId": "…", "trackingId": "", "action": "fetchAll", "operation":
"read", "source": "…" },
  "timing":  { "receivedAtUtc": "…", "completedAtUtc": "…", "durationMs": 0 },
  "data":    { },
  "errors":  [ ],
  "meta":    { "ts": "…", "runId": "…", "flowName": "…", "contractVersion": "2026-08-21.1"
}
}
```

`core/contracts.js` unwraps it on the client. A refusal keeps the shape, with `ok: false`, an HTTP status of 401, 403, 404, 409, 429 or 500, and the reason in `errors` .

2.7 Observability

Signal	Where
Every run of every flow	Power Automate run history (kept 28 days)
A record of each run, with secrets masked	SharePoint `Portal Flow Telemetry` , written by each flow's `Scope_Flow_Data_Capture`
Mail sent by a flow	`Portal Outbox Receipts`
Governed changes	`DGO_AuditLog` , plus audit columns on the records themselves

Signal	Where
In the browser	**System Health** (`#/diagnostics`): runtime checks, collections, the endpoint table and probes

2.8 Limits that shape the design

Limit	Value	Consequence
Actions in one flow	500	`USER_ADMIN` became its own flow when DYNAMIC_ACTIONS would have reached 524. `flows:update` refuses a package over the limit before writing.
Characters in one expression	8,192	`flows:update` refuses a package with a longer expression.
Indexed columns per list	20	Global Tracking Queue is full, which is why directors' directorate view waits on ITEM-91.
Items read per call	FETCH_ALL reads the newest 500 documents and 500 tasks	Older records are reached through Lookup.
Proof lifetime	4 hours	People sign in again after 4 hours.
FETCH_ALL reply	about 6.4 MB, 15 to 25 seconds, for an "all" caller	The first screen fills about 20 seconds after sign-in.

2.9 Related records

- Estate description (generated): `docs/PLATFORM.md`
- Target architecture: `docs/architecture/TARGET_ARCHITECTURE.md`
- Access specification: `docs/architecture/PLATFORM_ACCESS_SPEC.md`
- Every flow and service record: `docs/folio-Master-Flows-and-Endpoints/`

3 · Components

3.1 Folio Workspace: how the code is layered

```
index.html                entry page: boot screen, 15-second start-up watchdog (pauses
for sign-in)
.htaccess                 browser caching rules for the deployed site
config/                   configuration as ES modules: routes, roles, service contracts,
policies
  config.local.js         generated per deployment: signed addresses + auth switch (never
committed)
core/                     the runtime: state, sign-in, data access, domain rules, audit
shared/                   the shell (navigation, header, toasts), dialogs and shared UI
runtimes
modules/                  one file per screen (route id = file name)
styles/  assets/          design system CSS, fonts, icons, brand images
```

Rules the code follows (conventions, read from the code; no single check enforces them all):

- A **module** draws one screen and calls **core** services. It does not call `fetch` for business data itself; everything goes through `core/data-client.js`. The two exceptions are the Admin Suite's request composer and the Endpoint Console's probe, which call a service without a sign-in proof on purpose (§3.4).
- **config/** holds data, not behaviour. A rule that changes per role, status or category lives there.
- **core/boot.js** is the composition root: it is the only core file that imports the screens (to register their routes). Other core files do not import from **modules/**.

Drawn diagrams of these layers, and of trust zones and the lifecycle, are in `docs/architecture/components.html` and `docs/architecture/architecture.html`.

3.2 Core services

File	Responsibility
`core/boot.js`	Start-up: sign-in first (when enforced), register routes, draw the shell, resolve the deep link, then start the data load (`loadRuntimeData`).
`core/router.js`	Hash routes (`#/home` , `#/lookup` , …); asks `config/rbac.config.js` whether the role may open a route.
`core/state.js`	The single client state object. Persists to `localStorage` (`dgo.r11.viewport.runtime.state`), except `runtime` , which is never persisted. Signed addresses are never persisted.

File	Responsibility
`core/auth.js`	The sign-in gate for every governed call: token cache, `ensureAuthenticated`, `authHeaders`, the verified role.
`core/otp-identity.js`	The e-mailed-code provider: request a code, submit it, hold the proof in `localStorage` (`dgo.r11.identity.proof`), shared by every tab, follow a sign-in or sign-out made in another tab, sign out everywhere.
`config/auth.config.js`	Sign-in settings: provider `otp`, header `X-DGO-Proof`, renewal 120 s before expiry. Off by default; the deployed `config.local.js` turns it on.
`core/endpoint-registry.js`	Resolves a contract key to its address: deployment manifest, then an audited operator override, then the packaged address.
`core/data-client.js`	The only path to a service: sign-in check, address, confirmation for writes, a correlation ID, timeout and retry, the proof header, the envelope.
`core/server-writes.js`, `core/server-payloads.js`	Every write a screen records on the server (task updates, document state, minutes, approvals and decisions, briefs, meetings, projects, reminders, new records, registry files, audit events), built for the running Dynamic Actions package.
`core/fetch-manager.js`	De-duplicates identical requests and applies the read cache.
`core/cache-manager.js`	In-memory read cache (FETCH_ALL: 10 minutes). Cleared on reload.
`core/data-loader.js`	`loadRuntimeData()`: calls FETCH_ALL (falls back to FETCH_ACTIVITIES), maps the reply into state, records `runtime.lastLoad`. `requestSync()` is behind every refresh button.
`core/contracts.js`	Unwraps the standard reply envelope.
`core/flow-confirmation.js`	Shows a confirmation before any write or operation call.
`core/domain.js`, `core/entity-store.js`	Business rules: status from `Progress`, normalising documents and tasks, directorate filtering for display.
`core/audit-log.js`	In-browser audit trail of governed actions (sign-in events, loads, writes).
`core/pending-queue.js`, `core/offline-action-queue.js`	Writes that failed for want of a connection, kept for retry. A write the server refused (4xx) is shown, not kept, and an entry pushed out of the 250-entry queue is recorded in the audit trail.
`core/deeplink-resolver.js`	Opens the right screen and record from an e-mailed link (`?DGOPID=…`).
`core/scan-intake-service.js`	Lodging a scanned document through SCAN_INTAKE.
`shared/shell.js`	The frame: navigation by role, header buttons (search, help, ⟳ refresh, notifications, profile), toasts.

File	Responsibility
`shared/sign-in-dialog.js`	The "Sign in to continue" dialog.

3.3 Screens (31 routes, 13 workspaces)

The menu holds 13 workspaces, one per stage of the work (`config/workflow-clarity.config.js` , module rationalisation Phase 2, 2026-10-20). Each workspace is a set of tabs, and each tab is one route: every route keeps its own address, role rule and action owner. A role sees a workspace when it may open at least one of its tabs; the menu entry opens the first.

Group	Workspace	Tabs (route)	Purpose
START HERE	Command Center	Overview (`home`) · ERP–ECM Charter (`ecm-erp-charter`)	What needs attention now; the ERP–ECM boundary as reference.
OPERATIONS	Intake	To triage (`correspondence`) · Counter deposit (`scan-intake`) · Routed (`activities`) · Assignment Desk (`single-assignment`) · Bulk assignment (`bulk-assignment`)	Every incoming record in one list: triage, decide, assign, or route to SIWES, NYSC or archive.
	My Work	My tasks (`orchestrator`) · Acknowledge (`acknowledgment`) · Comments (`comments`)	Acknowledge, start, update and complete assigned work (Mine, Team, Overdue, Escalated).
	Registry	File custody (`registry`)	File jackets: open, route, receive and close; every move recorded.
	Lookup	Lookup & Direct Action (`lookup`)	Find any record by reference, sender or subject, and act on it.
CONTROL	SLA & Tracking	Monitoring (`response-tracking`) · SLA risk (`fasttrack`)	Responses and ageing; work at risk of missing its target.
	Decisions	Approvals (`approvals`) · DG/CEO (`executive`) · Briefs (`briefs`)	Everything waiting for a decision.
	Meetings	Meetings (`meetings`)	Request and record meetings; agreed actions are recorded for their owners.
	Projects	Projects (`projects`)	Projects and the measures they are tracked against.
	Insight	Statistics (`statistics`) · Reports (`reports`)	Trends, and the formal M1–M10 reports from one report engine.
CLOSURE	Outbound	Send by email (`correspondence-email`) · Record a dispatch (`dispatch`) · Close & archive (`archive`)	Outward correspondence, dispatch records and closure with evidence.

Group	Workspace	Tabs (route)	Purpose
SYSTEM	Admin Suite	Suite (`admin-suite`) · Administration (`settings`) · Health (`diagnostics`) · Runtime (`operator-hud`) · Endpoints (`endpoint-console`) · Users (`user-admin`)	Monitoring, control and management.
	Assistant	Assistant (`assistant`)	Guidance on the operating model.

What each screen records on the server, and how, is in `core/server-writes.js`: every write is built in the shape the running Dynamic Actions package accepts, and `tests/server-writes.test.mjs` runs each through the flow simulator.

Each route's code is `modules/<route>.js`. Who may open which route is in the Data model.

3.4 The two deliberate exceptions to "everything through the data client"

Screen	What it sends	Why
Endpoint Console probe (`modules/endpoint-console.js`)	`{"action":"<contract action>"}` to every read service, **with no sign-in proof**	It tests that each address is reachable and answers JSON. With sign-in enforced, the expected answer is **401**. A 401 there means the service is alive and protecting itself, not that it is broken.
Admin Suite request composer (`modules/admin-suite.js`)	A body an administrator writes, checked against the trigger schema	For administrators testing a service by hand. Also carries no proof.

3.5 NITDA Document Portal (`document-portal/`)

A public progressive web app (service worker, manifest, offline shell), deployed under `/CG/`.

Page	Purpose
`index.html`	Landing page
`submit.html`	Submit correspondence with attachments (e-mail verification first)
`track.html`	Track a submission by reference and e-mail
`support.html`	Raise a support case
`404.html`	Not found

Its seven services: `SUBMISSION`, `UPLOAD`, `STATUS`, `SUPPORT`, `VERIFY`, `VERIFY_CONFIRM` and `WRITEBACK`. The first six are public and rate-limited per IP address (20, 60, 30, 10, 5 and 5 per hour respectively). `WRITEBACK` is how Folio sets the status a submitter sees (20 per hour). Its records live in the `Portal …` lists (Data model §4.3).

3.6 DGO Correspondence Mailer (`mailer/`)

A single-page app that sends one personalised official letter per addressee, each with its own signed attachment, and logs every send. Deployed at `/mailer.html`. Its seven services: `MAIL_SEND`, `MAIL_LOG`, `MAIL_DIRECTORY`, `MAIL_TEMPLATES`, `MAIL_PROVISION`, `MAIL_AUTH` and `MAIL_DATA`. **Outbound letters are sent only through the Mailer**: Folio's Dispatch screen records that a dispatch happened; it does not send mail itself (owner decision on ITEM-25, 2026-10-08). See `mailer/README.md`.

3.7 Platform-team tools

Tool	What it is for	Start
Folio Estate Console (`tools/admin-console.html`)	One-file console over every service address and the flow catalogue	`npm run start:console`
Folio Flow Workbench (`tools/flow-workbench/`)	Inspect, compose, validate and send requests to catalogued flows	`npm run start:workbench` (Python 3.7+, binds 127.0.0.1)
Folio Vault (`tools/flow-capsule/`)	SQLite registry holding each signed URL behind an alias it never returns	`npm run start:capsule` (Python 3.11+)
Folio Central Administration Centre (`tools/admin-centre/`)	Monitors every platform; alerts, incidents, changes, a hash-chained audit trail	`npm run start:centre` (Python 3.9+, loopback only); see `docs/admin-centre/`
Platform Hub	The front door listing every platform and whether it is up	`node scripts/run-platform.mjs hub`

3.8 Power Automate flows

The Workspace's 15 delivered addresses resolve to 13 flows. The table is taken from the build ledger (`registers/config-build-ledger.json`), which is written from a live read of the tenant.

Contract key	Flow	Reads or writes
`FETCH_ALL`	IP_FETCH_ALL_BOOTSTRAP	Reads the Workspace's data, scoped by role
`FETCH_ACTIVITIES`, `GET_DOCS`	IP_Get_Docs_Endpoint	Reads documents (FETCH_ALL's fallback)
`REFERENCE_DATA`	IP_Reference_Data_Endpoint	Reads categories, departments, lookups
`FETCH_EMAIL_ATTACHMENTS`	IP_Retrieve_Email_Attachment_Endpoint	Reads attachments
`SINGLE_ASSIGNMENT`	Single_Task_Assignment	Creates a task for one assignee
`BULK_ASSIGNMENT`	BULK_OPS_DATA_RETRIEVAL_HTTP	Creates tasks in bulk

Contract key	Flow	Reads or writes
`DYNAMIC_ACTIONS`, `SUBSIDIARY_ACTIONS`	IP_Dynamic_Global_Actions_Endpoint	Task and record operations (§3.9)
`USER_ADMIN`	IP_User_Admin_Endpoint	Adds and changes people in the directory
`EMAIL`	IP_SEND_EMAIL	Sends an e-mail from the Workspace
`EMAIL_RELATED_TASK`	IP_Create__Task__for_received_Email_Endpoint	Creates a task from an e-mail
`OTP_GENERATE`	IP_OTP_GENERATE	Mails a sign-in code
`OTP_VERIFY`	IP_OTP_VERIFY	Exchanges a code for a proof
`SCAN_INTAKE`	IP_SCAN_INTAKE	Lodges a scanned document from the registry desk

Contract keys the client knows but the go-live build leaves empty: `AI_CHAT`, `AI_DOC_ANALYSIS` and `AI_EMAIL_ANALYSIS` (ITEM-88), and `DISPATCH_OUTBOUND` and `ARCHIVE_REFERENCE`, which are served as operations of DYNAMIC_ACTIONS instead.

Scheduled and event flows with no address: `Folio_Incoming_Correspondence_Scan_Processing_Flow` (watches `/E-DOCUMENT STATION`, every minute), `AUTO_SCHEDULED_SWEEP`, the Portal daily sweep, and the GOV provisioning flows.

Every flow record (definitions, packages, captures, evidence) is in `docs/folio-Master-Flows-and-Endpoints/`. A per-flow description, including FETCH_ALL action by action, is in its `provisioning/flows/` folder.

3.9 DYNAMIC_ACTIONS operations

The operations the released package recognises (`createArray(…)` in its operation switch): `update`, `update-task`, `transitionstatus`, `logauditevent`, `register`, `upsert-record`, `open-file`, `move-file`, `link`, `void`, `reparent` and `merge`.

Mapped onto those: **dispatch** and **dispatch-outbound** set the document's Status to *Treated*; **archive-reference** sets it to *Treated* and its IdState to *ARCHIVED* (ITEM-25, proven live 2026-10-08).

Still answering **501** (ITEM-25, decision E1 in the walkthrough): `activity-archive`, `activity-siwes` and `activity-nysc`.

4 · Data model

All business data lives in **SharePoint Online lists** on four sites. The browser holds a working copy for the session only. This chapter names every list Folio's adopted flows touch (25, read from 74 flow definitions by `scripts/flow-list-sweep.mjs` into `docs/folio-Master-Flows-and-Endpoints/registers/flow-list-map.json`), what each is for, and the columns that matter.

> **Column names are internal names.** SharePoint shows display names in the browser, but flows read and write **internal names** (for example `CC_x0027_dTo` for "CC'd To"). The complete, verified column lists are in `docs/deployment/RUNBOOK.md` §1.2 and §1.4. Use those when writing a flow.

4.1 Sites

Site (`https://nitdanigeria.sharepoint.com/sites/…`)	Holds
`NITDADGO-EAAACTIVITYTRACKING`	The operational records: documents, tasks, comments, categories, departments, Flow Configuration, sign-in codes
`DGO_ECM_GOVERNANCE`	People, roles, sign-in proofs, the audit log
`NEDMS`	The Portal's submissions, attachments, status timeline, upload tickets; the Central Registry
`Global_Digital_Documents_Centre`	The Portal's verification, rate limits, sequence counters, support cases; telemetry and mail receipts for all flows

4.2 Workspace lists

List	Site	What one row is	Key columns
DGO DIGITAL OPS (`1f1cb303…`)	activity tracking	A document or piece of correspondence (a *reference*)	`Title` , `RefIDD` (reference), `Category0` , `Status` , `OverallStatus` , `AssignedTo` , `RoutedToDSU` , `CC_x0027_dTo` , `StartDate` , `BodyExtract` , plus identity columns `EntityKey` , `ReferenceId` , `IdState`

List	Site	What one row is	Key columns
Global Tracking Queue (`ee82725a…`)	activity tracking	A task assigned to an officer	`Title` , `RefIDD` (the document), `Reference_ID` , `AssignedTo` (indexed), `AssignedBy` , `GDSUROUT` (the unit routed to), `DueDate` , `Priority` , `Progress` (the task's status), `TaskType` , `Acknowledge_x0020_Task` , `AcknowledgedBy` , plus `TaskRef` , `EntityKey` , `ParentKey` , `RootKey` , `IdState` , `Rev`
Task_Comments (`1932d687…`)	activity tracking	A comment on a document or task	`Title` , `RefIDD` , `Description` , `Created` , `EntityKey` , `AuthorKey`
Organizational_Categories_Matrix (`f64c8b65…`)	activity tracking	A correspondence category and its default routing	`Category` , `Subcategory` , `DSU_KEY` , `Priority` , `Timeline` , default primary and supporting units, units to inform
Organizaitonal_Departments_Information (`8aab9c4e…`)	activity tracking	A department or unit (spelling as in the tenant)	`Title` , `DSU_KEY` , `DSU_Email` , `DSU_HeadEmail` , `DSU_HeadTitle`
Flow Configuration (`9bc168c3…`)	activity tracking	One server-side setting	`Title` (the key), `ConfigValue`
OTP_Transactions (`9421d473…`)	activity tracking	A sign-in code issued to an address	`Title` (address), `OTP_Code` , `Expires_At` , `Is_Verified`
DGO_UserDirectory (`3d591f5b…`)	governance	A person who may use Folio	`Email` , `FullName` , `Directorate` , `Department` , `JobTitle` , `Role` , `Persona` , `Status` , `AccessScope` , `PilotCohort` , `DisabledReason` , `UserId`
DGO_RoleCatalogue (`f675598b…`)	governance	A role definition	`RoleId` , `Persona` , `PermissionsJson` , `AllowedRoutesJson` , `Active` , `Version`
DGO_SessionProofs (`444102d6…`)	governance	A sign-in proof	`Title` (the token), `Email` , `Purpose` (`internal-session`), `ExpiresAtUtc` , `CreatedAtUtc` , `Consumed`
DGO_AuditLog (`be0c7af1…`)	governance	A governed change	written by DYNAMIC_ACTIONS and USER_ADMIN

List	Site	What one row is	Key columns
DGO_UserRoleHistory (`9fe1872f…`)	governance	A role change	written on role assignment
NITDA_Central_Registry (`f553959f…`)	NEDMS	A registry file	`Title`

Two facts that catch people out:

- **Global Tracking Queue has no `Status` column.** A task's status is `Progress` (`core/domain.js`). A document's status is `DGO DIGITAL OPS/Status` . They are different lists and different columns.
- **Global Tracking Queue uses all 20 indexed columns SharePoint allows.** Filters that must be fast use indexed columns (`AssignedTo` for "my tasks"). Adding a filter on an unindexed column to a list this size (about 16,000 rows) gets throttled. ITEM-91 records the trade-off.

`DEV_OPS_Global_Tracking_Queue` (`22c4e38d…`) is a **test list**. Folio does not read it, and no production flow should write to it (ITEM-74).

4.3 Portal lists

List	Site	What one row is
Portal Registry	NEDMS	A public submission: reference, sender, subject, category, channel, status, submitted, updated, acknowledged and closed times
Portal Attachments	NEDMS	A file attached to a submission (declared name, stored name, size, SHA-256, link)
Portal Status Timeline	NEDMS	One status event a submitter can see (`SubmissionRef` , `AtUtc` , `Status` , `Label` , `Actor` , `Note`)
Portal Upload Tickets	NEDMS	A single-use permission to upload one file
Portal OTP Codes	Documents Centre	A Portal e-mail verification code (with attempt count)
Portal Verification Proofs	Documents Centre	A verified Portal e-mail (the Portal's equivalent of `DGO_SessionProofs`)
Portal Rate Limits	Documents Centre	A per-IP, per-service counter for a 60-minute window
Portal Sequence Counters	Documents Centre	The counter that mints reference numbers (`Prefix` , `Year` , `CurrentSequence` , `LastReferenceId` , `LockToken`)
Portal Support Cases	Documents Centre	A support request (case reference in `Title`)
Portal Audit Events	Documents Centre	A Portal audit event

4.4 Operational lists (all flows)

List	What one row is
Portal Flow Telemetry (`726c210d…`)	One flow run: flow, run ID, start, end, outcome, duration, error, and an attached run record with secrets masked
Portal Outbox Receipts (`88a81ca1…`)	One e-mail a flow sent: type, recipient, reference, time, status, attempts, last error
AI TEXT DATA LOG (`d46a2be7…`)	Text extracted by the legacy AI intake flow (retires with ITEM-80)

4.5 Identifiers

Defined by the identification system (`docs/reference/identification/IDENTIFICATION_SYSTEM.md` , catalogue version 1.2.6). The ones people meet:

Identifier	Format	On
Registry Reference	`NITDA-YYYY-NNNNN` (non-production: `ZNITDA-…`)	Correspondence
Task Reference	`TASK-YYYY-NNNNN` (`ZTASK-…`)	Tasks
Case Reference	`CASE-XXXXXX` , letters and digits without I, O, 0 or 1 (`ZCASE-…`)	Support cases
File Reference	`FILE-YYYY-NNNNN` (`ZFILE-…`)	Registry files

And the ones the system uses:

Identifier	Format	Purpose
Entity Key (`EntityKey`)	lower-case GUID v4	The permanent machine identity of every governed record. Relationships point at it; it is never shown as the reference.
Qualified Key	`<TYPE>:<entity key>` , where TYPE is one of REC, ATT, TSK, MIN, CAS, EVT, LNK, MAP, USR, ORG, BRF, MTG, PRJ, APR, REG	An entity key with its type
Identity state (`IdState`)	`ACTIVE` , `ARCHIVED` , `MERGED` , `VOID`	Whether an identifier resolves to itself (ACTIVE, ARCHIVED read-only), to another record (MERGED, see `SupersededBy`), or to nothing (VOID). It is independent of business status.
Revision (`Rev`)	positive integer	Optimistic concurrency: a write names the revision it read
Contract Key	`UPPER_SNAKE`	Names a service (`FETCH_ALL`)
Request ID / Correlation ID	GUID v4	One request; sent as `X-Correlation-Id`

All 26 identifiers, with patterns and owners, are in `identifier-catalogue.json`.

4.6 The Workspace's working copy (client state)

`core/state.js` holds one object. FETCH_ALL fills it at sign-in and on every refresh:

State key	Filled from FETCH_ALL `data.`	Holds
`activities`	`docs`	Documents (newest 500, or your own for the "own" tier)
`tracking`	`tasks`	Tasks (newest 500, or yours)
`comments`	`comments`	Comments
`users`	`users`	The directory (e-mail addresses: all for the "all" tier, only your own otherwise)
`categories`, `departments`	same names	Reference data
`portalRegister`	`register`	Public submissions (the "all" tier only)
`approvals`	`approvals`	Approvals (empty for the "own" tier)
`emails`	`emails`	Always empty in the go-live build (ITEM-74)

FETCH_ALL also returns `briefs`, `meetings`, `projects` and `files`; `core/data-loader.js` does not map those four keys into state.

Other keys (`settings`, `profile`, `pending`, `audit`, `notifications`, `dispatches`, ...) belong to the browser. Everything except `runtime` persists in `localStorage` under `dgo.r11.viewport.runtime.state`. `runtime` holds the outcome of the last load in `runtime.lastLoad` (`ok`, `key`, `counts`, `message`).

4.7 Roles, personas and statuses

From `config/rbac.config.js`. The role decides the screens and the server-side data scope.

Role	Label	Persona	Data scope (server)	Permissions
`systemAdmin`	System Administrator	admin	all	every permission
`userAdmin`	User Administrator	admin	all	user:view, create, update, disable; role:assign, role:view; audit:view
`executive`	Executive	executive	all	executive:view, executive:export, audit:view
`director`	Director / Directorate Lead	registry	directorate (own items until ITEM-91)	executive:view, route:manage, dispatch:approve, bulk:assign

Role	Label	Persona	Data scope (server)	Permissions
`operator`	Operator	registry	all	route:manage, bulk:assign
`viewer`	Read-only Viewer	general	own	none

Screens each role may open:

The menu shows each role the workspaces whose tabs it may open (Components §3.3); the screens are:

Role	Screens
systemAdmin	all
userAdmin	Command Center, Administration, User Administration, System Health, Operator HUD, Admin Suite
executive	Command Center, DGCEO Hub, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Review & Approval, Briefs, Meetings, Projects, Reports, Statistics, Lookup, Assistant, Archive Evidence
director	Command Center, Activities, Intake & Assignment, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Review & Approval, Dispatch, Registry Scan Intake, Briefs, Meetings, Projects, Reports, Statistics, Lookup, Assistant
operator	Command Center, Activities, Intake & Assignment, Tracking & Monitoring, Acknowledgment Queue, My Work, Assignment Desk, Bulk Assignment, Registry Scan Intake, Registry, Comments, Dispatch, Correspondence Email Desk, Meetings, Projects, Lookup, Assistant
viewer	Command Center, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Reports, Statistics, Lookup

Directory **statuses**: `staged`, `active`, `disabled`, `unregistered`. Only `active` can sign in or receive data.

Only a **System Administrator** may grant the System Administrator role (enforced in the USER_ADMIN flow).

4.8 Status vocabularies

What a Portal submitter sees (`config/status-vocabulary.config.js`):

Key	Label	Stage	Meaning
`received`	Received	1	Logged in the registry and queued for validation
`validation`	Validation	2	Documents are being checked for completeness
`review`	Under review	3	With the assigned unit for technical assessment
`action-required`	Action required	3	Something is needed from the submitter

Key	Label	Stage	Meaning
`approved`	Approved	4	Decision issued; outcome sent to the submitter
`declined`	Declined	4	Not approved on this submission
`withdrawn`	Withdrawn	4	Closed at the request of the submitter

Internal statuses map onto these: Pending → received; Accepted and Delegated → review; Declined → declined; Archived → approved.

Documents carry `Status` on DGO DIGITAL OPS; dispatch and archive set it to *Treated*. **Tasks** carry `Progress` on Global Tracking Queue.

5 · Processes

How work moves through Folio, end to end, and which screen, service and list each step uses. The exhaustive, evidence-graded catalogue (process hierarchy, step catalogue, rules, roles, status transitions, exceptions) is generated in `docs/process/`. This chapter is the readable view.

The identification system names eleven processes (`identifier-catalogue.json`): INTAKE, CLASSIFY, ASSIGN, ACTION, RESPOND, DISPATCH, VERIFY, SUPPORT, TRACK, ARCHIVE and GOVERN.

5.1 The correspondence lifecycle

```
 INTAKE ──▶ CLASSIFY ──▶ ASSIGN ──▶ ACKNOWLEDGE ──▶ ACTION ──▶ REVIEW ──▶ DISPATCH ──▶
ARCHIVE
 4 sources  Intake &     Intake &   Acknowledgment  My Work    Review &   Dispatch +
Archive
            Assignment   Assignment Queue / e-mail             Approval   Mailer
Evidence
```

Step 1 · Intake: four sources

Source	How it arrives	Service or flow	Lands in	State today
Public portal	A member of the public submits on the Portal (`/CG/submit.html`) after verifying their e-mail	SUBMISSION, UPLOAD, VERIFY, VERIFY_CONFIRM	Portal Registry, Portal Attachments	Live
Physical documents, scanned	(a) The registry scanner saves into `/E-DOCUMENT STATION`. (b) Registry staff lodge a scan from **Registry Scan Intake**.	(a) `Folio_Incoming_Correspondence_Scan_Processing_Flow` (every minute). (b) SCAN_INTAKE.	DGO DIGITAL OPS (and the intake register)	(a) Live: first scan processed 2026-10-16 (ITEM-80). (b) Live.
Customer-service e-mail	Mail to the monitored mailboxes	EMAIL_RELATED_TASK creates a task from an e-mail	Global Tracking Queue	**Not available at go-live**: the build has no e-mail source (ITEM-74, decision E2)
DGCEO outgoing correspondence	Raised internally	Workspace screens	DGO DIGITAL OPS	Live

Every new record receives a Registry Reference (`NITDA-YYYY-NNNNN`) and an Entity Key.

Step 2 · Classify and triage

Screen: Intake & Assignment (`#/correspondence`). **Who:** operators and directors.

Choose the category and sub-category from `Organizational_Categories_Matrix`. The category supplies the default responsible unit, the supporting units, the units to inform, the priority and the timeline. AI classification is switched off in the go-live build (ITEM-88); classification is by a person.

Step 3 · Assign

Screens: Intake & Assignment (assign in place), Assignment Desk, Bulk Assignment. **Services:** SINGLE_ASSIGNMENT, BULK_ASSIGNMENT. **Writes:** a task row in Global Tracking Queue (`AssignedTo`, `AssignedBy`, `GDSUROUT`, `DueDate`, `Priority`, `TaskRef`), linked to the document by `RefIDD`.

The assignee receives an e-mail with two buttons: **Acknowledge Receipt** (opens the Acknowledgment Queue on that task) and **Support Access** (opens the Assistant). Links open at `https://folio-platform.page.gd/` (ITEM-87, proven 2026-10-12: the Acknowledge Receipt button opened the task in the Acknowledgment Queue).

Not yet sent: notices to supporting and copy-to parties, and notices for approvals, escalations, delegations and reminders (ITEM-51, which waits on decisions E4 and E5).

Step 4 · Acknowledge

Screen: Acknowledgment Queue (`#/acknowledgment`), or the e-mail's button. **Service:** SUBSIDIARY_ACTIONS (`config/acknowledgement-flow.config.js`), which resolves to the same flow as DYNAMIC_ACTIONS. Records who acknowledged and when; the SLA clock is shown.

Step 5 · Act

Screen: My Work (`#/orchestrator`). The assignee starts, updates, comments on and completes the task. **Service:** DYNAMIC_ACTIONS (`update-task`, `transitionstatus`). A task's status is its `Progress`. Comments go to Task_Comments.

Step 6 · Review and approve

Screen: Review & Approval (`#/approvals`). Review, return, reject or approve, with escalation to the executive (DGCEO Hub, `#/executive`).

Step 7 · Dispatch

Screen: Dispatch (`#/dispatch`). **The letter itself is sent from the DGO Correspondence Mailer** (`/mailer.html`), one personalised e-mail per addressee with its signed attachment, and the Mailer logs each send. Folio's Dispatch screen **records** the dispatch: DYNAMIC_ACTIONS sets the document's `Status` to *Treated* (ITEM-25, owner decision 2026-10-08).

Step 8 · Archive

Screen: Archive Evidence (`#/archive`, reached from Dispatch). DYNAMIC_ACTIONS sets `Status` to *Treated* and `IdState` to *ARCHIVED*. The record stays readable and its references do not change.

Throughout · Track

Tracking & Monitoring (SLA ageing, responses, exports), SLA risk (items at risk), Reports, Statistics, Lookup & Direct Action (find anything by reference, sender or subject).

5.2 Telling the public submitter

A Portal submitter tracks progress on `/CG/track.html` with their reference and e-mail (STATUS service). An officer changes what they see from Folio; the WRITEBACK service writes the status and a Portal Status Timeline row. The labels they see are the governed vocabulary in the Data model §4.8.

5.3 Signing in

1. Open the Workspace. If no proof is held, the **Sign in to continue** dialog opens.
2. Enter your work e-mail and press **Email me a code** (OTP_GENERATE).
3. Enter the 6-digit code from the e-mail (OTP_VERIFY). Five wrong attempts lock that code.
4. The Workspace loads your data (about 20 seconds for a full view) and opens the Command Center.
5. The proof lasts 4 hours and ends when you close the tab. After that you sign in again.

Detail: Architecture §2.3.

5.4 Giving someone access

1. A User Administrator or System Administrator opens **Administration › User Administration**.
2. Adds the person: name, work e-mail, directorate, department, **role**, status **active** (USER_ADMIN writes `DGO_UserDirectory` and records the change).
3. The person can sign in at once.
4. To remove access, set the status to **disabled**. Their next request is refused.

Only a System Administrator can make someone a System Administrator.

5.5 Support

The public raises a case on `/CG/support.html` (SUPPORT service; case reference `CASE-XXXXXX`, stored in Portal Support Cases). Staff help is the Assistant (`#/assistant`) and the support routes in `config/support-routing.config.js`.

5.6 Scheduled work

Flow	Runs	Does
`Folio_Incoming_Correspondence_Scan_Processing_Flow`	every minute	Picks up new files in `/E-DOCUMENT STATION`
`AUTO_SCHEDULED_SWEEP`	on its schedule	Housekeeping; alerts the operator on failure (to be proven, ITEM-70)
`CG_Portal_Daily_Sweep`	on a recurrence	Portal housekeeping; its records are in the master folder's `provisioning/packages/`

5.7 Where the processes are specified in full

Need	Document
Every process, sub-process and step, with evidence	`docs/process/` (generated: `npm run process:docs`)
Status transitions	`docs/process/14-STATUS-AND-TRANSITION-CATALOGUE.md`
Who is responsible for what	`docs/process/13-ROLES-AND-RESPONSIBILITY-MATRIX.md`
Notifications and escalation	`docs/process/18-NOTIFICATION-AND-ESCALATION-CATALOGUE.md`
Exceptions and recovery	`docs/process/17-EXCEPTION-AND-RECOVERY-CATALOGUE.md`

6 · Service contracts

How the Workspace talks to its services. The map from each contract key to its flow is in Components §3.8. Everything about an individual flow (definition, released package, live captures, evidence) is in `docs/folio-Master-Flows-and-Endpoints/`.

6.1 A request

Every call is an HTTPS `POST` to the service's signed trigger URL, made by `core/data-client.js`:

```
POST https://defaultca6a4b3f…
environment.api.powerplatform.com/powerautomate/automations/direct/…/workflows/<id>/trigge
rs/manual/paths/invoke?api-version=1&sp=…&sv=1.0&sig=<signature>
Content-Type: application/json
X-Correlation-Id: <GUID>
X-DGO-Proof: <sign-in proof>          (every service except OTP_GENERATE and OTP_VERIFY)

{
  "action": "<the contract's action, e.g. fetchAll>",
  "payload": { … },
  "requestId": "<GUID>",
  "timestamp": "<ISO time>"
}
```

- OTP_GENERATE and OTP_VERIFY are called with `flatPayload`: the fields sit at the top level (`action`, `identifier`, `otp_code`), because those flows read them there.
- The browser sends a CORS preflight first. The Power Platform gateway answers it itself, with `Access-Control-Allow-Origin: *` and the requested headers allowed (accepted as ITEM-56: the proof, not the origin, is the protection).
- The reply is the standard envelope (Architecture §2.6).

6.2 The client's contracts

From `config/endpoints.config.js` and `config/fetch-policy.config.js`. *Timeout* is the client's wait before it gives up; *retry* is how many times it tries again; *cache* is how long a read is reused.

Contract key	Action	Kind	Timeout	Retry	Cache
`FETCH_ALL`	`fetchAll`	read	90 s	1	10 min
`FETCH_ACTIVITIES`	`LIST-ACTIVITIES`	read	15 s	1	5 min

Contract key	Action	Kind	Timeout	Retry	Cache
`REFERENCE_DATA`	`lookups`	read	15 s	1	5 min
`GET_DOCS`	`getDocs`	read	15 s	1	5 min
`FETCH_EMAIL_ATTACHMENTS`	`fetchEmailAttachments`	read	15 s	1	5 min
`SINGLE_ASSIGNMENT`	`singleassignment`	write	15 s	1	
`BULK_ASSIGNMENT`	`bulkassignment`	write	15 s	1	
`DYNAMIC_ACTIONS`	`dynamicGlobalAction`	write	15 s	0	
`SUBSIDIARY_ACTIONS`	`INIT`	write	15 s	1	
`USER_ADMIN`	`userAdmin`	write	15 s	1	
`EMAIL`	`dispatchEmail`	write	15 s	1	
`EMAIL_RELATED_TASK`	`emailtotaskassignment`	write	15 s	1	
`SCAN_INTAKE`		write	15 s	1	
`OTP_GENERATE`	`generate`	identity	20 s	0	
`OTP_VERIFY`	`verify`	identity	20 s	0	
`AI_CHAT`, `AI_DOC_ANALYSIS`, `AI_EMAIL_ANALYSIS`		empty in the go-live build			
`DISPATCH_OUTBOUND`, `ARCHIVE_REFERENCE`		served as DYNAMIC_ACTIONS operations			

Writes and retries. A write with retry 1 is sent again if the first attempt times out or fails. Of the write services, only DYNAMIC_ACTIONS de-duplicates a repeated request (by request hash); the others can apply a retried write twice. This was accepted as a deferred follow-up under G-04 (obligation 6).

Every write asks the person to confirm first (`core/flow-confirmation.js`), unless the screen has already asked (it marks the request `__confirmedByUI`) or the button itself is the confirmation (sign-in passes `skipConfirmation`).

6.3 Status codes a service returns

HTTP	Envelope code	Means	The Workspace does
200	`OK`	Done	Applies the result
401	`ERR401`	No valid proof, or the person is not active in the directory	Reports the failure (a refresh shows "The registry could not be reached"). It does not clear the proof by itself. An *expired* proof never gets this far: the client knows the expiry and opens the sign-in dialog before calling.
403	`ERR403`	Signed in, but the role may not do this	Shows the refusal
404	`ERR404`	Not found (for OTP_VERIFY: the code was not accepted)	Shows the message
409	`ERR409`	Conflict: duplicate person, or the record changed since you read it (`Rev`)	Asks you to reload and retry
429	`ERR429`	Too many attempts (sign-in codes, Portal rate limits)	Asks you to wait
500	`ERR500`	The flow failed (`Scope_Catch_…`)	Shows the failure; the run record is in Portal Flow Telemetry
501	`ERR501`	The operation is recognised but not built (ITEM-25)	Shows the refusal

6.4 Portal and Mailer services

Surface	Contract keys	Notes
Document Portal	`SUBMISSION` , `UPLOAD` , `STATUS` , `SUPPORT` , `VERIFY` , `VERIFY_CONFIRM` , `WRITEBACK`	Public; per-IP limits per hour: SUBMISSION 20, UPLOAD 60, STATUS 30, SUPPORT 10, VERIFY 5, VERIFY_CONFIRM 5, WRITEBACK 20
Correspondence Mailer	`MAIL_SEND` , `MAIL_LOG` , `MAIL_DIRECTORY` , `MAIL_TEMPLATES` , `MAIL_PROVISION` , `MAIL_AUTH` , `MAIL_DATA`	Its own signed addresses, delivered as `mailer-endpoints.config.local.js`

6.5 Where to look next

Need	Record
The released package for a flow	`docs/folio-Master-Flows-and-Endpoints/packages/`

Need	Record
A flow, action by action	`docs/folio-Master-Flows-and-Endpoints/provisioning/flows/`
Which flow each key resolves to, and its workflow ID	`registers/config-build-ledger.json`, `registers/endpoint-workflow-ids.json`
Which lists each flow touches	`registers/flow-list-map.json`
What each flow runs in the tenant, by fingerprint	`registers/identification-paste-ledger.json`
The authentication obligations	`docs/architecture/AUTHENTICATION_CONTRACT.md`

7 · Developer guide

> **Status:** handbook chapter, 2026-10-10. It carries steps, and owns none: flow changes are owned by `FLOWS_UPDATE.md`, builds by `PACKAGING.md`, cutover by `RUNBOOK.md`.

For anyone changing Folio's code, configuration, flow packages or records. Read Architecture and Components first.

7.1 Ground rules

The repository's working rules are in `CLAUDE.md`. Where `CONTRIBUTING.md` differs (it still describes feature branches and pull requests), `CLAUDE.md` **governs**:

1. **One branch:** all work is committed and pushed to `claude/system-remediation-gaps-ahpmsy`. No other branch, no pull requests, never a force-push. Pull with `--rebase` before you start and before you push.
2. **Closed register items stay closed** unless new, committed evidence contradicts them.
3. **Flow and endpoint records live only in** `docs/folio-Master-Flows-and-Endpoints/`. `npm run test:masterflows` fails on one anywhere else. (This handbook links to them; it holds none.)
4. **Flow packages are frozen at a release.** Change them only for a defect found in the tenant or an owner decision, and prefer a `Flow Configuration` row to a package change.
5. **Generated files are never edited by hand.** Re-run their generator (§7.7).
6. **Secrets never enter git:** signed URLs, `config.local.js`, sign-in sessions, proofs. `npm run test:secrets` checks.

7.2 Set up

Requirements: **Node 22 or later**; Python 3.9+ only for the Workbench, Vault and Administration Centre.

```
git clone https://github.com/dgoeaa/ECM_DOCS_DEV.git
cd ECM_DOCS_DEV
git checkout claude/system-remediation-gaps-ahpmsy
npm ci
```

7.3 Run it

Command	What you get
`npm run dev`	The Workspace on `http://localhost:8080/` **with a local stand-in for every service** (`scripts/dev-server.mjs`), so screens work without the tenant. Data is kept in a local file (`DGO_DEV_DATA`).
`DGO_DEV_AUTH=1 npm run dev`	The same, with **sign-in enforced**. The dev server accepts the fixed code `000000`.
`npm start`	A plain static server: the Workspace as files, with no services behind it (demo mode without `config/config.local.js`).
`npm run platforms`	Every surface and its port. Start one with `npm run start:<id>`: `runtime` 8080, `portal` 8081, `console` 8082, `mailer` 8083, `workbench` 8765, `capsule` 8787, `centre` 8844.
`npm run setup`	Scaffolds a local `config/config.local.js` (git-ignored).

More detail: `docs/deployment/LOCAL-DEV.md`.

7.4 Check it

Command	What it runs	When
`npm run test:node`	The full gate: about 175 checks (imports, hygiene, references, secrets, generated-file drift, registers, packages, simulated flow runs)	Before every push. Run it **on the branch**: one check stamps the branch name.
`npx playwright test`	The browser suite (192 tests in 17 spec files on 2026-10-10). In a cloud session set `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH=/opt/pw-browsers/chromium`.	Before pushing any change to screens, `core/` or `shared/`.
`DGO_BASE_URL=https://folio-platform.page.gd npx playwright test`	The same suite against a deployed site	Commissioning (MANUAL-3)
`npm run test:links`	Every relative link in the docs	After changing documents
`npm run outstanding`	Totals the open items in every register	Before choosing work

A gate failure names the command that fixes it. If it is a generated file, run that command; do not edit the file by hand.

7.5 Change a screen

1. The route id is the file name: `modules/<route>.js` exports `mount(el)`.
2. A new route needs: an entry in `config/routes.config.js`, the import in `core/boot.js` (the `modules` map), a place in `config/workflow-clarity.config.js` (visible group, or hidden with the

screen it is reached through), and a row in `config/rbac.config.js` `RoleRouteAccess` for each role that may open it.

3. Call services only through `core/data-client.js` (or `core/api.js`). Never call `fetch` for business data from a module.
4. Add or extend a browser test in `tests/*.spec.js`.

7.6 Change a flow

Flows are changed by **released packages**, not by hand in the designer.

```
edit the builder → build → simulate → release → check → apply → prove live → record
```

Step	Command	Notes
Edit	`scripts/build-identification-phase4.mjs` (internal flows) or `…-phase3.mjs` (Portal)	Each flow's package is built from these
Build	`npm run idphase4` / `npm run idphase3`	Writes `packages/identification-phase4/*.paste.json`
Simulate	`npm run test:idphase4` / `npm run test:idphase3`	Runs each package in a flow simulator against seeded lists
Release	`npm run idrelease -- --reason "…"`	Freezes every package's fingerprint in `registers/identification-release.json`. Say in your report which packages changed and why.
Check	`npm run flows:update`	Reads every live flow and compares it with its package. Writes nothing.
Apply	`npm run flows:update -- apply`	Backs up, writes, reads back. **Refuses** a package that would exceed **500 actions** or an expression longer than **8,192 characters**. Runs `config:drift` itself.
Prove	A live call against a test record, then remove the record	
Record	`npm run flows:update -- record <report>`	Files the report as evidence and updates `registers/identification-paste-ledger.json`

Sign-in for `flows:update` and `config:*`: set `FOLIO_MS_SESSION` to a saved sign-in file. Without one, it offers a code on a Microsoft page. Behind a proxy, set `NODE_USE_ENV_PROXY=1`. Only flows the account owns or co-owns can be read or written.

Full procedure: `docs/folio-Master-Flows-and-Endpoints/deployment/FLOWS_UPDATE.md`.

7.7 Build and deliver the Workspace

Step	Command
Has any address moved since the last delivered build?	`npm run config:drift` (reports `REBUILD SUGGESTED` if so)
Build the deployable files	`npm run config:rebuild -- --out <private folder> --mailer <published mailer.html>`
Commit the record of what was built	`git add docs/folio-Master-Flows-and-Endpoints/registers/config-build-ledger.json`

`config:rebuild` reads every signed address live, builds and verifies the packages, and writes, outside the repository (mode 0600):

- `FOLIO-WORKSPACE-v<version>-upload-to-site-root.zip`: upload its **contents** to the Workspace host's root, **including the hidden** `.htaccess`; it carries `mailer.html` when `--mailer` names it;
- `FOLIO-PORTAL-v<version>-upload-to-portal-site-root.zip`: the Portal with its files at the site root (`folio-portal.page.gd`); `FOLIO-PORTAL-v<version>-upload-to-site-root.zip` is the same Portal under `CG/`;
- `FOLIO-MAILER-v<version>-endpoints.config.local.js`: the Mailer's addresses.

The zips hold signed addresses: hand them over privately and delete them once deployed.

Every package is identified (`scripts/lib/package-identity.mjs`). One rebuild is one version, `YYYY.MM.DD-N` (the build date and its number that day). It appears in:

Where	What it shows
The zip's file name	`FOLIO-WORKSPACE-v2026.10.20-1-…zip`
`BUILD-ID.json` inside each site	version, package, commit, build time, `codeFingerprint` (the files without their config: the same code gives the same value in every build), `configDigest` (the config file hashed: two builds of the same code with different addresses differ here), and for the Workspace the Mailer's digest
The deployed site	Workspace: the account menu reads **Build v… · commit**; Portal: the foot of every page reads **Build v…**
`registers/config-build-ledger.json`	`version`, and per package its file name, fingerprints and file count

To check what is deployed, compare the build the site shows with the ledger, or open `/BUILD-ID.json` on the site.

7.8 Add a contract key (a new service)

This touches several records. The 2026-10-08 USER_ADMIN addition needed all of these:

1. `config/endpoints.config.js`: the contract (action, method, write, timeout).
2. The flow's package, built and simulated (§7.6), and the flow created and applied.
3. `docs/folio-Master-Flows-and-Endpoints/registers/internal-flow-register.json`: a row, and a changelog entry.
4. `docs/reference/identification/identifier-catalogue.json` `contractKeyProcess`: the process it belongs to, with a version bump and a "what changed" note in the specification.
5. `scripts/dev/endpoints.mjs`: a stand-in for the dev server, so the browser suite can call it.
6. The notification matrix: its failure-alert row.
7. Counts pinned by tests (for example the number of contract keys) and their reasons.
8. A new build (§7.7): `config:drift` reports the new key until it is delivered.

Run `npm run test:node`; each missing record fails a named check.

7.9 Generated files

Never edit these by hand. If a pull conflicts in one, take either side and re-run its generator.

File or folder	Generator
`docs/PLATFORM.md`	`npm run platform`
`docs/process/`	`npm run process:docs`
`docs/visual/platform-data.js`	`npm run visual`
`docs/deployment/outstanding-rollup.json`	`npm run outstanding`
Identification packages	`npm run idphase3`, `npm run idphase4`
E-mail bodies in packages	`npm run emailcatalogue`
Estate Console	`npm run console`

The failing check always names the command.

7.10 Debugging a live problem

Collect evidence before changing anything:

1. **Did the request reach the flow?** Read the flow's run history (or `api.flow.microsoft.com` runs). No run means the browser never sent it, or the gateway refused it.
2. **What did the flow receive and answer?** The run's trigger outputs (headers, body) and each action's inputs and outputs, from the run record. The host `…environment.api.powerplatformusercontent.com` must be reachable.
3. **What did the browser record?** In the browser console: `(await import('/core/state.js')).State.get().runtime.lastLoad` holds the last load's outcome and message.

4. **Reproduce with the real code.** Run the deployed site's own files in Playwright, replacing only the flow calls with recorded replies, then repeat with real calls. One variable at a time.

A worked example of this method is in Troubleshooting §10.2.

8 · Administrator guide

> **Status:** handbook chapter, 2026-10-10. It carries steps, and owns none: tenant procedures are owned by `docs/deployment/RUNBOOK.md` and `PLATFORM_ACCESS_RUNBOOK.md`.

For **System Administrators** and **User Administrators** of the Workspace, and for whoever looks after its SharePoint lists, flows and hosting. Day-to-day use of the screens is in the User guide.

8.1 Who administers what

Role	Can
System Administrator (`systemAdmin`)	Everything, including granting System Administrator
User Administrator (`userAdmin`)	Add, change and disable people; assign roles (not System Administrator); view audit; open Administration, User Administration, System Health, Operator HUD and Admin Suite
Flow owner (dgsregistry@nitda.gov.ng)	Owns the Power Automate flows and their connections; needed to read run history or apply packages
SharePoint site owner	Edits lists directly (for example `Flow Configuration`)

8.2 People and access

Add a person

1. **Administration › User Administration › add**.
2. Fill in at least: full name, work e-mail (it must be the address they receive mail at), directorate, department, **role** (see the table below), status **active**.
3. Save. They can sign in at once with their e-mail and a code.

If the person…	Give them
registers, classifies and assigns incoming correspondence	**Operator**
leads a directorate and approves or dispatches	**Director**
reviews and decides at executive level	**Executive**
only needs to see their own assigned work	**Read-only Viewer**
manages people	**User Administrator**

If the person…	Give them
administers the whole platform	**System Administrator** (only another System Administrator can grant it)

What each role can open is in Data model §4.7. What each role **receives** is decided by the server: Operators, Executives and administrators get every record; others get their own tasks only.

Change or remove access

- Change the role, or set the status to **disabled** and give a reason. A disabled person's next request is refused (401). Their open sign-in proof stops working at that request.
- Statuses: `staged` (prepared, not yet allowed), `active`, `disabled`, `unregistered`. Only `active` works.
- Every change made through the screen is written to `DGO_AuditLog` by the USER_ADMIN service. (The USER_ADMIN package does not write `DGO_UserRoleHistory`.)

The form's fields: full name, e-mail, phone, job title, directorate, department, unit, role, persona, access scope, pilot cohort, status and, when disabling, a reason.

Direct list edits

`DGO_UserDirectory` can be edited in SharePoint, but the USER_ADMIN service applies the checks: valid role and status values, no duplicate e-mail, and only a System Administrator granting System Administrator. A direct edit skips them and writes no audit row. Prefer the screen.

8.3 Server-side settings: `Flow Configuration`

The SharePoint list `Flow Configuration` (activity-tracking site) holds settings the flows read at run time. `Title` is the key, `ConfigValue` the value. Change it to change behaviour without changing a flow.

Key	Read by	Effect
`IDENTITY_PROOF_REQUIRED`	every internal service (11 packages)	`true`: a valid sign-in proof is required. **Keep it `true` in production.**
`ALLOWED_ORIGIN…` (any key starting so)	17 packages	Origins the flows name in their CORS reply (the gateway answers `*` regardless; see ITEM-56)
`NOTIFY_…`, `NOTIFY_REGISTRY_TO`	notification sends	Who receives registry notices
`DATASET_<NAME>_SITE_URL`, `DATASET_<NAME>_LIST_ID`	FETCH_ALL and others	Where the intake register, briefs, meetings, projects, approvals and registry files live
`ENVIRONMENT_CODE`	identification	`prd`, `uat` or `dev` (non-production references start with Z)

Key	Read by	Effect
`STATUS_REQUIRE_VERIFICATION`	Portal STATUS	Whether tracking a submission needs a verified e-mail
`OPENROUTER_API_KEY`	AI document processing	The AI provider key (AI is off at go-live). Restrict who can read the list.

Change one value at a time, note the old value, and check the next run of the flows that read it.

8.4 Monitoring

Daily check	Where	Healthy
Failed runs	Power Automate › My flows › flow › Run history (kept 28 days)	No `Failed` run, or each one explained
Mail actually sent	SharePoint `Portal Outbox Receipts`	A receipt for each notice
Run records	SharePoint `Portal Flow Telemetry`	No error outcomes
Scans picked up	Run history of `Folio_Incoming_Correspondence_Scan_Processing_Flow`	Runs each minute; a file → a record
Sweep	Run history of `AUTO_SCHEDULED_SWEEP`	Runs on schedule

In the Workspace (System Administrator):

- **System Health** (`#/diagnostics`): runtime checks (booted, routes, services configured, last data load), collections, the endpoint table and readiness panels.
- **Endpoint Console** (via Administration): a **probe** sends each read service a request **without a sign-in proof**. With sign-in enforced, **401 is the healthy answer**: the service is up and refusing anonymous calls. A non-JSON answer means the service was not reached at all.
- **Operator HUD**: integration health at a glance.
- **Admin Suite**: the service estate, flow catalogue, live checks, commissioning and audit trail.

8.5 Deploying a new Workspace build

You receive `FOLIO-WORKSPACE-v<version>-upload-to-site-root.zip` privately from the platform team, for example `FOLIO-WORKSPACE-v2026.10.20-1-upload-to-site-root.zip`. The version in its name is the build's identity: use the newest you were sent, and never mix files from two versions.

1. Unzip it on your computer.
2. Upload **all** its contents to the root of `folio-platform.page.gd`, replacing what is there. Make sure the hidden file `.htaccess` is uploaded; some file managers hide files starting with a dot.

3. In a private window, open the site, sign in, and wait about 20 seconds: the Command Center shows counts.
4. Open your profile (top right). It reads **Build v...**: check it is the version in the zip's name. If it shows an older version, the upload did not replace every file.
5. Tell the platform team which version is deployed, so they delete the private copy.

Because of `.htaccess`, browsers fetch the new files on their next load. Users do not need to clear anything. (Before 2026-10-09 the host kept JavaScript for 30 days, and a browser holding old files could sign in but never load data. If someone still sees that, clearing the site's data once fixes it.)

The Portal zip goes to the Portal host (`CG/` folder) and the Mailer file to the Mailer, the same way.

Rollback: upload the previous zip. The flows are not affected.

8.6 Security operations

Task	How
A signed address leaked	Regenerate that flow's trigger key in Power Automate. The platform team runs `npm run config:drift` (it reports the change) and `npm run config:rebuild`; you deploy the new zip. The old address stops working at once. See `docs/architecture/ROTATION_REGISTER.md`.
Remove someone's access now	Set their status to `disabled`.
A proof must stop working before its 4 hours	Set `Consumed` to yes on their row in `DGO_SessionProofs`, or disable the person.
Handling build zips	They contain signed addresses. Send them privately, delete them after deployment, and never commit them.
Hosting	The Workspace and Portal are on folio-platform.page.gd and folio-portal.page.gd. The host's browser check runs once every 6 hours per browser and drops a `#/` part of a link on that first visit; Folio's e-mails link to `/acknowledge.html` and query addresses (`?TaskID=`), which survive it.

8.7 Test data

Commissioning creates test rows. The owner approved their removal (2026-10-08, MANUAL-2). The platform team removes them after the last commissioning test, and records the reference sequence counter's starting value (`Portal Sequence Counters`). When you create test data yourself, give it a recognisable name (for example a `ZTEST` prefix) and tell the platform team.

8.8 Related procedures

Need	Document
Every cutover step, standing procedures	`docs/deployment/RUNBOOK.md`
Access setup and checks	`docs/deployment/PLATFORM_ACCESS_RUNBOOK.md`
Portal operations	`docs/deployment/DOCUMENT_PORTAL_LIVE_OPERATIONS.md`
The Central Administration Centre	`docs/admin-centre/`
RBAC in depth	`docs/architecture/AUTH_RBAC_GUIDE.md`

9 · User guide

> **Status:** handbook chapter, 2026-10-10; navigation and recording revised 2026-10-20 (module rationalisation). It carries steps for using the Workspace, and owns none of the tenant procedures.

For everyone who uses the Folio Workspace to handle correspondence. Find your role in §9.3 and read the tasks for it.

9.1 Signing in

1. Open **https://folio-platform.page.gd/** in an up-to-date browser (Chrome, Edge, Firefox or Safari).
2. The **Sign in to continue** box opens. Type your **work e-mail address** and press **Email me a code**.
3. Open the e-mail from the registry mailbox and type the **6-digit code**.
   - A wrong code is refused. After five wrong tries, request a new code.
   - If no e-mail arrives within a few minutes, check junk mail, then ask a User Administrator to check that your address is in Folio and **active**.
4. Folio loads your work. **Allow about 20 seconds** the first time: the first screen fills when the data arrives.
5. A welcome card may appear. Press **Continue to Command Center**, or **Skip for now**.

You stay signed in for **4 hours**, in every tab of the same browser: a link from a task e-mail, or a second tab, opens without a new code, and so does the browser after it is closed and reopened. After 4 hours Folio asks for a new code. To end it sooner, open your profile (top right) and press **Sign out**: every Folio tab in that browser signs out. On a shared computer, always sign out. Folio never asks for a password.

9.2 Finding your way

- **The menu** (☰, top left) lists the workspaces your role can open, in groups: START HERE, OPERATIONS, CONTROL, CLOSURE, and SYSTEM for administrators. There are 13 workspaces, one for each stage of the work.
- **Tabs**: a workspace that does more than one job shows its tabs across the top of the screen, for example **Decisions › Approvals · DG/CEO · Briefs**. You see only the tabs your role can open.
- **The header buttons**: search, help (?), **refresh (⟳)** to fetch the latest records, more options, language, notifications (bell), and your profile.
- **The Command Center** is the start page. Its tiles count open references, unassigned items, active tasks, overdue items and dispatched items. Each "Recommended next action" card opens the workspace that deals with it.

- **Continue in** (bottom bar) offers the next workspace for what you are doing.
- **Lookup** finds any record by reference (for example `NITDA-2026-00123`), sender or subject.

Workspace	Tabs
Command Center	Overview · ERP–ECM Charter
Intake	To triage · Counter deposit · Routed · Assignment Desk · Bulk assignment
My Work	My tasks · Acknowledge · Comments
Registry	File custody
Lookup	Lookup & Direct Action
SLA & Tracking	Monitoring · SLA risk
Decisions	Approvals · DG/CEO · Briefs
Meetings	Meetings
Projects	Projects
Insight	Statistics · Reports
Outbound	Send by email · Record a dispatch · Close & archive
Admin Suite	Suite · Administration · Health · Runtime · Endpoints · Users
Assistant	Assistant

What you see depends on your role. If you need a workspace you cannot see, ask a User Administrator. Every address that worked before still opens: it is now a tab of its workspace.

9.3 Tasks by role

Registry and operators (role: Operator)

You want to	Go to	Then
See what came in	**Intake › To triage**	New records from the Portal, e-mail, the counter and the scanner appear here
Lodge a paper document	**Intake › Counter deposit**	Upload the scan; or scan to the registry scanner, which files into `/E-DOCUMENT STATION` and appears within a few minutes
Log a memo	**Intake › To triage › Log New Memo**	Folio registers it and issues its reference
Triage it	**Intake › To triage**	Set the status (Pending, In Treatment, Accepted, Declined, Archived). The status is recorded on the server; priority, category and remarks are kept with your device's copy of the record

You want to	Go to	Then
Assign it to an officer	**Intake** (assign in place), or **Intake › Assignment Desk**	Choose the officer and due date, and confirm. The officer gets an e-mail.
Assign many at once	**Intake › Bulk assignment**	Select the items, choose the officer, confirm
Route to SIWES, NYSC or archive	**Intake › Routed**	The routed records, Treated and Not Treated
Keep the files in order	**Registry › File custody**	Open a file (the server issues its file number), route it, record receipt, close it. Every move is recorded on the server
Send official e-mail	**Outbound › Send by email**	Drafts, branded templates, the sent register
Record a dispatch	**Outbound › Record a dispatch**	An e-mail dispatch is sent through the Email Desk; any other channel is recorded here. Folio marks the document as treated

Officers with assigned tasks

You want to	Go to	Then	Roles that can open it
Confirm you received a task	The task e-mail's **Acknowledge Receipt** button, or **My Work › Acknowledge**	Folio records who acknowledged and when; the deadline clock is shown	every role
Work on it	**My Work › My tasks**	It opens on your own tasks (Mine); Team, Overdue and Escalated are beside it. **Start work**, then **Mark Completed**: each is recorded on the server	Operator, Director, Executive, Read-only Viewer, System Administrator
Set a reminder	**My Work › My tasks**	The reminder is recorded on the server and e-mailed to the assignee on the day	as above
Discuss it	**My Work › Comments**	A comment is recorded on the server against the record's task, for everyone on the record. A record with no task keeps the comment on your device, and says so	Operator, Director, Executive, Read-only Viewer, System Administrator
Check deadlines	**SLA & Tracking › Monitoring**; **SLA & Tracking › SLA risk**	Items at risk of missing their target	Monitoring: every role except User Administrator; SLA risk: System Administrator

> A **Read-only Viewer** or **Executive** can update and complete the tasks assigned to them in **My Work**, but not other people's tasks (2026-10-10).

Directors (role: Director)

As officers, plus **Intake**, **Decisions › Approvals** (review, return, reject, approve, escalate), **Outbound › Record a dispatch**, **Decisions › Briefs**, **Meetings**, **Projects** and **Insight**. A director sees the tasks assigned to them and the tasks routed to their directorate (ITEM-91).

Executives (role: Executive)

Decisions › DG/CEO for correspondence decisions (each is recorded on the server before anyone is e-mailed); **Decisions › Approvals**; **Decisions › Briefs**; **Meetings**; **Projects**; **Insight**; **Outbound › Close & archive**.

Read-only viewers (role: Read-only Viewer)

Command Center, SLA & Tracking, **My Work** (My tasks, Acknowledge, Comments), Insight and Lookup. You see your own tasks and the documents they belong to.

9.4 Confirmations and messages

- Before Folio changes anything, it shows what it will do and asks you to confirm. Nothing is saved until you do.
- A green message confirms a change. Most say **recorded on the server**: the change is held by the server and every officer sees it. A message that says **on this device only** means the record has not reached the server, so the change is kept in your browser alone.
- A red message means nothing was saved: read it, then try again or ask for help. A change the server refuses is not retried; the message says why.
- **"The server could not be reached, so nothing was reloaded"** after pressing ⟳: your screen still shows the records you had. Try once more; if it persists, press **Sign out** in your profile, then sign in again.
- **"NITDA Folio could not start"**: press **Try again**. If it repeats, tell IT support the time you saw it.
- AI buttons say the service is **unavailable**: AI analysis is switched off at go-live, by decision.

9.5 The public Document Portal

Members of the public use **https://folio-portal.page.gd/** to:

- **submit** correspondence with attachments, after confirming their e-mail with a code;
- **track** a submission with its reference and their e-mail;
- **ask for support** (they receive a case reference, `CASE-…`).

What they see as the status (Received, Validation, Under review, Action required, Approved, Declined, Withdrawn) is set by staff in Folio.

9.6 Getting help

Problem	Who
No code e-mail, cannot sign in, missing a workspace	Your User Administrator
Data missing or wrong	Registry (Operator) for the record; IT support for system faults
Anything else	The **Assistant** in Folio, then IT support

When you report a fault, give the time, the screen, the reference you were working on and the exact message.

10 · Troubleshooting

Start from evidence, not from a guess. For any fault, establish in this order:

1. **Did the request reach the flow?** Look in the flow's run history. No run means the browser never sent it, or the gateway refused it before a run started.
2. **What did the flow receive and answer?** Open the run: the trigger's headers and body, each action's inputs and outputs, the Response action's status.
3. **What did the browser record?** Use the command in §10.2.
4. **Reproduce with the real code**, changing one thing at a time.

10.1 Cannot sign in

Symptom	Cause	Fix
No code e-mail	Junk folder; mistyped address	Check junk; retype the address. A code is mailed only to an active Folio user, so a mistyped or unregistered address gets no e-mail.
The code is refused	Mistyped or expired code; or the address is not **active** in `DGO_UserDirectory` (OTP_VERIFY checks the code and the directory)	Request a new code. If it is still refused, an administrator checks the person's row: `Email` exactly as typed, `Status` = `active`.
Refused after several tries	Five wrong attempts	Request a new code
Asked to sign in again	The proof lasts 4 hours, and ends when the tab closes	Sign in again

10.2 Data does not load after sign-in

Symptom: the Workspace opens but every count is 0, and ⟳ says "The registry could not be reached".

Step 1: did FETCH_ALL run? Open the run history of `IP_FETCH_ALL_BOOTSTRAP` at the time of the sign-in.

What you find	Means	Go to
No run	The browser never sent the request	Step 2
A run answering **200**	The data was sent; the browser did not take it in, or was looked at before it arrived (it takes about 20 seconds)	Wait and recheck; then Step 2

What you find	Means	Go to
A run answering **401**, body `{"action":"fetchAll"}` only, no `X-Correlation-Id`	That was the **Endpoint Console probe**, not a data load. Expected.	Ignore it
A run answering **401** with `payload`, `requestId` and `X-Correlation-Id`	A real load whose proof was missing, expired, or whose person is not active	Check the person's directory row; sign in again
A run answering **500**	The flow failed	Open the failed action

Step 2: what did the browser record? In the affected browser, sign in, wait 30 seconds, press F12, open **Console**, paste this and press Enter. It prints no token. It runs one read-only data load and reports each stage, waiting up to 2 minutes.

```
(async()=>{const o=
{booted:!!window.__DGO_BOOTED__,secure:isSecureContext,href:location.origin};const
step=async(n,f,ms=60000)=>{const t=performance.now();try{const v=await
Promise.race([f(),new Promise((_,j)=>setTimeout(()=>j(new Error('no answer after
'+ms/1000+'s')),ms))]);o[n]={ok:true,ms:Math.round(performance.now()-t),v};}catch(e){o[n]=
{ok:false,ms:Math.round(performance.now()-t),error:(e&&e.name)+': '+
(e&&e.message)};}};const {State}=await
import('/core/state.js');o.loading=!!State.get().runtime?.loading;o.lastLoad=State.get().r
untime?.lastLoad||null;await step('auth',async()=>{const a=await
import('/core/auth.js');const i=await a.ensureAuthenticated('diagnostic');return
{role:i&&i.role};});await step('url',async()=>{const d=await import('/core/data-
client.js');const u=d.resolveUrl('FETCH_ALL');return {configured:!!u,host:u?new
URL(u).host:''};});await step('request',async()=>{const {FetchManager}=await
import('/core/fetch-manager.js');const r=await FetchManager.fetch('FETCH_ALL',
{force:true,requestedAt:new Date().toISOString()},
{force:true,cacheNamespace:'FETCH_ALL'});const d=r.data?.data||r.data||{};return
{source:r.source,docs:(d.docs||[]).length,tasks:(d.tasks||
[]).length};},120000);console.log(JSON.stringify(o,null,1));return o})()
```

Output	Means
`booted: false`	Start-up stopped after drawing the screen. Clear the site's data (padlock › Site settings › Delete data) and reload.
`auth` not ok	No valid sign-in: sign in again
`url` not ok	The build has no FETCH_ALL address: redeploy the build
`request` not ok	Read the error; check FETCH_ALL's run history at that time
all ok, with document and task counts	The service works. Press ⟳.

What it was on 2026-10-09. The deployed build, flows and account all worked: a clean browser received the data. The owner's browsers held JavaScript files cached for 30 days from earlier builds (the host's default), and sent no data request after signing in. After the site's data was cleared, every load succeeded. The root `.htaccess` now makes browsers revalidate every page, script and stylesheet, so a redeploy cannot leave a browser on old files.

10.3 "NITDA Folio could not start"

The start-up watchdog in `index.html` shows this if start-up has not finished within 15 seconds. Since build `5b9f7ed` it pauses while the sign-in box waits for you, so it should only appear for a real failure. Press **Try again**. If it repeats, open **Technical details** and send the failing file names to IT support.

10.4 401s in run history that nobody caused

Bodies of exactly `{"action":"<action>"}` with only a `Content-Type` header come from the **Endpoint Console probe** (System Health › Administration). The probe carries no proof on purpose. 401 is its healthy result.

10.5 A flow change is refused

`npm run flows:update -- apply` refuses, before writing, any package that would give a flow more than **500 actions** or contain an expression longer than **8,192 characters**: the tenant would reject either. Split the work into its own flow (as USER_ADMIN was) or shorten the expression.

10.6 Limits and design notes at commissioning

Facts read from the code, configuration and live flows, checked again at commissioning on 2026-10-19. Every item the register tracks is closed (`npm run outstanding`: 0 open); what remains here are design limits and notes for operators.

Gap	Effect	Tracked
AI analysis off	AI buttons say unavailable	ITEM-88, resolved by decision
Writes are not de-duplicated by the server, except in DYNAMIC_ACTIONS	The Workspace waits 90 s for a write and never repeats one itself (2026-10-10), so it creates no duplicate; a person pressing Assign twice still could	G-04 obligation 6
FETCH_ALL reads the newest 500 documents and 500 tasks	Older records are not on the dashboards; use Lookup	Design limit

Gap	Effect	Tracked
FETCH_ALL's role-catalogue check is a constant `true`	`DGO_RoleCatalogue` is read but decides nothing; data scope comes from the role and persona	Not in the register (scope is enforced; the catalogue check is redundant)
A link opens the Command Center instead of the screen it names	The host's browser check (first visit in 6 hours) dropped the `#/` part of the link	Open the link again; links Folio sends use `/acknowledge.html` or `?TaskID=`, which survive it (release 41)
`docs/PLATFORM.md` §4 names some superseded flows (for example `IP_FETCH_ALL__EXTENDED_ENDPOINT` for FETCH_ALL)	Readers of that page get a stale picture	Regenerate from current registers; the build ledger is current

11 · Commissioning report

System	NITDA Folio: the Folio Workspace, the NITDA Document Portal, the DGO Correspondence Mailer, their Power Automate flows and the platform team's tools
Commissioned	19 October 2026; live set read and built 20 October 2026
Prepared for	Office of the Director-General/CEO, NITDA
Prepared by	Folio commissioning, for DGO Platform Engineering

1. Purpose

This report records the commissioning of NITDA Folio: what was commissioned, how each part was verified, the defects found and corrected, and the position at handover. Every statement cites the committed record that proves it. Where a statement and a register disagree, the register prevails.

2. Declaration

Within the scope in section 3, NITDA Folio is production-ready, fully commissioned, and declared ready for live operation.

Basis	Position at handover	Section
Readiness registers	**0 open, 122 closed**: 81 readiness items, 12 governance findings and 29 flow truth findings	9
Deployed applications	Browser suite against the deployed Workspace and Portal: **159 passed, 0 failed**	6
Power Automate flows	**43** Folio flows in the tenant, 42 running and 1 off by design; all **27** released packages running at release 41	7
Notifications	**38 of 38** passed, each with its run and delivery record	8
Applications and tools	**3** applications and **6** tools verified; the **4** tool defects found were corrected	5, 10
Full check	`npm run test:node`, the repository's full gate, passes on the committed state	—

Two actions remain with the owner after handover (section 13): uploading the Workspace build of 20 October 2026, which also publishes the Mailer, and switching automated staff notifications on once the owner's own testing is finished.

3. Scope

In scope	Out of scope (owner direction, 19 October 2026)
The Folio Workspace and NITDA Document Portal builds, the DGO Correspondence Mailer, the Folio Power Automate flows and their packages, Flow Configuration, the SharePoint lists Folio uses, every notification, the platform team's tools, and the registers that record them	The licence of the account the flows run under, the web hosting, and the account's flow limit

4. The system as commissioned

4.1 Applications

Application	Used by	Address	Build at handover
Folio Workspace	Staff	https://folio-platform.page.gd/	Build of 20 October 2026: 16 signed flow addresses, sign-in enforced, staff notices held. It replaces the build of 19 October 2026, from which it differs in three data files used by the administration screens and the added `mailer.html`.
NITDA Document Portal	The public	https://folio-portal.page.gd/	Build of 19 October 2026: 7 signed flow addresses. Already deployed.
DGO Correspondence Mailer	DGO staff sending official letters	https://folio-platform.page.gd/mailer.html	Version 2.1.5: one file holding its 7 flow addresses, delivered inside the Workspace build of 20 October 2026

What each delivered build holds is recorded, as digests only, in the config build ledger. `npm run config:drift` reports any address that has moved since; none had at handover.

4.2 Power Automate flows

Group	Flows	Running
Folio Workspace	15	15
Document Portal	7	7
Scheduled and event-driven	6	5
Correspondence Mailer	7	7
Governance flow registry	8	8
Total	**43**	**42**

The one flow not running is the former library intake, `DGSO INCOMING AI PROCESSING`, replaced by the scan processing flow and turned off by design on 16 October 2026. Every flow was read live from the tenant on 20 October 2026 (final live set).

4.3 The platform team's tools

The Folio Estate Console, the Folio Central Administration Centre, the Folio Flow Workbench, the Folio Vault, the Platform Hub and the provisioning console. They are not published to a website: they hold or read the estate's signed flow addresses, so they run on a platform-team computer, where those addresses are kept, by design. They are handed over as one suite, the Central Administration Centre with the Folio Vault beside it: the Centre hosts the Estate Console and the provisioning console, its Portfolio replaces the Platform Hub, and its Test a flow replaces the Flow Workbench. The suite is ready to run in the Folio Administrator Toolkit.

5. Verification of each application and tool

Application or tool	Where it runs	How it was verified
Folio Workspace	folio-platform.page.gd	Browser suite against the deployed build, signed in (section 6); every governed flow live (sections 7 and 8)
NITDA Document Portal	folio-portal.page.gd	Browser suite against the deployed build, 17 passed (section 6); the Portal's flows and notices live (sections 7 and 8)
DGO Correspondence Mailer	folio-platform.page.gd/mailer.html	`npm run test:mailer`: 392 and 78 passed (flows, package, setup and the one-file build). Its seven flows were moved to the Workspace's address and each answered its health check live on 19 October 2026 (record). The browser check at its address follows the owner's upload.
Folio Estate Console	Opened from a file	Browser specification run from a file, 7 passed (section 6); opened from its local server with no error
Folio Central Administration Centre	A platform-team computer	Started and signed in: all 14 screens open with no error. Its installation check passed all of its checks: database integrity, an unbroken audit chain, and no credential stored. Browser specification, 3 passed. On 20 October 2026, extended to watch every Folio flow in the flow catalogue and run read-only against the tenant: all 43 flows watched, 33 of 33 endpoint identities verified, no flow out of its expected state (record). It probes only the tools on its own computer; the published sites are not probed.
Folio Flow Workbench	A platform-team computer, at http://127.0.0.1:8765/	Started and driven with no error; its relay forwarded a request. One defect corrected (section 10). Its flow catalogue predates the current estate: of its 50 flows, 2 are live Folio flows.

Application or tool	Where it runs	How it was verified
Folio Vault	A platform-team computer	Run end to end against the tenant: a Mailer flow was registered after its live identity check, its address was never returned, and it was invoked through the Vault; the administration routes refused the invocation token. One defect corrected (section 10). Its own tests, 8 passed.
Platform Hub	A platform-team computer	Started: it lists all eight platforms and whether each is running, and reports one that is stopped
Provisioning console	Opened from a file	Opened: 102 packages, search works, no error

On 19 October 2026 every tool was started as an administrator starts it, opened in a browser and used (record). Each tool's own test suite is part of the full check.

6. Tests against the deployed applications

The browser suite ran signed in against https://folio-platform.page.gd and https://folio-portal.page.gd (record): **159 passed, 0 failed**, and 19 not applicable to a deployment (demonstration sign-in paths and the Portal's local-only test fixtures).

Specification	Result	Specification	Result
smoke	7 passed	containment	18 passed
admin-centre	3 passed	document-flags	10 passed
admin-console	7 passed	dual-spine	9 passed
admin-suite	12 passed	endpoint-console	2 passed
in-app-notices	4 passed	lookup-commits	10 passed
audit-remediation	29 passed	scan-intake	8 passed
closure	23 passed	portal	17 passed

The run found one defect on the deployment, corrected by release 41 (section 10).

7. Flows and packages

7.1 Releases applied during commissioning

Release	Date	What it changed	Applied and read back
38	19 Oct 2026	Telemetry mail (RN-025), task reminders (RN-035) and the directorate view (ITEM-91): 8 flows	Record

Release	Date	What it changed	Applied and read back
39	19 Oct 2026	Links and images moved to folio-platform.page.gd and folio-portal.page.gd: 21 flows	Record
40	19 Oct 2026	The New E-Scan notice goes to dgs@nitda.gov.ng, as the owner set it in the designer: 1 flow	Record
41	19 Oct 2026	Digest, reminder and bulk-failure e-mail links go to `/acknowledge.html` : 2 flows	Record

7.2 Packages

The packages are frozen at release 41 (release register). What the tenant runs is recorded by fingerprint in the paste ledger: all 27 packages are running at their released version. The Mailer's seven flows are installed from their own import packages.

7.3 Flows without a run at their current version

- **Single_Task_Assignment** assigns a record to one officer. Its last run, at 01:30 UTC on 10 October 2026, created task TASK-2026-00002, updated the document and sent the assignment notice, whose acknowledgement link was opened on 12 October 2026 (recorded with RN-009, section 8). The run is marked Failed only because its reply to the caller timed out: 2 of its 214 steps failed, the reply and the scope that holds it (record). That definition was replaced at 16:22 UTC the same day. The current package passes the offline flow simulator and is running in the tenant; it has not run since. Its first assignment after go-live is the first run of the current definition, and should be checked in the flow's run history.
- **AI_Document_Processing** runs only when AI analysis is switched on in the Workspace. It is switched off by decision (ITEM-88), so the flow is not called.
- **03, 05 and 07 - GOV** (record an execution, retire a flow, and the self-registration template) run on request and have not been needed.

7.4 The governance flow registry

`01 - GOV - Provision HTTP Flow Registry` had a defect that stopped every run before its report (section 10). It was corrected and read back on 19 October 2026, and the owner's run then validated its 7 lists, 101 fields, 7 unique keys, 12 seed rows and 7 views with 0 errors (record).

8. Notifications

All 38 notifications Folio sends have passed, each with its run and, where it sends mail, its Portal Outbox Receipts row (results). The notification matrix describes each one. SMS (RN-033) was withdrawn by the owner and is not part of Folio.

The last to pass, on 19 October 2026:

Notification	How it was proven	Record
RN-020 comment notice, RN-022 Email dispatch	Sent from the live Workspace	Record
RN-023, RN-030, RN-031, RN-032 in-app notices	Shown on the live Workspace	Record
RN-027 abandoned message reported	One sweep run: a message that failed three times was marked abandoned and reported	Record
RN-028 provisioning report	The owner's run of the governance registry flow	Record

9. Readiness registers

Every item is RESOLVED, ACCEPTED or DISCHARGED_UNTRACKABLE, each with the evidence it cites (readiness register; `npm run outstanding` totals all three registers). The last to close:

Item	Closed	Record
ITEM-91 A director receives their directorate's tasks	19 Oct 2026	Record
MANUAL-2 Test records cleared	19 Oct 2026	Record
MANUAL-3 Browser suite against the deployed build	19 Oct 2026	Record

CFG-1 and CFG-2, the deployed `config.local.js` files, are DISCHARGED_UNTRACKABLE: the owner holds them outside the repository by design. They are closed.

10. Defects found during commissioning and corrected

The final commissioning runs found four defects, and the handover work two more. Each was corrected and the correction verified. Defects found earlier are recorded with the register items they closed.

Defect	Found by	Correction	Record
A first visit through the link in a digest, reminder or bulk-failure e-mail opened the Command Center, not the acknowledgment queue: the host's browser check drops the `#/` route	Browser suite against the deployment	Release 41: the links go to `/acknowledge.html`, which opens the acknowledgment queue	Record
`01 - GOV - Provision HTTP Flow Registry` stopped before its report: a filter returned text instead of true or false	The owner's run	Filter expression corrected and read back; the owner's next run passed	Record
The Flow Workbench opened at `localhost`, where its relay refuses every call	Tools commissioning	It now opens at `127.0.0.1`	Record

Defect	Found by	Correction	Record
The Folio Vault refused the Mailer flows' contract version (`dgo-mail/2.0`)	Tools commissioning	Contract versions are checked by their own rule; the Mailer flows register	Record
The Administration Centre watched only the flows in the flow owner's own list; that list leaves out the bulk assignment flow, so it was not watched	The Centre run live, 20 Oct 2026	Each Folio flow the list leaves out is read by its id; all 43 are watched	Record
The first issue of the Administrator Toolkit turned off the Centre's health monitor, which also runs its scheduled flow sync	Review of the toolkit, 20 Oct 2026	The monitor stays on; the toolkit names the tools that run on the computer, and only those are probed	The Administrator Toolkit of 20 Oct 2026, second issue

11. Security

- The deployed Workspace enforces sign-in. Roles come from the verified sign-in, and every governed flow checks the session proof itself. Calls without a proof were refused throughout the commissioning runs.
- Power Automate's gateway allows any calling page for HTTP flows, whatever a flow sets, so access is decided by sign-in (the Workspace's session proof, the Mailer's operator sign-in), not by the calling page's address.
- No signed address, sign-in code or session proof is in the repository. The build ledger keeps digests only, and the build files are handed over privately in the Deployment Kit.

12. Test records removed

With the owner's approval, the commissioning test records were removed: Portal records NITDA-2026-00001 to 00018, and Portal Support Cases items 1 to 87 and 92 (first pass, second pass, third pass). Production numbering continues at NITDA-2026-00021 and TASK-2026-00006. Portal Outbox Receipts, the delivery audit trail, is kept.

13. Actions for the owner after handover

1. **Upload the Workspace build** of 20 October 2026 to folio-platform.page.gd, as the Deployment Kit's instructions describe. This brings the site to the commissioned build and publishes the Correspondence Mailer.
2. **Switch staff notifications on** when your testing is finished. By the owner's instruction of 19 October 2026, no automated notice reaches staff yet: each goes to the registry mailbox, marked for the officer it is meant for (record). Notices to citizens are not held. To switch staff notices on:
    1. In `config/notification-delivery.config.js` , set `StaffNoticeDelivery.recipients` to `'staff'` , rebuild with `npm run config:rebuild` , and upload the Workspace build.

2. In Flow Configuration, set `NOTIFY_SWEEP_RECIPIENTS` to `staff`.

14. Operating notes

- Every Workspace load reads the registry through FETCH_ALL. Under the test load of 18 and 19 October 2026, Power Automate held that flow back twice, and it recovered without intervention (record). If the Workspace loads slowly, that record says what to look at.
- After go-live, check the run history of Single_Task_Assignment after its first assignment (section 7.3).
- The Administration Centre, connected as the flow owner, watches every Folio flow and raises an alert for a failed run or a flow out of its expected state. It runs from the Administrator Toolkit on a platform-team computer.
- Live browser runs read FETCH_ALL at most once per run and refuse every other flow call, so they leave no record.

15. Related documents

Need	Where
The complete documentation	The Folio Handbook: architecture, components, data model, processes, service contracts, the developer, administrator and user guides, troubleshooting, and this report
Every item, open or closed	The readiness register; `npm run outstanding`
The commissioning steps and checks	The runbook; `npm run commission`
Every flow, endpoint, package and record	The master flows folder
Every e-mail Folio sends	The e-mail catalogue

Appendix A · Commissioning runbook

> **This document carries steps, and owns none.** The install route is Part 2 of this runbook, §6.0's and that remains the source for HOW a package is installed. What this adds is the ORDER, the target flow for each package, and the check after each — read from the platform owner's mapping, the endpoint register and the packages themselves, not typed here.

Four open items close along one path. This is that path, in the only order that does not create rework.

Phase	Item	What it is
0	ITEM-64	The internal proof ledger is decided separate and is not yet provisioned
1	ITEM-63	Four contract keys are wired to a different flow from the one intended
2	G-04	11 identification packages that resolve the caller from the sign-in proof
3	ITEM-2	The tenant write path has never been executed

Why this order. Phase 0 changes which list the packages name, so doing it after Phase 2 means rebuilding and re-pasting all 11 packages a second time. Phase 1 is settled: every key runs on its adopted flow. Phase 3 is not a separate task: ITEM-2 closes on the first write Phase 2's verification performs, provided the result is kept.

Phase 0 — the internal proof ledger (ITEM-64)

Decision, already taken: The internal platform and the Correspondence Gateway portal use SEPARATE proof ledgers. Taken by the platform owner on 2026-09-24, reversing an argument this repository made for sharing one list divided by a Purpose value.

Why it is first: One value. scripts/build-identification-phase4.mjs reads listId from this file and uses it the moment it is set: the proof mint (OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde) writes there and every identification package reads there. Until then they use the shared ledger and its internal-session Purpose filter, which is the arrangement this decision supersedes. A GUID cannot be invented here; a package naming a list that does not exist fails at run time. (The DGO_* generator that read this file before 2026-10-06 is retired.)

0.1 Create the list

On **https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE**, create a list named `DGO_SessionProofs`.

Internal identity already lives there — DGO_UserDirectory and DGO_RoleCatalogue are on the governance site, and a session proof is an identity artefact. The portal's ledger stays where it is, on

Global_Digital_Documents_Centre.

Column	Type	Required	What it holds
`Title`	Text	yes	the proof token itself, an opaque GUID
`Email`	Text	yes	the directory address the proof was issued to; this is what the endpoints read the caller from
`Purpose`	Text	yes	'internal-session'. Retained even with a separate list: it costs nothing and keeps the filter that makes the boundary checkable
`ExpiresAtUtc`	DateTime	yes	when the proof dies. Four hours after issue, per AUTHENTICATION_CONTRACT.md §2a
`CreatedAtUtc`	DateTime	yes	when it was issued
`Consumed`	Boolean	yes	false on issue. Setting it true is how a single proof is revoked without a key rotation

Index `Title`. Every endpoint call does one lookup filtered on Title. Unindexed, that is a full-list scan on the hot path of every authenticated request in the estate.

0.2 Tell the repository

Put the new list's GUID into `listId` in `docs/reference/internal-proof-ledger.json`, then:

```
npm run idphase4          # rebuilds the identification packages against the new list
npm run test:proofidentity
```

That last command changes behaviour at this point: while `listId` was null it permitted the shared ledger as a declared interim. Once set, a package still naming the shared ledger is a failure. If it passes, Phase 0 is done.

Phase 1 — the four migration decisions (ITEM-63) — settled

For every contract key, two questions: which flow SHOULD serve it (the platform owner's mapping) and which flow DOES (the endpoint register and the tenant reading). For 15 keys the answers matched; for 5 the mapping supplied the answer; 4 differed. The adoption ledger (`FLOW_ADOPTION.md`, 2026-10-05) settles every one: each key runs on one adopted flow, and every package below installs into it.

Contract key	The map named	Wired on 2026-09-18	Settled
`FETCH_ALL`	`ce2cbed8-d180-4160-a0c7-f552101bc412` IP_FETCH_ALL__EXTENDED_ENDPOINT	`b79e6707-df4d-4ae7-8a65-738aa45e9a73`, `be1f89bd-d7ea-4a15-87c9-2ce6fd4d0eaa`	the adopted flow is IP_FETCH_ALL_BOOTSTRAP (b79e6707-df4d-4ae7-8a65-738aa45e9a73), not the map's IP_FETCH_ALL__EXTENDED_ENDPOINT; the map's flow is retired
`BULK_ASSIGNMENT_DIRECT`	`122bb363-ddea-40c7-8578-d0d613a17bf7` Deployed Bulk Task Assignment_Create Task	`0d1e8df5-30b3-4069-8903-665b87c197eb`	RETIRED: called by nothing in Folio; bulk assignment is BULK_ASSIGNMENT (identification Phase 4 README; the owner pasted the consolidated bulk scope 2026-10-05)
`DYNAMIC_GLOBAL_ACTIONS`	`31c0aaf0-ddbb-478c-9a98-bc013265c4c7` IP_Dynamic_Global_Actions_Endpoint	`08da1783-927f-4662-a429-45e1249a989f`	the adopted flow is the one the map names; the 2026-09-27 live read (ITEM-65) has the key on it
`SCAN_INTAKE`	`12b6e690-a065-468d-b470-fcfba015f1f8` IP_SCAN_INTAKE_Endpoint_v2.1	`5504d7f9-cc96-470c-be79-217582faf415`	the adopted flow is IP_SCAN_INTAKE (5504d7f9-cc96-470c-be79-217582faf415), not the map's IP_SCAN_INTAKE_Endpoint_v2.1; the map's flow is retired

Nothing to decide here. **The trigger URL belongs to the flow that is wired**, so a package is installed into the adopted flow, which is the one the endpoint register names.

Phase 2 — paste the 11 packages (G-04)

Each package is the adopted flow's root scope with the identification changes, the caller resolution among them. Until Flow Configuration `IDENTITY_PROOF_REQUIRED` is `true` the flows answer as before; set it only after the first package below has returned a proof to a real sign-in.

> **The order below is a dependency, not a preference.** With the switch on, an endpoint refuses a caller with no valid proof, and only the first package mints one.

2.1 The route

Per `packages/identification-phase4/README.md` : open the flow, **Export** it first (that export is the rollback), then replace the named root scope with the file and save.

2.2 The flows, in order

2.2.1 `OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde` — mints the proof

Mails the code and exchanges it for the proof every package below verifies. Nothing else works with the switch on until this is in.

Package	`OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde.paste.json`
Actions in the pasted scope	51
Contract keys	`OTP_GENERATE` , `OTP_VERIFY`
Install into	**IP_OTP_GENERATE** `e5632336-641f-4b9b-bd93-99185b5eb975` (adopted)
Install into	**IP_OTP_VERIFY** `4dbe6dde-37d5-47b4-ba32-d4ae5c9f8b61` (adopted)

Check before moving on:

Request a code for a real directory address and complete it. It must return a **token**. Before this package the flow returned the literal string `optional-server-issued-token` , which the client refuses — so if sign-in still fails with a missing-token error, the paste did not take.

Then look at the ledger. A new row must be there: that address, `Purpose` `internal-session` , and an `ExpiresAtUtc` four hours out.

> **This is also ITEM-2's evidence.** It is the first tenant write in this sequence: export the flow fresh afterwards and confirm the export carries the change. See §3.2.

2.2.2 `FETCH_ALL__IP_FETCH_ALL_BOOTSTRAP__b79e6707` — verifies the proof

The boot call, and the endpoint that leaked. Prove the refusal here before going further.

Package	`FETCH_ALL__IP_FETCH_ALL_BOOTSTRAP__b79e6707.paste.json`
Actions in the pasted scope	96
Contract keys	`FETCH_ALL`
Install into	**IP_FETCH_ALL_BOOTSTRAP** `b79e6707-df4d-4ae7-8a65-738aa45e9a73` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.3 `SINGLE_ASSIGNMENT__Single_Task_Assignment__3caf9458` — verifies the proof

First write.

Package	`SINGLE_ASSIGNMENT__Single_Task_Assignment__3caf9458.paste.json`
Actions in the pasted scope	191
Contract keys	`SINGLE_ASSIGNMENT`
Install into	**Single_Task_Assignment** `3caf9458-8d95-4740-a01b-6803dad97d34` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.4 `BULK_ASSIGNMENT__BULK_OPS_DATA_RETRIEVAL_HTTP__5aab9e61` — verifies the proof

Batch of the same write.

Package	`BULK_ASSIGNMENT__BULK_OPS_DATA_RETRIEVAL_HTTP__5aab9e61.paste.json`
Actions in the pasted scope	177
Contract keys	`BULK_ASSIGNMENT`
Install into	**BULK_OPS_DATA_RETRIEVAL_HTTP** `5aab9e61-ab9f-4af1-9f2b-eaba23a1bf06` (adopted)

The mapping is the only source naming a flow for this key — no tenant reading resolved it. Confirm the trigger URL matches before importing, per Appendix B.

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.5 `DYNAMIC_ACTIONS__IP_Dynamic_Global_Actions_Endpoint__31c0aaf0` — verifies the proof

The operation switch; SUBSIDIARY_ACTIONS shares the flow.

Package	`DYNAMIC_ACTIONS__IP_Dynamic_Global_Actions_Endpoint__31c0aaf0.paste.json`
Actions in the pasted scope	480

Contract keys	`DYNAMIC_GLOBAL_ACTIONS`, `SUBSIDIARY_ACTIONS`
Install into	**IP_Dynamic_Global_Actions_Endpoint** `31c0aaf0-ddbb-478c-9a98-bc013265c4c7` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.6 `EMAIL_RELATED_TASK__IP_Create_Task_for_received_Email_Endpoint__ab1a3555` — verifies the proof

A task raised from an e-mail.

Package	`EMAIL_RELATED_TASK__IP_Create_Task_for_received_Email_Endpoint__ab1a3555.paste.json`
Actions in the pasted scope	179
Contract keys	`EMAIL_RELATED_TASK`
Install into	**IP_Create__Task__for_received_Email_Endpoint** `ab1a3555-7fce-4bfe-a193-b8b84654b0a4` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.7 `REFERENCE_DATA__IP_Reference_Data_Endpoint__9a4dea8e` — verifies the proof

The directory and pickers. Its gate was hard-coded to pass; it also serves the full directory only to a holder of user:view.

Package	`REFERENCE_DATA__IP_Reference_Data_Endpoint__9a4dea8e.paste.json`
Actions in the pasted scope	56
Contract keys	`REFERENCE_DATA`
Install into	**IP_Reference_Data_Endpoint** `9a4dea8e-15f2-42b6-904d-3655056fa3c2` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.8 `GET_DOCS__IP_Get_Docs_Endpoint__28f6e62a` — verifies the proof

Documents, and FETCH_ACTIVITIES, which the same flow serves; its gate was hard-coded to pass.

Package	`GET_DOCS__IP_Get_Docs_Endpoint__28f6e62a.paste.json`
Actions in the pasted scope	52
Contract keys	`GET_DOCS`
Install into	**IP_Get_Docs_Endpoint** `28f6e62a-fea8-437c-a61a-47aec706c4e2` (adopted)

The mapping is the only source naming a flow for this key — no tenant reading resolved it. Confirm the trigger URL matches before importing, per Appendix B.

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.9 `FETCH_EMAIL_ATTACHMENTS__IP_Retrieve_Email_Attachment_Endpoint__5bc87cf3` — verifies the proof

Had no caller check at all. The gate reads SharePoint, which this flow did not: pick the SharePoint connection (3f1943c5) when the designer asks.

Package	`FETCH_EMAIL_ATTACHMENTS__IP_Retrieve_Email_Attachment_Endpoint__5bc87cf3.paste.json`
Actions in the pasted scope	38
Contract keys	`FETCH_EMAIL_ATTACHMENTS`
Install into	**IP_Retrieve_Email_Attachment_Endpoint** `5bc87cf3-7a99-4259-b6a2-315fbc794d9b` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.10 `EMAIL__IP_SEND_EMAIL__df59a69f` — verifies the proof

Outgoing mail; its own directory check then refuses a caller with no proof.

Package	`EMAIL__IP_SEND_EMAIL__df59a69f.paste.json`
Actions in the pasted scope	73
Contract keys	`EMAIL`
Install into	**IP_SEND_EMAIL** `df59a69f-92ca-4554-ae68-6b464aa73ef7` (adopted)

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

2.2.11 `AI_DOC_ANALYSIS__AI_Document_Processing__fefa43c6` — verifies the proof

Had no caller check at all. Added, not replaced: paste it between Initialize_variable_2 and Get_item (the identification-phase4 README).

Package	`AI_DOC_ANALYSIS__AI_Document_Processing__fefa43c6.paste.json`
Actions in the pasted scope	12
Contract keys	`AI_EMAIL_ANALYSIS`, `AI_DOC_ANALYSIS`, `AI_CHAT`
Install into	**AI_Document_Processing** `fefa43c6-fb9d-4337-8b65-7fd19fa09b6d` (adopted)

The mapping is the only source naming a flow for this key — no tenant reading resolved it. Confirm the trigger URL matches before importing, per Appendix B.

Check before moving on:

With `IDENTITY_PROOF_REQUIRED` set to `true`, call the endpoint **with no** `Authorization` **header**. It must answer **401**. Then call it with the proof from 2.2.1 and confirm it answers normally.

Phase 3 — verification, and the write path (ITEM-2)

3.1 The test that decides whether any of this worked

Everything above can be green and the estate still be open, because the defect this closes is not "no check runs" — it is "the check ran against an identity the caller supplied". So the acceptance test is not that a call with no proof is refused. It is this:

> **Call** `FETCH_ALL` **with no** `Authorization` **header and a body naming an active administrator.** It must answer **401**.

A call with an EMPTY caller was already refused on 2026-09-24, before any of this. The request above is the one that still worked: the gate was correct, and the identity it checked came from the request body. That is what the ledger replaces.

Then call it again with a valid proof and confirm it answers normally. A gate that refuses everyone is not a fix.

3.2 ITEM-2 — the tenant write path

What it is: The tenant write path has never been executed. Ten items name it as a prerequisite. It is not extra work here: Phase 2 performs exactly the write it asks for, and it closes on the evidence being kept.

What closes it: One flow carries a change that was applied from outside the Power Automate designer, and a fresh export of that flow contains it. The dry run is the mandatory first pass and is not on its own sufficient: it reads and reports, and this item exists to prove that a WRITE reaches the tenant.

So on the FIRST package you install:

1. After the paste, export that flow again — fresh, from the tenant.
2. Confirm the export carries the change: its root scope holds the number of actions in that package's row in §2.2, and for the first package (`OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde`) it contains `Create_Otp_Proof` .
3. Keep that export. It is the evidence, and it is all that stands between ITEM-2 and closed.

3.3 What to record, once

One dated file holding: the new list GUID; for each package, the flow it went into and its action count; the two `FETCH_ALL` calls from §3.1 with their status codes; and the fresh export from §3.2.

That is every open criterion across all four items. Nothing else on this path is outstanding.

Appendix A · The one key the mapping does not name

The mapping covers 24 contract keys. The endpoint register wires 25. The one it does not name is `FETCH_ACTIVITIES` , and it is not a gap: the register wires it to the same workflow as `GET_DOCS` , so the flow named for `GET_DOCS` serves it too.

`EMAIL` was the other, until 2026-09-24. Two candidate packages were supplied for it and they turned out to be one flow saved twice eight minutes apart — identical across all 70 actions and both connector maps, differing only in two metadata timestamps and the display name. `IP_SEND_EMAIL` (`df59a69f`) is adopted because it is what the key is wired to; `IP_DGO_SEND_EXTERNAL_CORRESPONDENCE` (`faceec58`) is wired to nothing and is recorded as a duplicate in `docs/folio-Master-Flows-and-Endpoints/deployment/internal/evidence/2026-09-24-email-flow-adoption.json` so nobody adopts it later.

Appendix B · Confirming a flow before you import

Do not guess from the name. Several of these flows have near-identical names and one is a superseded predecessor of another; ITEM-60 records six supplied packages that are exports of flows no wired endpoint resolves to at all.

The trigger URL is what decides. Take the configured URL for the contract key — from `config/config.local.js`, or from the endpoint register — and read the 32-hex id in its path:

```
.../workflows/<32-hex workflow id>/triggers/manual/paths/invoke?...
```

Look that id up in `docs/folio-Master-Flows-and-Endpoints/registers/endpoint-register.json` under `current_configuration`. The `flow_name` beside it is the flow to export. If two flows in the tenant carry that name, open each and compare its trigger URL to the configured one: only one will match.

Appendix C · If something is wrong

Every caller refused after setting the switch. Expected, if the IP_OTP_VERIFY package is not in yet. Paste it, or set the switch back; nothing is damaged in the meantime.

Sign-in returns a token but calls still 401. The proof is minted with `Purpose` `internal-session` and read back on exactly that value. Check the row the mint wrote.

A caller is refused near the end of a session. The row and the response read one expiry expression, so they cannot disagree — if this happens the paste did not take the mint.

Rolling back. Re-import the export taken in §2.1, choosing UPDATE. It is per-flow, so a bad paste is undone without touching the others.

Appendix D · What this does not do

- `SCAN_INTAKE` is not in this list and must not be. It takes the tenant-authenticated principal header, which is a stronger identity than a bearer proof.
- Obligations 5, 6 and 7 of `AUTHENTICATION_CONTRACT.md` §2 — per-action authorisation, idempotency, and server-side audit of the proof-derived identity — are untouched by this build and keep G-04 open after the paste.
- The proof is an opaque token in a ledger, not a signed structure. Power Automate has no HMAC function. The ledger is what makes the token mean anything, and revoking one is a `Consumed` flag rather than a key rotation.

Parts 2–5 — the standing procedures

Phases 0 to 3 above are the work in front of you. What follows is the rest of the estate's tenant procedure, carried here so there is one runbook rather than five. Nothing below is rewritten: these are the four documents that used to stand separately, in the order they run.

Part 2 · DGO ECM — Implementation Execution Guide

> **This document carries no commands.** It is background and rationale: why each decision was taken, what the evidence was, and what the alternatives cost. The five **Procedure** blocks that used to be buried in it — 3% of 1,925 lines — are now §0 of **Part 3 of this runbook**, with the rest of that domain's steps.
>
> The three documents that carry steps are **Part 3 of this runbook**, **Part 5 of this runbook** and **Part 4 of this runbook**. If you are here to execute, go to one of those.

Document status: Operational. This is the instruction set for taking the platform from its current state to a working deployment. It is self-contained: every list GUID, column name, site URL, request shape, response shape, rate limit, threshold and verification command needed to execute it is reproduced here.

Repository: `dgoeaa/ECM_DOCS_DEV` **Branch:** `claude/sharepoint-lists-gap-bqo7j7` **Guide revision:** 2026-08-23

Two roles execute this guide.

Role	Means	Does
Operator	Tenant access — Power Automate, SharePoint site settings, the mailbox	Phases 1–7
Repository owner	`dgoeaa` GitHub org admin	Phase 0 only

Read before starting: §0.3 Two delivery routes. Applying the older patch-track artifacts on top of a pasted package will silently undo the fixes.

Table of contents

- 0 · Before you start
- 1 · Reference data
- 2 · Phase 0 — Repository access and the dry run
- 3 · Phase 1 — SharePoint preparation
- 4 · Phase 2 — Decisions taken before any paste
- 5 · Phase 3 — The portal flows
- 6 · Phase 4 — The internal flows
- 7 · Phase 5 — Capture endpoint URLs and configure the portal
- 8 · Phase 6 — End-to-end verification
- 9 · Phase 7 — Security actions
- 10 · Deferred work and what unblocks it
- 11 · Command reference
- 12 · Troubleshooting

- 13 · Sign-off record

---

0 · Before you start

0.1 What you need in hand

Requirement	Why	Verify you have it
Power Automate maker access to the environment holding the `Portal_*` and `DGO_*` flows	Every paste is a designer edit	Open any `Portal_*` flow and click **Edit** without an error
SharePoint **Manage Lists** on `NEDMS`, `Global_Digital_Documents_Centre`, `DGO_ECM_GOVERNANCE`	Phase 1 adds column indexes	Open a list → **Settings** → the **Indexed columns** link is present
A SharePoint connection in the environment	102 of 105 connector actions bind to it	Power Automate → **Data** → **Connections** shows a SharePoint connection you own
An Office 365 Outlook connection for the sending mailbox	Three mail actions	Same page shows the Outlook connection for the mailbox chosen in §4.2
Node.js ≥ 22 and the repository cloned	Every verification command	`node --version` prints v22 or higher
PowerShell 5.1 or later	The export and patch scripts	`$PSVersionTable.PSVersion` prints 5.1 or higher

0.2 Confirm the repository is at the right revision

```
git clone https://github.com/dgoeaa/ECM_DOCS_DEV.git
cd ECM_DOCS_DEV
git checkout claude/sharepoint-lists-gap-bqo7j7
npm install
npm test
```

`npm test` must report every script passing. `test:smoke` requires Playwright browsers; if it is the only failure and the message is `Cannot find package '@playwright/test'`, that is expected in an environment without them and does not block this guide.

Then confirm the flow packages are intact:

```
npm run test:idphase3        # the portal identification packages, run in the simulator
npm run test:idphase4        # the internal identification packages, run in the simulator
npm run triggers             # must read: 0 field read(s) the contract does not define
npm run test:dynamicops      # must read: 19 DYNAMIC_ACTIONS discriminator(s), in sync
```

If any of these fails, **stop**. The packages on disk do not match the specification and pasting them will deploy something that has not been checked.

0.3 One delivery route

Every endpoint runs on one adopted flow (`FLOW_ADOPTION.md`), and the changes to it are the identification packages for the adopted flows: `packages/identification-phase3/README.md` (portal) and `packages/identification-phase4/README.md` (internal). Each package is the flow's root scope with the changes applied: export the flow first (the export is the rollback), then replace the named scope with the file.

The DGO_* and Portal_*_ECM_DOCS replacement bodies and the patch-track placements that stood here were retired on 2026-10-05 and 2026-10-06 (`registers/flow-retirement.json`), except remediation visit 4 (`ECM_DOCS_INTAKE`).

0.4 The order is not arbitrary

Later steps depend on earlier ones in ways that are not obvious:

- **Indexes before traffic.** A list that crosses 5,000 items with an unindexed filter column stops answering. Adding the index afterwards requires the list to be under the threshold again.
- **Decisions before pasting.** The CORS origin and the mail connection are set inside each flow. A decision taken after pasting means opening all fourteen flows a second time.
- `IP_OTP_VERIFY` **before the switch.** Every internal identification package resolves its caller from the sign-in proof once `IDENTITY_PROOF_REQUIRED` is set, and only IP_OTP_VERIFY's package mints one. With `DGO_UserDirectory` empty, no proof is issued and every internal endpoint answers `401`.
- `VERIFY` **and** `VERIFY_CONFIRM` **before** `SUBMISSION`**,** `STATUS` **and** `WRITEBACK`**.** Those three consume a verification proof. Without the pair working, no proof exists to consume.

---

1 · Reference data

Everything in this section is reproduced from the authoritative files in the repository. Where this guide and one of those files disagree, the file wins and the disagreement is a defect in this guide.

Subject	Authoritative file
Portal request/response contracts	`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/portal-data-contract.json` (status: **AUTHORITATIVE**)
Portal list GUIDs, sites, columns	`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/portal-field-spec.json`
Internal list GUIDs, columns, provenance	`docs/folio-Master-Flows-and-Endpoints/deployment/internal/internal-field-evidence.json`
Build standard (the ten checks)	`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/flow-standard.json`

Subject	Authoritative file
`DYNAMIC_ACTIONS` discriminators	`docs/folio-Master-Flows-and-Endpoints/deployment/internal/dynamic-operations.json` (generated)
Decisions D1–D14	`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/DECISIONS.md`
Open items	`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/OPEN_ITEMS.md`

1.1 Portal lists — thirteen, across three sites

List	GUID	Site
Portal Registry	`4c49f66a-23cd-4e1f-8ce7-ec1bb40eb667`	`https://nitdanigeria.sharepoint.com/sites/NEDMS`
Portal Attachments	`ecf2ba9b-968f-4fbc-ac9c-7b5e36d5099e`	`https://nitdanigeria.sharepoint.com/sites/NEDMS`
Portal Status Timeline	`5b486a5e-0ce7-46d7-a159-d84c77f3d1fd`	`https://nitdanigeria.sharepoint.com/sites/NEDMS`
Portal Upload Tickets	`d777f3cd-0696-426e-8096-ffc860e7c0d4`	`https://nitdanigeria.sharepoint.com/sites/NEDMS`
Portal Rate Limits	`d6b97198-489c-4bd7-8647-1133c55efdf9`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal OTP Codes	`3f85213c-1fdc-4be0-8c0c-e28aa77fbe56`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Verification Proofs	`f5c9698e-4b6c-4ec7-97ee-dc540efab8a9`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Support Cases	`b984645b-8e3a-457f-a305-b95650fd3f23`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Sequence Counters	`d95409a4-2b48-4d34-9f60-da5d27db862d`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Audit Events	`f59026cc-9fd7-4322-b681-73c599b852c6`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Flow Telemetry	`726c210d-09d5-45d9-952d-7a506b644b13`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
Portal Outbox Receipts	`88a81ca1-319a-45f5-8409-f91a24538ffa`	`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
DGO_AccessScopes	`f2ffd2fa-901e-4f28-8f28-957da0fe05e4`	`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`

1.2 Portal list columns — internal names

These are the internal names the flows write. SharePoint internal names are not display names; do not substitute a display name for one of these.

List	Columns
Portal Registry	`ReferenceId`, `SenderEmail`, `Status`, `Subject`, `Category`, `CorrespondenceType`, `Channel`, `SenderName`, `SenderPhone`, `SenderOrganisation`, `SenderOrganisationType`, `EventDate`, `Description`, `AttachmentCount`, `SubmittedAtUtc`, `UpdatedAtUtc`, `AcknowledgedAtUtc`, `ClosedAtUtc`, `StatusLabel`, `ActionRequired`, `VerifiedSubmission`, `SourceIp`, `LocalId`
Portal Attachments	`SubmissionRef`, `DeclaredName`, `StoredName`, `DeclaredSizeBytes`, `DeclaredSha256`, `AttachmentLink`, `CreatedAtUtc`, `Status`
Portal Status Timeline	`SubmissionRef`, `AtUtc`, `Status`, `Label`, `Actor`, `Note`
Portal Upload Tickets	`SubmissionRef`, `DeclaredName`, `StoredName`, `DeclaredSizeBytes`, `DeclaredSha256`, `ExpiresAtUtc`, `CreatedAtUtc`, `Redeemed`, `Status`
Portal Rate Limits	`WindowStartUtc`, `UpdatedAtUtc`, `RequestCount` (bucket key is in `Title`)
Portal OTP Codes	`OTP_Code`, `Expires_At`, `Email`, `Attempts`, `Consumed`, `SourceIp`, `CreatedAtUtc`
Portal Verification Proofs	`Email`, `Purpose`, `ExpiresAtUtc`, `CreatedAtUtc`, `Consumed` (proof value is in `Title`)
Portal Support Cases	`Name`, `Email`, `Topic`, `AboutReference`, `Message`, `Status`, `SubmittedAtUtc`, `SourceIp` (case reference is in `Title`)
Portal Sequence Counters	`CurrentSequence`, `LastIssuedAt`, `LastReferenceId`, `LockToken`, `ModifiedByFlowRun`, `Prefix`, `Year`
Portal Audit Events	`Flow`, `EventType`, `AtUtc`, `Reference`, `SourceIp`, `Detail`
Portal Flow Telemetry	`Flow`, `RunId`, `StartedAtUtc`, `CompletedAtUtc`, `Outcome`, `DurationMs`, `ErrorMessage`
Portal Outbox Receipts	`MessageType`, `RecipientEmail`, `Reference`, `SentAtUtc`, `Status`, `Attempts`, `LastError`

1.3 Internal lists — eleven, all on `DGO_ECM_GOVERNANCE` or the activity-tracking site

The tenant carries **34** lists whose title begins `DGO_`, across two sites, with `_2` and `_02` duplicates. Only the eleven below are `adopted: true`. Targeting a duplicate writes to a list nothing reads.

List	GUID	Site
Global Tracking Queue	`ee82725a-c408-45e2-a8e8-facf7a092047`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
DGO DIGITAL OPS	`1f1cb303-3fd2-43c8-8f24-c409b6c2fde3`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
Task_Comments	`1932d687-e77a-4929-9047-a1f544c68a0b`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
Organizational_Categories_Matrix	`f64c8b65-c921-46e9-8814-a8cef61f6016`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
Organizaitonal_Departments_Information	`8aab9c4e-001f-4e32-862c-8a50e750f04e`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
OTP_Transactions	`9421d473-8906-43b7-a41f-a213046683c1`	`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`
DGO_UserDirectory	`3d591f5b-3f2f-409c-983a-a77b5c174834`	`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`
DGO_RoleCatalogue	`f675598b-271d-4200-8d75-2597aad4057f`	`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`
DGO_AuditLog	`be0c7af1-b21d-4efe-8c30-53fc55598d95`	`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`
DGO_UserRoleHistory	`9fe1872f-3ae0-4b5f-96fc-5d3331c1d07c`	`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`
NITDA_Central_Registry	`f553959f-d55b-4591-aa87-d61fb6f9eda9`	`https://nitdanigeria.sharepoint.com/sites/NEDMS`

> **Known defect in a related file.** `sharepoint-provisioning-spec.json` gives `TargetSite` as `NITDADGO-EAAACTIVITYTRACKING` for all ten governance lists. The adopted copies of `DGO_UserDirectory`, `DGO_RoleCatalogue`, `DGO_AuditLog` and `DGO_UserRoleHistory` are on `DGO_ECM_GOVERNANCE`. Provisioning from that spec targets the dead duplicates. Use the table above.

1.4 Internal list columns

List	Columns
Global Tracking Queue (32)	`Acknowledge_x0020_Task`, `AcknowledgedBy`, `Acknowledgement_x0020_Due_x0020_`, `ActivityTrackingID`, `AssgnedTpUserDSU`, `Assigned`, `AssignedBy`, `AssignedTo`, `AssignedToUserTitle`, `AttachmentLink`, `Classification`, `Comments`, `Description`, `DraftResponse`, `DueDate`, `GDSUROUT`, `GTQGrouping`, `HDSUTitltetxt`, `LinktoDoc`, `OData__x0033_rdAssigned`, `ParentTask`, `Priority`, `Progress`, `RefID`, `RefIDD`, `RefIDDN`, `Reference_ID`, `RoutedToDSU`, `StartDate`, `TaskType`, `TaskTypeChoice`, `Title`
DGO DIGITAL OPS (17)	`AppFlter`, `Assigned`, `AssignedTo`, `AssignmentStatus`, `BodyExtract`, `CC_x0027_dTo`, `Category0`, `Marked_Item`, `OverallStatus`, `RefIDD`, `RoutedToDSU`, `RoutedToHDSUTitle`, `Routed_x0020_To`, `StartDate`, `Status`, `TextExtraxt`, `Title`
DGO_UserDirectory (14)	`AccessScope`, `Department`, `Directorate`, `DisabledReason`, `Email`, `FullName`, `JobTitle`, `Persona`, `Phone`, `PilotCohort`, `Role`, `Status`, `Title`, `UserId`
DGO_RoleCatalogue (10)	`Active`, `AllowedRoutesJson`, `CanAssignRoles`, `CanManageSettings`, `CanViewAudit`, `PermissionsJson`, `Persona`, `RoleId`, `Title`, `Version`
OTP_Transactions (4)	`Expires_At`, `Is_Verified`, `OTP_Code`, `Title`
Task_Comments (4)	`Created`, `Description`, `RefIDD`, `Title`
Organizational_Categories_Matrix (6)	`Category`, `DSU_KEY`, `Priority`, `Subcategory`, `Timeline`, `Title`
Organizaitonal_Departments_Information (6)	`DSU_Email`, `DSU_HeadEmail`, `DSU_HeadPersonalEmail`, `DSU_HeadTitle`, `DSU_KEY`, `Title`
DGO_AuditLog (1)	`Title`
DGO_UserRoleHistory (1)	`Title`
NITDA_Central_Registry (1)	`Title`

> `Global Tracking Queue` **has no** `Status` **column.** `core/domain.js` resolves a task's status from `Progress`. A status change on a task writes `Progress`. A status change on a *reference* writes `DGO DIGITAL OPS/Status`. These are different lists and different columns; they are not interchangeable.

1.5 Rate limits, as built

A fixed 60-minute window per endpoint per source IP. The bucket key is written to `Portal Rate Limits/Title` as `<ENDPOINT>_IP:<X-Forwarded-For>`. A caller with no `X-Forwarded-For` buckets as

`unknown`, which throttles harder, never softer.

Endpoint	Requests per hour per IP	Bucket key prefix
SUBMISSION	20	`SUBMISSION_IP:`
UPLOAD	60	`UPLOAD_IP:`
SUPPORT	10	`SUPPORT_IP:`
VERIFY	5	`VERIFY_IP:`
VERIFY_CONFIRM	5	`VERIFY_CONFIRM_IP:`
STATUS	30	`STATUS_IP:`
WRITEBACK	20	`WRITEBACK_IP:`

A refusal is `429` **through the standard response path**, not an early terminate — the refusal is recorded in `Portal Flow Telemetry` like any other outcome.

This is a fixed window, not a sliding one (D8). A caller can spend its allowance at the end of one window and again at the start of the next. This is an accepted trade-off, recorded, and is not a defect to raise.

1.6 Endpoint contracts

The portal posts a **flat** JSON body and reads a **flat** JSON response. There is no envelope in either direction on the portal side.

SUBMISSION — `POST`, JSON

Register a correspondence, mint its reference, issue one upload ticket per declared attachment.

Request field	Type	Required	Note
`localId`	string	yes	Client correlation id. Lets a retry be recognised as the same submission.
`channel`	string	yes	
`correspondenceType`	string	yes	
`subject`	string	yes	
`category`	string	yes	One of the eight correspondence types. A value outside the set must be rejected.
`sender.name`	string	yes	
`sender.organisation`	string	no	
`sender.organisationType`	string	no	

Request field	Type	Required	Note
`senderEmail`	string	yes	Stored lowercased and trimmed. STATUS matches on it.
`senderPhone`	string	no	
`eventDate`	string	no	Empty string unless the correspondence type carries one.
`description`	string	yes	
`submittedAt`	iso8601	yes	
`attachments[]`	array	no	Count only on the registry row; each element becomes a ticket.
`attachments[].name`	string	yes	
`attachments[].size`	number	yes	
`attachments[].sha256`	string	yes	Declared so UPLOAD can verify against it.
`verification`	string	no	The single-use proof from VERIFY_CONFIRM, when the wizard holds one.

Response field	Type	Required
`referenceId`	string	**yes** — without it the citizen cannot track the submission
`uploads[]`	array	when attachments were declared. **An empty array when attachments were declared means the attachments are silently never sent.**
`uploads[].ticket`	string	yes
`uploads[].name`	string	yes
`error`	string	on 403, value `verification_required`

Status	Meaning
`200`	accepted
`403 + error=verification_required`	the wizard asks for a code
other 4xx/5xx	queued for retry by the client

UPLOAD — `PUT`, raw bytes

Redeem one single-use ticket with the raw file.

Request	Type	Required	Note
`Content-Type`	header	yes	

Request	Type	Required	Note
`X-Upload-Ticket`	header	yes	The ticket from SUBMISSION. Single use — the flow sets `Redeemed` and refuses a second attempt.
body	bytes	yes	**Raw, never base64.**

Response field	Type	Required	Note
`stored`	boolean	yes	**Must be truthful.** `200 {stored:false}` means the bytes were not filed.
`attachmentLink`	string	when stored	
`reason`	string	when not stored	`core.js upload()` also accepts `error` as a fallback key; `reason` is the specified name.

SUPPORT — `POST`, JSON

Raise a helpdesk case. Never enters the registry.

Request field	Type	Required	Note
`name`	string	yes	
`email`	string	yes	
`topic`	string	yes	The topic's label, not an internal key.
`aboutReference`	string	no	A tracking id, **unverified**. Empty string when blank.
`message`	string	yes	

Response field	Type	Required	Note
`caseRef`	string	yes	**The reference the citizen is given must be this one.** A locally generated reference exists in no system of record.

VERIFY — `POST`, JSON

Mail a one-time code to an address.

Request field	Type	Required	Note
`email`	string	yes	Stored lowercased and trimmed. VERIFY_CONFIRM matches on it.

Response field	Type	Required	Note
`sent`	boolean	yes	**Must be present and truthful.** The client tests `=== true`, so an absent field reads as failure.

Response field	Type	Required	Note
`expiresAt`	iso8601	yes	

VERIFY_CONFIRM — `POST`, JSON

Exchange a code for a single-use proof.

Request field	Type	Required	Note
`email`	string	yes	Must match the address VERIFY was called with.
`code`	string	yes	Six digits. **The flow compares it to the stored code** — the lookup filter is not a comparison.

Response field	Type	Required	Note
`verification`	string	yes	**The proof. Absent means a correct code was refused.**
`expiresAt`	iso8601	yes	

> **How each half is selected.** Neither call sends an `action` field. The flow routes on the presence of `code`: a request carrying one is a verify, a request without one is a generate. An explicit `action` still wins when a caller supplies one.

STATUS — `POST`, JSON

Read one submission back, in the public projection only.

Request field	Type	Required	Note
`referenceId`	string	yes	Trimmed and uppercased before matching.
`email`	string	when no proof is held	**Structurally absent** from the body when a proof is held — omitted, not blank.
`verification`	string	when a proof is held	Redeemed and consumed by this call.

Response projection — this is a **closed allow-list**:

Field	Type	Source column
`record.referenceId`	string	`Portal Registry/ReferenceId`
`record.status`	string	`Portal Registry/Status`
`record.statusLabel`	string	`Portal Registry/StatusLabel`

Field	Type	Source column
`record.category`	string	`Portal Registry/Category`
`record.subject`	string	`Portal Registry/Subject`
`record.receivedAt`	iso8601	`Portal Registry/SubmittedAtUtc`
`record.acknowledgedAt`	iso8601	`Portal Registry/AcknowledgedAtUtc`
`record.updatedAt`	iso8601	`Portal Registry/UpdatedAtUtc`
`record.closedAt`	iso8601	`Portal Registry/ClosedAtUtc`
`record.actionRequired`	boolean	`Portal Registry/ActionRequired`
`record.timeline[].at`	iso8601	`Portal Status Timeline/AtUtc`
`record.timeline[].status`	string	`Portal Status Timeline/Status`
`record.timeline[].label`	string	`Portal Status Timeline/Label`
`record.timeline[].note`	string	`Portal Status Timeline/Note`

Excluded from the projection, deliberately: `description`, attachments, assigned officer, handling unit, internal notes, `SenderName`, `SenderPhone`, `SenderOrganisation` and every other sender field, `SourceIp`, `LocalId`, `VerifiedSubmission`.

Status	Meaning
`200`	record returned
`403 + error=verification_required`	the email check is demanded
`404`	**byte-identical denial** — the same response whether the reference does not exist or is not yours
`429`	rate limited

WRITEBACK — `POST`, JSON

A verified citizen responds, adds a note, or withdraws. Appends to the timeline; **only withdrawal changes status**.

Request field	Type	Required	Note
`referenceId`	string	yes	
`verification`	string	yes	A proof identifies the submitter. **There is no other authentication.**
`action`	string	yes	One of `respond`, `note`, `withdraw`
`body`	string	yes	

Response field	Type	Required	Note
`ok`	boolean	yes	
`queued`	boolean	no	The flow's own signal that it accepted the write but has not finished applying it. **Not** a client-side retry queue.
`reason`	string	when not ok	

Status	Meaning
`200`	accepted
`401` / `403`	the proof is missing, invalid, expired, or already consumed
`404`	byte-identical denial, matching STATUS's INT-006
`429`	rate limited

> **WRITEBACK is never queued in the portal's outbox.** The proof is single-use, so a queued retry would carry one already spent. A failed call is reported to the citizen immediately and a fresh attempt mints a fresh proof.

1.7 The internal platform's response shape is different

The internal flows answer the **envelope**, because `core/contracts.js` `assertEnvelope()` returns `response.data ?? response`:

```
{
  "ok": true,
  "status": { "http": 200, "code": "OK", "message": "Success" },
  "request": { "requestId": "…", "trackingId": "", "action": "fetchAll", "operation":
"read", "source": "portal" },
  "timing": { "receivedAtUtc": "…", "completedAtUtc": "…", "durationMs": 0 },
  "data": { },
  "errors": [],
  "meta": { "ts": "…", "runId": "…", "flowName": "…", "contractVersion": "2026-08-21.1" }
}
```

The internal client also **nests its request**:

```
{ "action": "<contract action>", "payload": { }, "userEmail": "…", "requestId": "…",
"timestamp": "…" }
```

Only `action`, `userEmail`, `requestId` and `timestamp` are top-level, and `action` is the endpoint's fixed contract action — `dynamicGlobalAction`, `otpGenerate`, `fetchAll` — never the caller's own discriminator. One call site differs: `modules/single-assignment.js` sends `flatPayload: true` and puts the caller's fields alongside the envelope keys.

Do not normalise the two response shapes to one. The simulated runs in `npm run test:idphase3` and `test:idphase4` read each reply in the shape its own client reads.

---

2 · Phase 0 — Repository access and the dry run

2.1 Grant write access and land the pending commit — retired

Retired 2026-10-05. The pending commit carried the `Portal_Verify` (86897b2f) export, which proved remediation visit 1. `Portal_Verify` and visit 1 are retired; `VERIFY` runs on `CG_Portal_Verification_Endpoint` (ba47f79b), whose definition is held in `docs/folio-Master-Flows-and-Endpoints/contracts/deployed/`. Nothing to do.

2.2 Prove tenant connectivity with the dry run *(open item 2)*

Owner: operator.

`update-flow-definition.ps1` has never run against the tenant. The entire patch track depends on it, and although the paste track supersedes it for these fourteen flows, this run is the cheapest possible proof that your credentials, environment id and flow ids are all correct — before you spend an hour pasting.

It sends nothing. Without `-Apply` the script reads a flow definition and prints what it found.

The steps are §0.2 of Part 3 of this runbook, which is the only document that carries them. Everything here is the reasoning behind them.

Verification

The script prints the flow's display name and its action count without error. If it prints an authentication failure, stop and resolve that before Phase 1 — every later phase needs the same credentials.

---

3 · Phase 1 — SharePoint preparation

3.1 Index three hot-path columns *(open item 23 — highest risk in this guide)*

Owner: operator. Roughly ten minutes. Do it before any traffic.

Three lists are queried on `Title` with an equality filter on the hot path:

List	Filtered on	Frequency	Growth
Portal Rate Limits	`Title`	**every request to every endpoint**	one row per source IP per endpoint

List	Filtered on	Frequency	Growth
Portal Verification Proofs	`Title`	every proof presentation	one row per verification
Portal Support Cases	`Title`	every case, for collision detection	one row per case

Why this is urgent and not an optimisation. SharePoint's 5,000-item list view threshold makes an unindexed equality filter **fail**, not slow. The query returns an error, the flow's catch scope fires, and the endpoint answers `500`. `Portal Rate Limits` is read on every request to every endpoint, so the first list to cross the threshold **stops every endpoint at once**. Adding the index after the list has crossed 5,000 items is materially harder — SharePoint may refuse to index a list already over the threshold, requiring items to be deleted first.

`Portal Registry/ReferenceId` is already indexed, which shows the estate knows the pattern.

Procedure — repeat for each of the three lists

1. Open the list in SharePoint:
   - Portal Rate Limits — `https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`
   - Portal Verification Proofs — same site
   - Portal Support Cases — same site
2. **Settings** (gear) → **List settings**.
3. Under **Columns**, click **Indexed columns**.
4. **Create a new index**.
5. **Primary column for this index** → `Title`.
6. **Create**.

Verification

Return to **Indexed columns** on each list. `Title` must be listed. Record the three confirmations in §13.

3.2 Seed the role catalogue *(Wave 0 — required before any internal flow works)*

Owner: operator.

Every internal package resolves its caller through the identity gate:

1. `Compose_<tag>_Caller` — the address from the request body.
2. `Get_<tag>_Directory_User` — `DGO_UserDirectory` filtered on `Email` **and** `Status eq 'active'`.
3. `Compose_<tag>_Role` — the `Role` column off that row.
4. `Get_<tag>_Role_Permissions` — `DGO_RoleCatalogue` on `RoleId`, `Active eq 1`.
5. `Compose_<tag>_Permitted` — the permission is in `PermissionsJson`, or `AllowedRoutesJson` is `["*"]`.

With `DGO_RoleCatalogue` and `DGO_UserDirectory` empty, every internal endpoint answers `401`. This is correct behaviour, not a fault — but it means the internal platform does nothing until both lists are populated.

The steps are §0.3 of Part 3 of this runbook, which is the only document that carries them. Everything here is the reasoning behind them.

3.3 Populate the user directory

Owner: operator.

One row per officer in `DGO_UserDirectory` (`3d591f5b-3f2f-409c-983a-a77b5c174834`, on `DGO_ECM_GOVERNANCE`).

Column	Required	Note
`Email`	**yes**	The gate filters on this. Lowercase and trimmed.
`Status`	**yes**	Must be exactly `active` — the gate filters `Status eq 'active'`. Any other value denies the caller.
`Role`	**yes**	Must match a `RoleId` in `DGO_RoleCatalogue`. A role with no matching catalogue row denies every permission.
`FullName`	recommended	Read by `normalizeUser`
`Title`	recommended	
`Department`, `Directorate`, `JobTitle`, `Phone`, `Persona`, `AccessScope`, `UserId`, `PilotCohort`	optional	
`DisabledReason`	optional	Set when `Status` is not `active`

Verification

Populate one row for yourself first, then verify the whole chain at the end of Phase 4 (§6.3) rather than now — the flow that reads it is not pasted yet.

> **Until this is done the platform fails open.** `core/state.js` seeds a bootstrap administrator, so every browser is a System Administrator. The identity gate in the internal packages is what closes that, and it only closes it once these two lists have rows.

---

4 · Phase 2 — Decisions taken before any paste

Both of these are set **inside** each flow. Taking them after pasting means opening all fourteen flows a second time.

4.1 Decide the portal's CORS origin *(open item 8)*

Owner: operator. **Decision required — this guide cannot supply the value.**

Every portal package ships `Access-Control-Allow-Origin: https://your-host` in its `Response_*` action. Until that is the portal's real origin, **a browser refuses every response** — the request succeeds, the flow runs, the row is written, and the page sees a network error. This is the single most likely cause of "nothing works" after a correct deployment.

What to decide: the exact scheme and host the static portal is served from, with no trailing slash and no path. Examples of the correct form: `https://portal.nitda.gov.ng`, `https://nitdanigeria.github.io`. If the portal is served from more than one origin, you must decide one — the header takes a single origin, and `*` must not be used because these endpoints are credentialed by their signed URL.

Record the decision in §13 before pasting. Every one of the seven portal flows uses the same value.

4.2 Choose the sending mailbox *(open item 24)*

Owner: operator. **Decision required — this guide cannot supply the value.**

Three actions send mail, and all three ship `REPLACE_WITH_OFFICE365_CONNECTION_ID`:

Action	Package	Sends
`Send_Otp_Email`	`Portal_VERIFY_ECM_DOCS` **and** `Portal_VERIFY_CONFIRM_ECM_DOCS`	The citizen's one-time code
`Send_Support_Acknowledgement`	`Portal_SUPPORT_ECM_DOCS`	The support case acknowledgement
`Send_Otp_Mail`	`DGO_OTP_GENERATE`	The officer's one-time code

Every SharePoint action ships already bound to `3f1943c5955a4cb8b301e8f22f2b590d` — the one SharePoint connection named in the tenant's own flow records. Only these three mail actions do not, because the tenant's records carry **three** Office 365 Outlook connections (`0e5c949a72f64e8db50ec220815a2ff3`, `c0b9e7a5b0854c39a435fd8ce92f48ad`, `shared-office365-1fb94487-c973-4c56-a1f4-1ac16768a7a7`) and say nothing about which sends what.

What to decide: the mailbox whose address citizens and officers see as the sender of a one-time code. This is a service-identity decision, not a technical one — the address appears in every verification mail the public receives.

Record the decision in §13. You will pick this connection on each of the three actions at paste time.

> A connection id names a resource in the environment. The credential lives in the connection itself, so nothing in this repository or this guide carries a secret.

---

5 · Phase 3 — The portal flows

The portal endpoints run on their adopted flows (`CG_Submission_Endpoint` , `CG_Upload_Endpoint` , `CG_Portal_Status_Check_Endpoint` , `CG_Portal_Support_Endpoint` , `CG_Portal_Writeback_Endpoint` , `CG_Portal_Verification_Endpoint` , `CG_Portal_Verification_Confirmation_Endpoint`). Their changes, and the order and check for each, are in `packages/identification-phase3/README.md` .

The seven Portal_*_ECM_DOCS replacement bodies this section pasted are retired (2026-10-06).

---

6 · Phase 4 — The internal flows

The internal endpoints run on their adopted flows. Their changes, the caller from the sign-in proof (G-04) among them, are in `packages/identification-phase4/README.md` , in the order Part 1 of this runbook, Phase 2, gives.

The DGO_* replacement bodies, their preflight and the export-patch-import install route this section carried are retired (2026-10-06).

---

7 · Phase 5 — Capture endpoint URLs and configure the portal

7.1 Record the workflow ids *(open item 21)*

Owner: operator. Roughly ten minutes.

`FLOW_CATALOGUE.json` gives a 32-hex id per endpoint; the exports carry only dashed internal names. The two schemes have never been joined, and three endpoints have more than one candidate flow with nothing settling which is live.

The steps are §0.5 of Part 3 of this runbook, which is the only document that carries them. Everything here is the reasoning behind them.

7.2 Configure the portal

Owner: operator.

```
cp document-portal/config.example.js document-portal/config.local.js
```

`config.local.js` is git-ignored, so no URL is ever committed. Fill in all seven:

```
window.PF_CONFIG = {
  endpoints: {
    SUBMISSION:     "<Portal_UBMISSION_ECM_DOCS trigger URL>",
    UPLOAD:         "<Portal_UPLOAD_ECM_DOCS trigger URL>",
    SUPPORT:        "<Portal_ECM_DOCS_SUPPORT trigger URL>",
    VERIFY:         "<Portal_Verify trigger URL>",
    VERIFY_CONFIRM: "<Portal_Verify_Confirm trigger URL>",
    STATUS:         "<Portal_ECM_DOCS_STATUS trigger URL>",
    WRITEBACK:      "<Portal_ECM_DOCS_WRITEBACK trigger URL>"
  }
};
```

Behaviour when a slot is left empty

Left empty	Result
`SUBMISSION`	The whole portal stays in **DEMO MODE** — everything stays local, nothing is transmitted
`VERIFY` or `VERIFY_CONFIRM`	Verification reports itself unavailable. **Both are required**; one without the other counts as unavailable, because offering a code the citizen cannot redeem is worse than offering nothing
`WRITEBACK`	The write-back UI stays dormant. It also requires both verification halves
Any other	That feature reports itself unconfigured

> **Every URL in this file is delivered to every visitor's browser.** Configure only endpoints whose flows are built to be invoked by an anonymous stranger — each must validate its own input, rate-limit its own callers, return only what the caller is entitled to see, and be rotated on a schedule. All seven packages in this guide are built to that standard; a flow that is not must never be listed here.

7.3 Verify the wiring from the repository

```
npm run wiring
```

---

8 · Phase 6 — End-to-end verification

Run this as a citizen would, in a browser, against the deployed portal. Repository tests cannot prove any of it — they check the artifacts, not the tenant.

8.0 Run the probe first

Before working through the journeys by hand, exercise every endpoint from a terminal:

```
cp document-portal/config.example.js document-portal/config.local.js   # if not already done
# fill in all seven URLs, then:
npm run verify:endpoints -- --surface portal
```

That call is **read-only by default** — it probes STATUS only. To exercise the rest:

```
npm run verify:endpoints -- --surface portal --include-writes
```

Writes are opt-in because they create real rows in a real register. Every write probe carries a `__DGO_PROBE__` marker and a per-run id, so the rows it creates can be found and deleted afterwards. Search each list's `Title`, `Description` or `Message` column for `__DGO_PROBE__` and remove what you find before the portal goes live.

What a pass looks like, per endpoint

Endpoint	Probe sends	Pass
STATUS	`referenceId: __DGO_PROBE__`, `email`	`404` — the uniform denial. A `200` means the register confirms whether a reference exists.
SUBMISSION	A complete contract-shaped submission with no attachments	`200` carrying `referenceId` **at the top level**
SUPPORT	`name`, `email`, `topic`, `message`, `aboutReference`	`200` carrying `caseRef` at the top level
VERIFY	`email` only — **no** `action`	`200` carrying `sent` and `expiresAt` at the top level
VERIFY_CONFIRM	`email`, `code: 000000`	`401`, `404` or `410` — a code of six zeros matches no live challenge. A `200` with a `verification` means the flow mints proofs for anyone.
WRITEBACK	A well-formed body with an invalid proof	`401`, `403` or `404` — the proof is the only authentication on this endpoint
UPLOAD	Raw bytes, no ticket	`401` or `403` — a flow that accepts bytes without a redeemable ticket is one anyone can write into

A failure the probe names explicitly: if a portal endpoint answers with the contract's fields inside `data`, the probe reports

```
✗ answered the envelope — referenceId is inside 'data', where the portal cannot read it
```

That is not a near miss. It means the flow is answering the internal platform's shape and the portal will read `undefined` for every field while seeing HTTP 200.

No secret is printed. Endpoints are identified by contract key and workflow id; the signature never reaches the terminal or the JSON report, so a report can be pasted into an issue.

For the internal surface:

```
npm run verify:endpoints -- --surface runtime
```

8.1 The citizen journey

#	Step	Expected	If it fails
1	Open the portal	The submission wizard renders; no "demo mode" banner	`SUBMISSION` is empty in `config.local.js`
2	Complete a submission with one attachment	A reference is shown, of the form the sequence counter mints	Check `Portal Registry` for the row; if the row exists but no reference is shown, the response is not reaching the browser — see §12.1
3	Check `Portal Attachments`	One row, `Status` filed	If absent, the ticket was issued but not redeemed — check `Portal Upload Tickets/Redeemed`
4	Track the reference with the submitting email	The record renders with status and timeline	
5	Track the same reference with a different email	"Not found" — and the response body is byte-identical to an unknown reference	If they differ, the denial is distinguishable and INT-006 is not met
6	Request a verification code	The code arrives in the mailbox; the page says a code was sent	If the page says it could not be sent but the code arrives, `sent` is not truthful
7	Enter the code	The proof is held; the verified view opens	If a correct code is refused, see §12.3
8	Respond through write-back	The response appears on the timeline; status unchanged	
9	Withdraw	Status changes; the timeline records it	
10	Repeat step 8 without re-verifying	Refused — the proof was single-use	If accepted, the proof is not being consumed
11	Raise a support case	A `CASE-` reference is shown, matching `Portal Support Cases/Title`	If the shown reference is not in the list, the client is generating it locally
12	Request six verification codes within an hour from one IP	The sixth returns `429`	

8.2 The officer journey

#	Step	Expected
1	Sign in as an officer with a `DGO_UserDirectory` row, `Status = active`	A code arrives; the role in `claims.roles` matches the directory row
2	Sign in as an address with no directory row	`401` — and the response reveals nothing about whether the address exists
3	Load the workspace	`FETCH_ALL` returns all eight collections
4	Assign a task	`Global Tracking Queue/AssignedTo` updates
5	Repeat the identical assignment	`changed: false`, nothing written
6	Flag a document	`DGO DIGITAL OPS/Marked_Item` set
7	Attempt a dispatch	`501`, `applied: false`, and the UI reports it as not applied — **not** as done

8.3 Telemetry

After the two journeys, `Portal Flow Telemetry` must carry one row per flow run, with `Outcome`, `DurationMs` and `RunId`. Confirm that:

- Refusals (`429`, `404`, `403`) appear as rows — they run through the standard response path, not an early terminate.
- No row's `ErrorMessage` contains a one-time code, a proof value, or a `sig=` token.
- `Portal Audit Events` rows carry the action and the body **length**, never the body.

---

9 · Phase 7 — Security actions

9.1 Rotate seven live third-party API keys *(open item 7)*

Owner: operator. **These are live now.**

Seven keys were found in the estate: **Google ×4, OpenRouter, OpenAI, Hugging Face.** They are redacted in the repository, and **redaction is not rotation** — a redacted key in a document is still a live key at the provider.

Procedure, per key, in this order

1. Open the provider console and check usage for signs of misuse.
2. Mint a replacement key.
3. Move the new value into a **Power Automate environment variable** or a **Key Vault reference** — not into a flow action, and not into this repository. A key pasted into an action is a key the next export carries out again.
4. Update whatever consumes it and confirm it works.
5. **Revoke the old key last.**

Verification: the old keys return `401` from their provider.

9.2 Decide on the six circulated trigger tokens *(open item 22)*

Owner: security authority. This is a decision, not a task.

Six live `sig=` trigger tokens were circulated in the closure package. They are **bearer credentials** for all six public endpoints.

The decision: whether to regenerate the six trigger URLs.

What has changed since they were circulated: all seven portal endpoints now validate their input, rate-limit their callers, return only the allow-listed projection, and answer a byte-identical denial. The exposure a leaked token represents is materially smaller than it was — but it is not zero, and the tokens are still valid.

If the decision is to rotate: regenerating a trigger URL changes it, which means repeating §7.1 and §7.2 for every rotated endpoint. Plan it as one pass.

Regardless of the decision: do not attach the closure package archive to tickets, and do not circulate it further.

---

10 · Deferred work and what unblocks it

These four items cannot be completed by executing this guide. Each states precisely what is missing and what would close it.

10.1 Fifteen `DYNAMIC_ACTIONS` operations answer `501` *(open item 25)*

Blocked on: column evidence for the lists those operations touch.

`dispatch`, `dispatchoutbound`, `archivereference` and the twelve activity-parity operations (`activity-{archive,siwes,nysc}` and their nine `:create-queue-record` / `:set-reference-id` / `:update-activity` steps) are recognised, authorised and audited, then explicitly refused with `501 NOT_IMPLEMENTED` and `applied: false`.

Why they are refused rather than implemented. The lists they write — the dispatch register and the DGOFASTTRACK queue — have no column evidence in `internal-field-evidence.json`. Every column in that file was harvested from a deployed flow definition or from a `core/domain.js` normaliser. Writing a column that is in neither would be a guess. A `200` with `applied: true` would tell the client the registry changed when it did not, and `PendingQueue` would stop retrying.

What closes it

1. Open each target list in SharePoint → **List settings** → record every column's **internal name** (visible in the `Field=` parameter of the column-settings URL).
2. Add them to `docs/folio-Master-Flows-and-Endpoints/deployment/internal/internal-field-evidence.json` under that list's GUID, each tagged `tenant`.

3. Write the operation bodies in the DYNAMIC_ACTIONS identification package (`dynamicActions()` in `scripts/build-identification-phase4.mjs`) against those columns.
4. `npm run idphase4 && npm run test:idphase4`
5. Re-paste `IP_Dynamic_Global_Actions_Endpoint` 's scope.

Done when no operation answers `501` .

10.2 The SHA-256 upload digest is not verified *(open item 18, INT-002)*

Blocked on: a licensing answer, then implementation.

INT-002 requires UPLOAD to verify both size and SHA-256 against the received bytes. **Size is verified** — the package compares `DeclaredSizeBytes` to `Content-Length` and answers `422` on a mismatch. **The digest is not.**

Why. Workflow Definition Language has no hashing function; a digest cannot be computed in an expression. Power Automate *can* compute one, through an `Execute JavaScript Code` action (`JavaScriptCode`), and the estate's own `04-portal-upload` design specifies exactly that. But `JavaScriptCode` appears in **none** of the 57 exported flow definitions and none of the 14 packages.

So this is unclosed, not unachievable.

What closes it

1. Confirm the environment's licensing permits `Execute JavaScript Code` — it is a premium action and its availability depends on the plan.
2. Add the action to `Portal_UPLOAD_ECM_DOCS` between reading the body and filing it, computing the SHA-256 of the received bytes.
3. Compare it to the ticket's `DeclaredSha256` ; answer `422` on a mismatch, as the size check does.
4. Prove it against a real upload — both a matching and a deliberately corrupted file.

Until then: no document, screen, report or statement may claim the digest is verified.

10.3 Seventeen of 57 flows have no stated purpose *(open item 9)*

Blocked on: knowledge of the estate that is not in the repository.

`docs/folio-Master-Flows-and-Endpoints/FLOW_CATALOGUE.md` records what each flow *does* — trigger, parameters, lists, structure — but not why it exists. Seventeen are attributed to no contract key and no portal endpoint.

What closes it: one sentence each, into the contract register, from someone who knows why the flow was built. **Done when** the catalogue reports zero with no stated purpose.

Why it matters: a flow nobody can account for cannot be safely retired, and cannot be safely kept. Several are candidates for deletion — `Portal_Status_Enquiry` is already marked delete-do-not-patch — but that call needs someone who knows what depended on them.

10.4 The patch-track visits *(open items 3, 4, 6)*

These are **superseded** for every flow the paste track covers, which is all seven portal flows. They remain in `OPEN_ITEMS.md` because the placement files record the reasoning behind each change and because the patch track is the only route for a flow with no package.

Do not execute them after pasting. Applying the `ECM_DOCS_INTAKE` write-back placements on top of a pasted package will reintroduce the defects the package fixes. The status and submission placements (`03-status-registry-read.json`, `02-submission-registry-write.json`) are retired with the flows they targeted.

If a flow outside these fourteen needs changing, the patch track is still the route: `docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/remediation/README.md` is its procedure.

---

11 · Command reference

Every command, what it proves, and what a passing result reads.

11.1 Verification

Command	Proves	Passing output
`npm test`	Everything below, plus the client-side suites	every script passing
`npm run test:idphase3` / `test:idphase4`	Each identification package parses, uses only functions the tenant runs, stays within the designer's limits, writes only columns its lists have, and runs end to end in the simulator	`0 failed`
`npm run triggers`	Every trigger field the artifacts read is one the contract defines	`0 field read(s) the contract does not define`
`npm run test:dynamicops`	The recognised `DYNAMIC_ACTIONS` discriminators match what the client sends	`19 DYNAMIC_ACTIONS discriminator(s), in sync with the client`
`npm run flowstandard`	Build-standard conformance across the estate	as the report states
`npm run wiring`	Operations, endpoints and boundary crossings, measured from the repository	see §2.1 and §7.3 for the target
`npm run test:roles`	The role seed matches `config/rbac.config.js`	all passed
`npm run test:datacontract`	The portal data contract is internally consistent and its persistence targets exist	`0 error(s), 0 gap(s), 0 unwritten column(s)`
`npm run test:probecontract`	The live-endpoint probes send the contract's field names, omit `action` on the verify pair, and expect a flat portal response	`29 passed, 0 failed`
`npm run test:guide`	This guide still describes the packages that exist — paths, action counts, list GUIDs	`0 failure(s).`

11.2 Regeneration

Run these only when changing the builders. They overwrite the packages.

Command	Does
`npm run idphase3`	Rebuilds the portal identification packages
`npm run idphase4`	Rebuilds the internal identification packages
`npm run dynamicops`	Regenerates `dynamic-operations.json` from the client source
`npm run seed:roles`	Regenerates `role-catalogue-seed.json` from `config/rbac.config.js`
`npm run visual`	Regenerates the platform atlas dataset

After any regeneration, `npm run test:idphase3 && npm run test:idphase4` must both pass before the result is pasted anywhere.

11.3 Live endpoint probes

Command	Does
`npm run verify:endpoints`	Probes the read-only endpoints on both surfaces
`npm run verify:endpoints -- --surface portal`	Portal only
`npm run verify:endpoints -- --surface runtime`	Internal only
`npm run verify:endpoints -- --include-writes`	Also probes the mutating endpoints. **Creates real rows.**
`npm run verify:endpoints -- --only VERIFY,VERIFY_CONFIRM`	A named subset
`npm run verify:endpoints -- --json report.json`	Writes a transcript. Carries no signature.

Reads the URLs from `document-portal/config.local.js`, which is git-ignored. **Nothing here is safe to point at a production tenant** while the write probes are enabled.

11.4 Tenant scripts

Script	Does	Sends anything?
`scripts/update-flow-definition.ps1`	Reads or patches a deployed flow definition	Only with `-Apply`
`scripts/export-power-automate-flows.ps1`	Exports flow definitions to the repository	No — reads only
`scripts/redact-signed-urls.ps1`	Strips `sig=` tokens from an export before it is committed	No
`scripts/setup-sharepoint.ps1`	Provisions lists and columns	Yes

Script	Does	Sends anything?
`scripts/provision-sharepoint-fields.ps1`	Adds columns to existing lists	Yes

Run `redact-signed-urls.ps1` on every export before committing it. An export carries the trigger URLs, and a `sig=` token in the repository is a published credential.

---

12 · Troubleshooting

12.1 The flow runs, the row is written, and the page shows an error

Cause: the CORS origin. The browser refused the response.

Check: browser devtools → Console. A message naming `Access-Control-Allow-Origin` confirms it.

Fix: §4.1. The `Response_*` action of that flow still carries `https://your-host`, or carries an origin that does not exactly match the page's — scheme, host and port must all match, and there must be no trailing slash.

12.2 Every internal endpoint answers `401`

Cause: `DGO_UserDirectory` has no row for the caller, or the row's `Status` is not exactly `active`, or `DGO_RoleCatalogue` has no row whose `RoleId` matches the directory row's `Role` with `Active = 1`.

Check, in order:

1. Is there a row in `DGO_UserDirectory` with `Email` exactly matching, lowercased and trimmed?
2. Is its `Status` exactly `active` — lowercase, no trailing space?
3. Is there a `DGO_RoleCatalogue` row whose `RoleId` equals that row's `Role`?
4. Is that catalogue row's `Active` set to `1`?

A `403` rather than a `401` means the caller was found but the role lacks the permission — check `PermissionsJson` and `AllowedRoutesJson` on the catalogue row.

12.3 A correct verification code is refused

Check, in order:

1. Is `Portal_Verify_Confirm` pasted with the current package? The defect where both halves routed to `generate` was fixed on 2026-08-22 — an older body mails a fresh code instead of verifying.
2. Is the `Portal OTP Codes` row's `Consumed` already true? Five wrong attempts consume the challenge.
3. Has `Expires_At` passed? The window is five minutes.
4. Does the `email` in the confirm request match the one VERIFY was called with, exactly?

12.4 An endpoint answers `500` after working for weeks

Cause, most likely: a hot-path list crossed 5,000 items with an unindexed filter column.

Check: `Portal Rate Limits` item count. If it is near or over 5,000, §3.1 was skipped or the index was removed.

Fix: index `Title` per §3.1. If SharePoint refuses because the list is already over the threshold, delete rows whose `WindowStartUtc` is older than a day — they are spent windows and nothing reads them — until the list is under 5,000, then index.

12.5 A submission reports success but the attachments never arrive

Check: did the response's `uploads[]` carry one element per declared attachment? An empty array when attachments were declared means the tickets were not issued, and the client has nothing to redeem. Check `Portal Upload Tickets` for rows against that `SubmissionRef`.

If tickets exist but `Portal Attachments` is empty, UPLOAD is refusing them — check `Portal Upload Tickets/Redeemed` and the size comparison in §5.5.

12.6 An expression shows as literal text at run time

Cause: it was pasted into the dynamic-content box rather than the expression editor.

Fix: click **fx** (or the **Expression** tab) and paste there. This applies to every value in the packages beginning with `@`.

12.7 The designer rejects the paste

Check:

1. Is this the **modern** designer? The classic designer rejects the clipboard format.
2. Was the **entire** file contents pasted, including the outermost braces?
3. Was a previous partial paste left on the canvas? Delete everything below the trigger and paste again.

12.8 A pasted action shows a broken-connection badge

Cause: the pasting account cannot use the connection the package names, or the connection has been deleted.

Fix: open the action, pick a connection you own. This is expected on the three mail actions (§4.2) and unexpected on the SharePoint actions — if a SharePoint action is flagged, your account does not have access to `3f1943c5955a4cb8b301e8f22f2b590d` and you should pick your own SharePoint connection.

---

13 · Sign-off record

Complete this as the guide is executed. The two decisions must be recorded **before** Phase 3.

Decisions

Decision	Value	Decided by	Date
§4.1 Portal CORS origin			
§4.2 Sending mailbox / Office 365 connection			
§9.2 Rotate the six trigger URLs?			

Phase completion

Phase	Item	Acceptance test	Done by	Date
0	Write access granted, `ba7c94a` landed	`npm run wiring` reads 8/59		
0	Dry run against the tenant	Prints flow name and action count, no error		
1	`Portal Rate Limits/Title` indexed	Shown in **Indexed columns**		
1	`Portal Verification Proofs/Title` indexed	Shown in **Indexed columns**		
1	`Portal Support Cases/Title` indexed	Shown in **Indexed columns**		
1	`DGO_RoleCatalogue` seeded	`npm run test:roles` passes; rows present with `Active = 1`		
1	`DGO_UserDirectory` populated	One row per officer, `Status = active`		
3	`Portal_Verify` pasted	§5.3 confirmations		
3	`Portal_Verify_Confirm` pasted	§5.3 confirmations		
3	`Portal_UBMISSION_ECM_DOCS` pasted	§5.4 confirmations		
3	`Portal_UPLOAD_ECM_DOCS` pasted	§5.5 confirmations		
3	`Portal_ECM_DOCS_STATUS` pasted	§5.6 confirmations		
3	`Portal_ECM_DOCS_SUPPORT` pasted	§5.7 confirmations		
3	`Portal_ECM_DOCS_WRITEBACK` created and pasted	§5.8 confirmations		
4	`DGO_OTP_GENERATE` pasted	§6.3 confirmations		
4b	`DGO_OTP_VERIFY` pasted	§6.3 confirmations		
4	`DGO_FETCH_ALL` pasted	§6.4 confirmations		
4	`DGO_REFERENCE_DATA` pasted	§6.4 confirmations		
4	`DGO_GET_DOCS` pasted	§6.4 confirmations		

Phase	Item	Acceptance test	Done by	Date
4	`DGO_SINGLE_ASSIGNMENT` pasted	§6.5 confirmations		
4	`DGO_BULK_ASSIGNMENT` pasted	§6.5 confirmations		
4	`DGO_DYNAMIC_GLOBAL_ACTIONS` pasted	§6.6 confirmations		
5	Workflow ids recorded	Seven pairs, ids only		
5	`config.local.js` completed	All seven slots filled		
6	Citizen journey	All 12 steps in §8.1		
6	Officer journey	All 7 steps in §8.2		
6	Telemetry check	§8.3 — no secret in any row		
7	Seven API keys rotated	Old keys return `401`		
7	Trigger-token decision taken	Recorded above		

Deferred, with owner

Item	Blocked on	Owner
§10.1 Fifteen `DYNAMIC_ACTIONS` operations answer `501`	Column evidence from the tenant	
§10.2 SHA-256 upload digest	Licensing answer for `Execute JavaScript Code`	
§10.3 Seventeen flows with no stated purpose	Estate knowledge	

Appendix A — The flow packages

The packages are the identification packages for the adopted flows, listed with their target flow, scope and action count in `packages/identification-phase3/README.md` and `packages/identification-phase4/README.md`, which their builders generate. The fourteen replacement packages this appendix listed are retired (2026-10-06, `registers/flow-retirement.json`).

Appendix B — Standing constraints

These hold regardless of what any other document says. Each is recorded with its reason.

Four of these are about handling and are enforced by the secret ratchet and the exposure scan. **The first three are about speech, and until 2026-08-31 nothing enforced them** — which is why item 19 could rewrite seven documents and miss an eighth, and why two live over-claims survived until a test went looking. `npm run test:claims` scans every markdown file outside the verbatim

corpus and the audit record, and each of its three rules is mutation-tested against the sentence it exists to reject.

Constraint	Reason	Enforced by
Do not claim the one-time code comparison is constant-time.	WDL has no bitwise operators. `equals()` cannot be made constant-time.	`test:claims`
Do not claim one-time codes are hashed.	Same limitation. They are stored in plaintext; the protection is the lifecycle (SC-003).	`test:claims`
Do not claim the upload SHA-256 is verified.	It is not computed. See §10.2.	`test:claims`
A `sig=` trigger token is a bearer credential.	It must never reach the repository, a ticket, an email or a chat message.	`test:secrets`, `test:exposure`
Redaction is not rotation.	A redacted key in a document is still live at the provider.	judgement — open items 7 and 22
Audit rows carry the action and the body length, never the body.	The body may contain anything a citizen typed.	judgement (its check retired with the replacement packages, 2026-10-06)
Outbox receipts never carry the code.	A receipt proves a code was sent; carrying it would defeat the point of expiry.	judgement (its check retired with the replacement packages, 2026-10-06)
The rate limit is a fixed window, not sliding.	D8, accepted trade-off. Not a defect to raise.	
Portal responses are flat; internal responses are enveloped.	Two clients, two contracts. Checked per flow by the simulated runs in `npm run test:idphase3` and `test:idphase4`.	
`Global Tracking Queue` **has no** `Status` **column.**	A task's status is `Progress`. A reference's status is `DGO DIGITAL OPS/Status`.	
Submissions already in `NITDA_Central_Registry` **stay put.**	Standing instruction. Handled after provisioning concludes; nothing is migrated mid-flight.	

Part 3 · Portal tenant runbook — the console steps for portal and endpoint commissioning

> **This document carries the steps for portal and endpoint commissioning, and it is the only one that does.** Nine console scripts are run from here. Every other document in this repository may name a step and say that it lives here; none of them restates it. There is no precedence rule to apply, because there is nothing to have a conflict with.

> This part is reached through the one runbook that contains it, on a desktop or a phone alike; **Part 1 — the cutover** is the command-line path that gets you here. The open-item register is `PLAN.md`.

Every console artifact of this domain, in the order the runbook runs them

Nothing in this table is typed: the artifacts come from `docs/reference/document-ownership.json`, the section from the owning runbook's own headings, the verdict from `PRODUCTION_READINESS_REGISTER.json`. The governance domain has had this view since `governance/GOVERNANCE-STATUS.md`; the portal domain's equivalent was a join three files wide that each reader had to make by hand. It stood in `IMPLEMENTATION_WALKTHROUGH.md` until 2026-09-24 and moved here, into the part that actually runs these artifacts, when the deployment documents were consolidated. `tests/portal-script-status.test.mjs` holds it to all three sources.

Artifact	Runbook section	What the register says	Replaces
`harvest-flow-metadata.browser.js`	§0.6 — Capture what is inside each flow, for the audit record	no item names it	reading 39 flow definitions one designer tab at a time
`provision-sharepoint-fields.browser.js`	§2.2 — Provision columns	`ITEM-23` APPLIED_UNVERIFIED · `ITEM-39` RESOLVED	a PnP PowerShell session and its app registration
`harvest-trigger-urls.browser.js`	§3.2 — Get the worklist	no item names it	opening 20 flows and copying 25 URLs by hand
`acceptance-internal-endpoints.browser.js`	§6.3 — The browser acceptance run	no item names it	—
`diagnose-deployed-config.browser.js`	§7.4 — Confirm the deployed config is served	no item names it	—
`verify-deployed-package.browser.js`	§8.1 — Verify the deployed package is the package that was built	no item names it	—
`otp-cutover-tests.browser.js`	§9.0 — The OTP verify patch, and the cutover it belongs to	no item names it	—
`set-content-approval.browser.js`	§10 — Content approval — item 6.6	`ITEM-41` RESOLVED	—

Read that third column exactly. It is the status of the items an artifact serves — never proof the artifact was run. `APPLIED_UNVERIFIED` is the register's own term for *executed against the tenant, with a run record naming the evidence*, and deliberately not the same as resolved. `no item names it` is what a verifier looks like: it closes no item, it tells you whether one closed.

Artifacts recorded rather than owned

Artifact	Instructed by	What the register says
`scripts/harvest-trigger-urls.bookmarklet.txt`	**Part 8 of this runbook**	`CFG-1` DISCHARGED_UNTRACKABLE · `CFG-2` DISCHARGED_UNTRACKABLE

`npm run prove:tenant` is Node in this repository rather than a console artifact. It replaces six separate manual round-trips, for ITEM-11, ITEM-12, ITEM-6, ITEM-44 and ITEM-56.

Regenerate the generated sets when the register changes:

```
npm run harvest:console      # rebuilds the console harvester from the endpoint register
npm run harvest:bookmarklet  # rebuilds the phone harvester from the same register
npm run console:apply        # rebuilds one console snippet per flow package
```

Every remaining open item, in the order they must be done, with the exact command or click for each. Nothing here is deferred to a later document. Where a step needs a value only the tenant can give, the step says how to obtain it rather than leaving a blank.

Who runs this. One person with: Power Automate maker access in environment `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1`, Site Owner on the four SharePoint sites listed in §2.1, write access to both GitHub repositories, and a machine with Node 22 or newer (`package.json` sets `engines.node: ">=22"`) and `git`.

How long. §1 is ten minutes. §3 is the long one — 39 flows, roughly two minutes each, about 90 minutes with checks. §4–§9 are under an hour. Budget half a day, and do not split §3 across days: a half-rotated estate has some flows on new URLs and some on old, and the configuration file cannot describe both at once.

The one rule. Never paste a signed trigger URL into a ticket, an email, a chat message, or any file except `config.local.js`. The `sig=` parameter *is* the authentication. Possession of the URL alone authorises invoking the flow.

Contents

§	Phase	Closes
1	Repository lineage	the `main` divergence
2	SharePoint estate	prerequisite for everything
3	Rotate 39 trigger URLs	item 6.4, warning in `npm run commission`

§	Phase	Closes
4	Configure both surfaces	item 6.2
5	Commissioning gate	blocks §6 until clear
6	First live invocation	**item 6.1**
7	Deploy both front ends	item 6.10
8	Verify against the deployed host	item 6.9's runtime half
9	Trigger hardening	item 6.3
10	Content approval	item 6.6
11	Close-out	the brief's §6 list

---

§0 · Prerequisites — five steps that were filed under rationale

These five were the only executable steps in Part 2 of this runbook, a 1,925-line document of which they were 3%. They are moved here because the steps for this domain belong in one place, and Part 2 of this runbook is where you go for *why*, not *what to do*.

0.1 Grant repository write access and land the pending commit — retired

Retired 2026-10-05. The pending commit carried the `Portal_Verify` (86897b2f) export. `Portal_Verify` is retired; `VERIFY` runs on `CG_Portal_Verification_Endpoint` (ba47f79b), whose definition is in `docs/folio-Master-Flows-and-Endpoints/contracts/deployed/` (`FLOW_ADOPTION.md`). Nothing to do.

0.2 Prove tenant connectivity with a dry run

```
cd <repository root>
.\scripts\update-flow-definition.ps1 `
    -FlowId "86897b2f-9770-4efa-8486-2642f24bb947" `
    -WhatIf
```

`docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/remediation/patched/README.md` names the environment-id argument and authentication method your environment needs.

0.3 Seed the role catalogue — required before any internal flow works

```
npm run seed:roles      # generates role-catalogue-seed.json from config/rbac.config.js
npm run test:roles      # verify it
```

Load the generated rows into `DGO_RoleCatalogue` (`f675598b-271d-4200-8d75-2597aad4057f`, on `DGO_ECM_GOVERNANCE`). Columns: `Title`, `RoleId`, `Persona`, `PermissionsJson`, `AllowedRoutesJson`,

`Active`, `Version`, `CanAssignRoles`, `CanManageSettings`, `CanViewAudit`.

`Active` **must be** `1` **for every row you intend to be live** — the gate filters on `Active eq 1`.

0.4 `CG_Writeback_Endpoint` — retired

Retired 2026-10-05. `WRITEBACK` runs on `CG_Portal_Writeback_Endpoint` (bde13d27), which exists in the tenant and is the one flow the endpoint register names. `CG_Writeback_Endpoint` (21d4bfd3) is not to be created or exported. Nothing to do.

0.5 Record the workflow ids

For each of the seven portal flows, open it in Power Automate, open the trigger, and read the `workflows/<32-hex>/` segment of the HTTP POST URL. Record the pairs.

Record the id only, never the whole URL. The full URL carries a `sig=` parameter which is a bearer credential — anyone holding it can invoke the endpoint. It must not reach the repository, a ticket, an email or a chat message.

0.6 Capture what is inside each flow, for the audit record

§0.5 gets you the identity of each flow. This gets you its contents: the definition, the properties, the owners and the trigger shape, as datasets an auditor can open in Excel. It is evidence-gathering, not commissioning — nothing here changes a flow — so it can be run at any point, and is worth running before §3 so there is a record of each flow as it was before its trigger was rotated.

1. Sign in to `https://make.powerautomate.com` and open any flow you own, so the tab holds a token for the Flow service.
2. Devtools (F12) → Console. Paste `scripts/harvest-flow-metadata.browser.js` entire.
3. **Navigate inside the portal — do not press F5.** Go to My flows and open a flow, or switch between two flows. The script does not know which host serves this tenant and will not guess one; it reads the hostname off a request the portal itself makes, and navigating is what makes the portal make one. It then reports what it would collect and fetches nothing.

   A reload would destroy the script along with the rest of the page's JavaScript, before the request it is waiting for is ever made. The portal is a single-page app, so moving around inside it calls the API without reloading. If you do reload by accident, just paste again.
4. Type `flowHarvest.run()`. It reads every flow you can see — or set `CONFIG.FLOW_IDS` first for a specific list — and downloads one JSON bundle plus a CSV per dataset.

The definition and properties are split across numbered `PartNo` rows because a flow definition is far longer than the 32,767 characters a spreadsheet cell holds. Sort by `FlowName` then `PartNo` and concatenate to reassemble one exactly.

It does not fetch trigger URLs, and you should leave it that way. `CONFIG.INCLUDE_TRIGGER_URLS` exists and defaults to `false`. Every other field it collects is ordinary metadata; a `sig=` value is a bearer credential, and the moment that flag is `true` the downloaded files are credentials sitting in your Downloads folder, which — unlike `~/dgo-values.txt` — nothing git-ignores. Any signature

already embedded inside a definition is redacted before it reaches a cell. If you need the trigger URLs, §3.2 is the step that fetches them, into the one file built to hold them.

---

§1 · Repository lineage — decide once, then never again

1.1 The situation, stated exactly

`dgoeaa/ecm_docs_dev` has two lineages that **share no common ancestor**. `git merge-base origin/main HEAD` returns nothing.

	`main`	`claude/system-remediation-gaps-ahpmsy`
tracked files	679	1,052
`docs/deployment/RUNBOOK.md`	absent	present
`scripts/verify-portal-wiring.mjs`	absent	present
`docs/folio-Master-Flows-and-Endpoints/contracts/deployed/`	absent	present
files the other lacks	34	407

The working branch carries every remediation in this programme. `main` carries 34 files the working branch has never had — most importantly `07-portal-provisioning.flow.json` and its code-view twin, added by PR #24.

1.2 The decision, and it is not a coin toss

Make the working branch the trunk, after porting `main`'s 34 unique files onto it. The working branch is a superset of the estate in every direction that matters — it holds the flow exports, the verifiers, the wiring specification and the review brief, none of which exist on `main`. Rebasing 62 commits of remediation onto `main` would mean re-deriving all of it against a tree that cannot even run `npm run wiring`.

1.3 Do it

```
cd ~/ecm_docs_dev                       # your clone
git fetch origin
git checkout claude/system-remediation-gaps-ahpmsy
git pull --ff-only

# 1. List exactly what main has that this branch does not.
git ls-tree -r --name-only origin/main   | sort > /tmp/main.txt
git ls-tree -r --name-only HEAD          | sort > /tmp/ours.txt
comm -23 /tmp/main.txt /tmp/ours.txt > /tmp/port.txt
wc -l /tmp/port.txt                      # expect 34

# 2. Read the list before porting it. Anything you do not want, delete from /tmp/port.txt
now.
cat /tmp/port.txt

# 3. Port them, preserving paths.
while IFS= read -r f; do
  mkdir -p "$(dirname "$f")"
  git show "origin/main:$f" > "$f"
done < /tmp/port.txt

# 4. The gate must still pass with them in.
npm test

git add -A
git commit -m "Port the 34 files unique to main onto the remediation lineage

main and this branch share no common ancestor. This branch is the superset in
every direction that matters, so it becomes the trunk; these 34 files are what
main held that it did not, chiefly 07-portal-provisioning.flow.json from PR #24."
git push -u origin claude/system-remediation-gaps-ahpmsy
```

1.4 Promote it to `main`

On GitHub → **Settings → Branches → Default branch** → switch to `claude/system-remediation-gaps-ahpmsy` → **Update**. Then rename it: **Branches → ⋯ → Rename** → `main-remediated`, or leave the name and archive the old `main`:

```
git push origin origin/main:refs/heads/main-archived-2026-09-04
```

Keep the old `main` as `main-archived-2026-09-04`. Deleting it discards PR #24's review history.

Verify: `git ls-tree -r --name-only origin/HEAD | wc -l` returns 1,086 (1,052 + 34).

1.5 `internal_platform` — already done

Default branch `claude/internal-platform-package-1x0107` is at `b483d48`. Archive removed, `.gitignore` added, `package.json` + `tests/verify.mjs` + CI present, `npm test` 13/13. **No action.** Confirm with:

```
git ls-tree -r --name-only origin/claude/internal-platform-package-1x0107 | grep -c
'\.zip$'   # 0
```

---

§2 · SharePoint estate

2.1 The four sites

Site	Holds
`https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`	`DGO_UserDirectory`, `DGO_RoleCatalogue`, `DGO_AuditLog`, `DGO_AccessScopes`
`https://nitdanigeria.sharepoint.com/sites/Global_Digital_Documents_Centre`	Portal Rate Limits, OTP Codes, Verification Proofs, Support Cases, Sequence Counters, Audit Events, Flow Telemetry, Outbox Receipts
`https://nitdanigeria.sharepoint.com/sites/NEDMS`	Portal Registry, Portal Attachments, Portal Status Timeline, Portal Upload Tickets, `NITDA_Central_Registry` (library)
`https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING`	Global Tracking Queue, DGO Digital Ops, Comments, Categories, Departments, Flow Configuration

You need **Manage Lists** on all four. Site Owner suffices; tenant admin is not required.

2.2 Provision columns

PowerShell path (repeatable, needs PnP.PowerShell + an Entra app registration):

```
./scripts/setup-sharepoint.ps1 -SiteUrl
"https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE" -WhatIf
./scripts/setup-sharepoint.ps1 -SiteUrl
"https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE"
```

Browser path (no install, uses the session you are already signed into) — **use this if PnP is not set up**:

1. Sign in to `https://nitdanigeria.sharepoint.com`, open any page on any of the four sites.
2. F12 → Console.
3. Paste the whole of `scripts/provision-sharepoint-fields.browser.js`, press Enter.
4. It prints a **DRY RUN** table. Read it.

5. Edit `DRY_RUN = false` at the top of the pasted text, paste again.

It never creates a list and never re-types an existing column. All 14 lists already exist and are addressed by GUID, so a renamed list still resolves.

2.3 Seed the role catalogue and directory

```
npm run seed:roles          # writes the six DGO_RoleCatalogue rows from
config/rbac.config.js
```

Then populate `DGO_UserDirectory` by hand — one row per officer:

Column	Value
`UserId`	any stable id, e.g. `u-1042`
`FullName`	officer's name
`Email`	their tenant sign-in address, **lower case**
`Directorate` / `Department`	free text
`Role`	one of the six catalogue values
`Status`	`active` — the flows filter on this exact string
`AccessScope`	JSON array, e.g. `["Registry"]`, or `["all"]`

This is not optional. Every internal flow resolves its caller against this list. An officer absent from it, or with `Status` anything but `active`, is refused 401 by every endpoint.

2.4 Preflight — retired

Retired 2026-10-06 with the DGO_* replacement packages whose lists it checked (`registers/flow-retirement.json`). The identification packages' tests check every column they write against the Phase 1 list check (`npm run test:idphase3` , `npm run test:idphase4`).

---

§3 · Rotate all 39 trigger URLs

3.1 Why this is first, and why all of it

43 distinct signatures across 28 tracked files are in git history. Anyone who has ever cloned this repository holds every one. **Deleting a file revokes nothing.** Only regenerating the trigger in Power Automate does.

What is already behind you, and what is not. `npm run rotation` reports **ROTATED** for that published corpus, and ITEM-22 is RESOLVED on the same evidence: all 25 contract keys now resolve to workflows other than the ones those signatures belong to, so none of them authenticates an endpoint the platform calls. What the report states it cannot establish is that a flow *outside* the 25 was regenerated — and the worklist below is 39 flows. So the 25 keys are not what

this section is for; the other flows are. Do not regenerate a contract key's trigger while §4 is configuring it: that revokes the URL the configuration is about to carry.

Three of them are worse than the rest: they were inside `Platform frontend review.zip` at the root of `internal_platform`, which *is* the deployed public site. They are removed from the branch now but remain in history and in every clone. **Rotate those three first.** They are in `_ds/nitda-design-system-…/_ds_bundle.js` — open the archive from git history to identify which flows they belong to:

```
cd ~/internal_platform
git show 7ef5e89:"Platform frontend review.zip" > /tmp/pfr.zip
unzip -p /tmp/pfr.zip '_ds/*/_ds_bundle.js' | grep -oE 'workflows/[a-f0-9]{32}' | sort -u
```

3.2 Get the worklist

Harvest the trigger URLs from the tenant rather than transcribing them. Thirty-nine URLs copied by hand is thirty-nine chances to transpose a character in a 43-character signature, and a transposed signature fails as 401 with no indication of which character.

1. Sign in to `https://make.powerautomate.com` and open any flow you own.
2. Devtools (F12) → Console. Paste `scripts/harvest-trigger-urls.browser.js` entire.
3. **Now navigate inside the portal without reloading** — open My flows, or switch between two flows. The script reads the API address and the token off the portal's own traffic rather than guessing either, so it needs to see one call go past. **Do not press F5**: a reload destroys the script before the traffic it is waiting for happens. If you do reload, paste it again.
4. It prints the host and path it observed and the flows it will fetch, and changes nothing. Read that, then type `dgoHarvest.run()`.
5. Save the printed block as `~/dgo-values.txt`.

There is a second route, and it needs no API call at all. Opening a flow's details page makes the portal fetch that flow's HTTP POST URL in order to display it — and that answer goes past the same hook, so the URL is banked without the script asking anyone for anything. It is filed under the workflow id the URL names and matched to a contract key only by that id, so opening the wrong flow cannot produce a wrong credential.

That matters because the fast route can be refused outright: a session can be authorised for the maker UI and refused at the management API, which is what this tenant produced — twenty-five 401s beside a portal that was working perfectly well. When that happens, no amount of re-observing the token helps. Run `dgoHarvest.pending()` for the list of flows to open, open each one, and run `dgoHarvest.run()` again. Every key answered this way is verified exactly as a fetched one is.

`dgoHarvest.status()` says how many keys are already held and how many requests have gone past the hook. `dgoHarvest.wipe()` drops the banked URLs — they are credentials, and they live on `window` until the tab is closed.

Every URL is checked before it is kept. A flow is addressed by its *tenant flow id* and the URL that comes back must name the *application workflow id* the register holds for that key — two different identifiers, and neither is derivable from the other (`docs/folio-Master-Flows-and-Endpoints/registers/flow-identity-crosswalk.json` is the mapping). A flow that answers for a different workflow is discarded and named, never written. Seven keys have two candidate flows and this is how the live one is chosen; two keys have none recorded, and are found by display name and then verified the same way.

If any key fails, nothing is printed. A half-complete values file wires some endpoints and leaves the rest answering 401 with no pattern to it. `serves workflow X, not Y` is not a permissions problem — it is the wrong flow, and the register or the crosswalk has moved.

If EVERY key answers 401, that is one token, not twenty-five permissions problems — the script says so rather than leaving you to read a list of 25. The token is read off portal traffic and can go stale in a long-lived tab. Run `dgoHarvest.forget()`, click into a flow so the portal calls its API again, then `dgoHarvest.run()`. If it persists, reload the portal, wait until **My flows** lists, and paste the file again. **If it still persists after that, stop trying to fix the token** — the session is authorised for the maker UI and refused at the management API, and the second route above is the answer.

A `sig=` value is the authentication. Possession alone authorises invoking the flow. Never paste a harvested URL into a ticket, an email, a chat message, or any file other than `config.local.js`.

```
cd ~/ecm_docs_dev
npm run rotation
```

39 rows, one per workflow. Rotation is **per flow**, not per file — regenerating one trigger invalidates every copy of its URL at once. Two rows carry a warning that two *different* signatures exist on one flow: an older trigger URL is still live, and one regeneration revokes both.

3.3 The loop, per flow

For each of the 39 workflow ids:

1. Power Automate → **My flows** → open the flow. Find it by id: `https://make.powerautomate.com/environments/Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1/flows/<workflowId>/details`
2. **Edit** → click the **When an HTTP request is received** trigger.
3. **⋯ → Settings → Regenerate** (or delete and re-add the trigger if your tenant does not offer regenerate; re-adding produces a new signature).
4. **Save**.
5. Copy the **HTTP POST URL** shown on the trigger card. Newer designer builds label the same field **HTTP URL**. The label is the field's name, not the method: `CG_Upload_Endpoint` restricts its trigger to **PUT** and still shows its URL there. Copy the whole string, query included — the `sig=` parameter is the credential, and a URL truncated before it is not a working endpoint.
6. Paste it into your values file (§4.1) against the right key **immediately** — do not batch, and do not keep a list of URLs anywhere else.

Order matters at the end, not the start. Do not revoke an old signature until §6 has confirmed the new one answers. The sequence is: regenerate → configure → confirm → *then* the old one is already dead, because regenerating replaces it.

3.4 Which flows carry which keys

16 flows serve 18 internal keys. Where two keys share a heading they are the **same flow**, distinguished by the fixed `action` string in the body — rotating that flow rotates both keys.

Reconciled against `docs/folio-Master-Flows-and-Endpoints/registers/endpoint-register.json`, the tenant's own export. Every id below replaced a different one this repository previously recorded — all 25 keys moved. Regenerate with `npm run reconcile`; never hand-edit.

Workflow id	Flow	Serves
`c4c26f93ba1e4d7db5247536c30cdc11`	`IP_Get_Docs_Endpoint`	`FETCH_ACTIVITIES`, `GET_DOCS`
`d5e4b3da41e34819b3f953d2acbc2dd7`	`IP_Dynamic_Global_Actions_Endpoint`	`SUBSIDIARY_ACTIONS`, `DYNAMIC_ACTIONS`
`aa662769f13a4666bfadf3039cd8d247`	`FETCH_ALL`	`FETCH_ALL`
`885fad5ec91f460490e8c772d5d10cb8`	`IP_Reference_Data_Endpoint`	`REFERENCE_DATA`
`d2c773332f6f4fbf97ee4a5baa01f70b`	`IP_Retrieve_Email_Attachment_Endpoint`	`FETCH_EMAIL_ATTACHMENTS`
`72acfc6a694f414f9fe39c939ead7f08`	`Single_Task_Assignment`	`SINGLE_ASSIGNMENT`
`a705e453f922457c95ca19f32731e4d7`	`BULK OPS DATA RETRIEVAL HTTP`	`BULK_ASSIGNMENT`
`378b491ec39a4beca886ddec250e6961`	`IP_SEND_EMAIL`	`EMAIL`
`a942d230337c4ddfa9a386e92bbd048b`	`IP_Create_Email_Assignment_Endpoint`	`EMAIL_RELATED_TASK`
`3f018fea1031490fb73dff6a8d6341f2`	`AI_Document_Processing`	`AI_EMAIL_ANALYSIS`, `AI_DOC_ANALYSIS`, `AI_CHAT`
`b372d45e4b2a47d88b8e8b032da67fcd`	`IP_OTP_VERIFY`	`OTP_GENERATE`, `OTP_VERIFY`
`88401ce1f44148539c0da53c9491f8e6`	`IP_SCAN_INTAKE`	`SCAN_INTAKE`

Portal, 7 keys, 7 flows:

Flow	Key	Workflow id
`CG_Submission_Endpoint`	`SUBMISSION`	`1041ed37ce924e3c886d891f23e8142c`
`CG_Upload_Endpoint`	`UPLOAD`	`62fe121e5a95416bb91275e43dd0e37e`
`CG_Portal_Support_Endpoint`	`SUPPORT`	`052013da80724713a4285edee72ccb4a`
`CG_Portal_Verification_Endpoint`	`VERIFY`	`c0d58004c53348539d0d5aeaf49dcded`
`CG_Portal_Verification_Confirmation_Endpoint`	`VERIFY_CONFIRM`	`b270894ff410499ebbc090e32ffe899b`
`CG_Portal_Status_Check_Endpoint`	`STATUS`	`34dd28b4a5664927b66e581c74a0ab94`
`CG_Portal_Writeback_Endpoint`	`WRITEBACK`	`abf3a3ca53e64ba58c9ce5933e4e97e3`

Every id above is a 32-hex trigger workflow id from the tenant's register — the previous "internal name (dashed)" ids this document carried, and the note that SUBMISSION, SUPPORT and UPLOAD had never been captured, are both superseded. All seven are now on record.

3.5 Also rotate the API keys

Any AI-provider key used by `AI_CHAT`, `AI_EMAIL_ANALYSIS`, `AI_DOC_ANALYSIS` and `Instant OpenRouter AI Chat`. These sit in the flows' own actions, not in this repository. Regenerate at the provider, then update each flow's action.

---

§4 · Configure both surfaces

4.0 The short path: two URLs, one blocker

> **If you want this spelled out command by command — which terminal, which folder, what each output should say, what to do when it doesn't — read Part 1 — the cutover of this runbook instead. It assumes no prior knowledge, covers exactly this section, and is written to be followed on a phone as readily as a desktop. The summary below assumes you have done this before.**

§3 rotates all 39 flows and is what production needs. It is not what the *current* blocker needs. `npm run commission` today reports exactly one:

```
Public portal: 2 required endpoint(s) unwired
  Correspondence cannot flow end to end without: SUBMISSION, UPLOAD.
```

Both flows exist and both triggers are already correct. Nothing has to be built, and nothing has to be regenerated to clear this. What is missing is only their URLs, which are credentials and are therefore recorded nowhere in this repository — they are copied by hand, once.

Key	Flow	Open it by id	Trigger method	Trigger auth
`SUBMISSION`	`CG_Submission_Endpoint`	`de9ef13b-ae4c-42b0-9afa-20e71a180759`	`POST`	`All` — anonymous
`UPLOAD`	`CG_Upload_Endpoint`	`df82e331-b800-4a8d-996e-d2b2ca846c77`	`PUT`	`All` — anonymous

`All` is correct here and must not be "fixed" to `Tenant`. These are the two flows a member of the public calls from the portal, with no tenant account; `Tenant` would refuse every real caller. That also means each flow is the only thing standing between a stranger and your data, which is what §5's posture choice is about.

Per flow:

1. Open `https://make.powerautomate.com/environments/Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1/flows/<id>/details` with the id from the table. Confirm the flow name matches before copying anything — an id pasted one character short opens a different flow, or none.
2. **Edit** → click the **When an HTTP request is received** trigger to expand it.
3. Copy the value of **HTTP POST URL** (newer designer builds label it **HTTP URL**; the label is the field's name, not the method — `CG_Upload_Endpoint` is a `PUT` and shows its URL there). Take the entire string including everything after `?`. The `sig=` parameter is the credential; a URL cut short of it is not an endpoint.
4. Paste it straight into `~/dgo-values.txt` (4.1). Do not stage it in a chat window, a ticket, an email or a note first — every one of those is a place the credential then lives permanently.
5. Leave the flow. **Do not** press Regenerate. Regenerating is §3's job; doing it here revokes the URL any currently deployed portal is using, for no gain, and you would have to come back and copy the new one anyway.

Then go to 4.1 for the file, 4.2 for the command, 4.3 to confirm. The whole of §3 is still owed before real correspondence — 4.0 gets you a working pilot to test against, not a rotated estate.

4.1 Generate the values file

```
umask 077
npm run values:template ~/dgo-values.txt
```

It writes all 24 keys, each with its **complete trigger URL already filled in** from `docs/folio-Master-Flows-and-Endpoints/registers/endpoint-register.json` and ending at a bare `sig=`. You add the 43-character signature and nothing else — the host, routing segment and workflow id are already reconciled against the tenant, and retyping them can only introduce an error.

Per key, without opening an editor:

```
npm run values:sign -- ~/dgo-values.txt FETCH_ALL     # paste, Enter, Ctrl-D
```

It accepts a whole URL too and lifts the signature out. It refuses a short paste, an unknown key, and a re-run over a key already signed (`--replace` overrides). Then:

```
npm run check:values -- ~/dgo-values.txt
```

Expected: ✅ `25 value(s), all with a complete 43-character signature.` It prints no URL, host or signature, so its output is safe to paste anywhere.

Never commit this file. Delete it when §6 passes.

4.2 Generate both config files

```
npm run setup -- --values ~/dgo-values.txt --force
```

Writes `config/config.local.js` and `document-portal/config.local.js`. Both are git-ignored.

`--force` **is not optional here.** `setup` never overwrites an existing `config.local.js` without it, and if you have ever run `npm run setup` in this tree, both files already exist — so without `--force` every value you just pasted is discarded. `setup` now refuses that combination outright: values supplied plus a file it may not write is **exit 2** with the corrected command. Older revisions of this page omitted `--force` here, and that command silently changed nothing while exiting 0.

`--force` **replaces the whole file, so the values file must be complete.** A key absent from `~/dgo-values.txt` is written empty — it is not carried over from the previous file. That is why §4.1 lists all 24 keys: after rotation the values file is the only source, and anything you leave out is silently unwired. Check the counts in 4.3 rather than trusting the exit code.

`--recover` **is retired and refuses to run.** It read URLs out of this repository's reference corpus. The rotation revoked every one of those signatures, and reconciliation against the register showed the workflows behind them superseded — `npm run check:config` reported 17 of 17 recovered keys as "points at the WRONG FLOW". It produced a config that looked complete and answered 401 on every call, which is worse than no config at all.

Nothing replaces it because nothing needs to: `npm run values:template` (4.1) writes all 24 keys, so `--force` alone is correct and there is nothing left for a fallback to supply.

4.3 Check them before going further

```
npm run check:config            # internal
npm run check:config:portal     # portal
```

These catch an empty value, a placeholder left in, a non-HTTPS URL, a malformed signature, and — importantly — a URL still carrying a **published** signature, i.e. one you did not actually rotate. That last check is why §3 comes before §4.

Read the counts, not just the verdict. Expect `17/18` internal after 4.0 (`SCAN_INTAKE` is a new flow with no URL yet) and `2/7` portal.

`check:config:portal` **will report four problems after 4.0, and that is correct.** `STATUS`, `VERIFY`, `VERIFY_CONFIRM` and `SUPPORT` come back "empty — the platform cannot call this endpoint at all", because they are. They are features — status lookup, the verification pair, the support form — not the correspondence path, so §5 does not treat them as blockers. Each closes later with one line in the values file and a re-run. What you must not see here is a problem against `SUBMISSION` or `UPLOAD`; those two are the pilot set, and anything reported against them is the blocker still standing.

---

§5 · Commissioning gate

```
npm run commission -- --posture pilot
```

Expect **0 blockers** — the gate prints `No automated blocker for pilot usage.` Warnings and manuals remain and are meant to; they are the published-signature warning, the posture warning, and the four things only your tenant can do.

If it still reports "not configured", `config.local.js` did not write — re-run §4.2 and read its output. If it still names `SUBMISSION` or `UPLOAD`, the values file did not reach the config: check the key spelling, and check that 4.2's command carried `--force`.

The `pilot` posture is the honest one for this estate: authentication is inert, caller identity is a client-asserted `userEmail` from `localStorage`, and RBAC is advisory. Editing one storage key escalates a viewer to `systemAdmin`. That gate does not sit between the page and the flows.

Do not run `--posture enforced` **and expect it to clear.** It requires each flow to verify the bearer proof itself, which is server-side work in Power Automate that this repository cannot do.

---

§6 · First live invocation — this is item 6.1

Nothing in this estate has ever been invoked. Every claim about behaviour is derived from reading definitions. This section is the only thing that changes that.

6.1 Read-only endpoints first

```
npm run verify:endpoints
```

Calls every configured read endpoint on both surfaces and reports status, latency, whether the body parsed, and which top-level keys came back against the shape the contract expects.

Expected: `200` for each, with the contract's keys present. Anything else is a real finding — record it, do not retry until it passes.

6.2 Writes, deliberately

```
npm run verify:endpoints -- --include-writes --json ~/endpoint-report.json
```

This creates real rows. Run it once, in a window where you can identify and remove what it creates.

6.3 The browser acceptance run

The above calls from Node. The platform calls from a browser, with the exact envelope `core/data-client.js` builds. Both matter.

1. `npm start` → open `http://localhost:8080`
2. F12 → Console
3. Paste `scripts/acceptance-internal-endpoints.browser.js`

It reads URLs from `window.DGO_CONFIG.endpoints`, so no signed URL passes through a file. It reports endpoint **keys** and response shapes only, never a URL.

6.4 Exercise SCAN_INTAKE specifically

It is the newest flow and the only one that takes raw bytes.

1. `http://localhost:8080/#/scan-intake`
2. Deposit a small PDF.
3. Expect: a reference of the form `NITDA-2026-00001`, a link into `/sites/NEDMS/NITDA_Central_Registry`, and a correspondence record with `channel: 'Registry'`.
4. Confirm in SharePoint: the file is in the library named `<reference>-<filename>`, and a row exists in `DGO_AuditLog` with `Event = audit:scan-deposited`.

If it returns 401: the depositing officer is not an `active` row in `DGO_UserDirectory` (§2.3), or the trigger is not `Tenant`-authenticated.

6.5 Exercise the portal end to end

1. `npm run serve:portal` → `http://localhost:8080`
2. Submit a correspondence with one attachment. Expect a `NITDA-` reference, the file in `/sites/NEDMS/Shared Documents/Portal Attachments`, and one row in the `Portal Attachments` list (a Generic List, not a library, so it cannot hold the file itself).
3. Track it by reference. Expect the public projection only.
4. Request a verification code; confirm the email arrives **with a code in it** — the estate has a recorded defect where `CG_Verification_Confirmation_Endpoint`'s `Send_Otp_Email` body was a literal empty `<div>`. If the mail is empty, that is the bug, and it is in the flow.
5. Write back with the proof. Expect a timeline row and, for `withdraw`, a status change.

6.6 Record what happened

```
cp ~/endpoint-report.json docs/folio-Master-Flows-and-
Endpoints/deployment/verification/endpoint-verification-$(date +%Y-%m-%d).json
git add docs/folio-Master-Flows-and-Endpoints/deployment/verification/
git commit -m "First live endpoint verification run against the tenant"
```

This is what closes item 6.1, and only this. Not a further reading of definitions.

---

§7 · Deploy both front ends — item 6.10

7.1 Build

```
npm run package
```

Writes `dist/dgo-internal-platform/` and `dist/dgo-document-portal/`, each with its `config.local.js`, a `PACKAGE_MANIFEST.json`, a `DEPLOY.md` and a standalone endpoint checker.

```
npm run check:package
npm run check:package:portal
```

7.2 The internal platform

`dgoeaa/internal_platform` **is** the deployed site (`https://activityweb.page.gd`). Copy the contents of `dist/dgo-internal-platform/` over the repository working tree, then:

```
cd ~/internal_platform
git checkout claude/internal-platform-package-1x0107
# copy files in, then:
npm test                      # must be 13/13 — it will refuse a committed signature
git status --porcelain | grep -i config.local && echo "STOP: config.local.js must not be
tracked"
git add -A && git commit -m "Deploy build <date>" && git push
```

`config/config.local.js` is git-ignored by the `.gitignore` added in `b483d48`. **Verify it is not staged before committing** — the command above does that. If it ever is, `npm test` fails on the credential check.

7.3 The portal

Deploy `dist/dgo-document-portal/` to wherever the portal is hosted. It is static; any web server serves it. `config.local.js` must sit beside `index.html`.

7.4 Confirm the deployed config is served

Item 6.10 exists because a live check returned an **empty endpoint set**. Confirm it is no longer empty:

1. Open the deployed URL.
2. F12 → Console.
3. Paste `scripts/diagnose-deployed-config.browser.js`.

It reports which keys the deployed page actually resolves. Expect 18 internal / 7 portal, none empty.

---

§8 · Verify against the deployed host

```
DGO_BASE_URL=https://activityweb.page.gd npm run test:smoke
```

The suite covers boot, accessibility entry points, all 29 routes, themes, and the portal — 150 tests across 10 files. Run it against the deployed hostname, not just locally: deployment is exactly where `config.local.js` presence differs, so a suite that only ever runs locally cannot answer whether the deployed build boots.

`DGO_BASE_URL` also suppresses the local web server. Without it Playwright starts `npm run start` and serves this working tree, which would make a green run say nothing about the host under test.

If Chromium is missing or its version does not match the pinned Playwright build:

```
DGO_CHROME_PATH=/path/to/chrome DGO_CHROME_NO_SANDBOX=1 npm run test:smoke
```

Both variables are read by `playwright.config.js`. Combine them freely.

8.1 Verify the deployed package is the package that was built

A green smoke suite says the host boots. It does not say the host is serving the bytes the manifest hashes — a partial upload boots.

1. Open the deployed URL.
2. F12 → Console. Paste `scripts/verify-deployed-package.browser.js`. It **reads only**.
3. Every file should report `matches manifest`. Any row that does not is a byte the host is serving that the build did not produce; re-upload before continuing.

---

§9 · Trigger hardening — item 6.3

`npm run triggerauth` states the current posture of all 27 keys. Four findings, each with a fix.

9.0 The OTP verify patch, and the cutover it belongs to

The patch is applied and closed (`ITEM-38` RESOLVED). The flow it patched, `Web - OTP Verify` (3e201620), is retired with its console patch script: `IP_OTP_VERIFY` (4dbe6dde) carries the same hardening and serves both `OTP_GENERATE` and `OTP_VERIFY` (`FLOW_ADOPTION.md`). Nothing is to be patched here.

1. Sign in to `https://make.powerautomate.com` and open `IP_OTP_VERIFY` , so the tab holds a token for the Flow service.

Then, in that same console, **paste** `scripts/otp-cutover-tests.browser.js` . Set `A` and `B` at the top to two different addresses you control before pasting the whole file. It exercises both the request and the verify legs and reports each. A leg that does not report is a leg that did not run — do not read silence as a pass.

The record of what changed and why is `folio-Master-Flows-and-Endpoints/packages/fresh-flow/CUTOVER-D-to-F.md` and `folio-Master-Flows-and-Endpoints/packages/OTP-VERIFY-SECURITY-PATCH.md` . Neither carries the steps; they are here.

9.1 Four endpoints answer every HTTP verb

`GET_DOCS` , `FETCH_EMAIL_ATTACHMENTS` , `AI_DOC_ANALYSIS` , `AI_CHAT` declare no method, so they answer `GET` , `DELETE` , anything.

Fix, per flow: Edit → trigger → **⋯ → Settings → Method → POST** → Save.

9.2 Four declare no request schema

`AI_EMAIL_ANALYSIS` , `SCAN_INTAKE` , `STATUS` , `UPLOAD` .

`SCAN_INTAKE` and `UPLOAD` are **correct as they are** — their bodies are raw bytes, and a JSON schema on a binary body would reject every valid call. Leave them.

For `AI_EMAIL_ANALYSIS` and `STATUS` , add the schema from the contract: `docs/folio-Master-Flows-and-Endpoints/deployment/sharepoint/portal-data-contract.json` → the key's `request` array gives every field and type.

9.3 Twenty-three declare a schema that still accepts unknown fields

A schema without `"additionalProperties": false` documents the expected body without constraining it. **Do not read "has a request schema" as "validates its input" anywhere in this estate.**

Fix, per flow: Edit → trigger → **Use sample payload to generate schema** is not enough; open the schema editor and add `"additionalProperties": false` at the top level.

Do this **after** §6 passes. Tightening a schema before you have seen a real request is how you reject a field the client actually sends.

9.4 One key, one definition — resolved 2026-10-06

Seven keys once resolved to more than one definition. Each contract key now names exactly one adopted flow (`registers/flow-adoption.json`); every other definition is retired and removed,

recorded with its SHA-256 in `registers/flow-retirement.json`. `BULK_ASSIGNMENT_DIRECT` is retired as a key: bulk assignment is `BULK_ASSIGNMENT`. In the tenant, turn off each retired flow the ledger lists. `npm run test:flowadoption` fails if a retired definition returns.

---

§10 · Content approval — item 6.6

Content approval was on for the two largest lists, which meant every row the flows created was invisible to readers while the write reported success — 14,882 hidden rows on one list, 12,079 on the other. It is off for `DGO DIGITAL OPS`. It is still on for `Global Tracking Queue`, which four internal flows write to, and the platform owner directed on 2026-10-03 that it be turned off (ITEM-41).

```
# In the browser console, on the NITDADGO-EAAACTIVITYTRACKING site:
```

Paste `scripts/set-content-approval.browser.js` once: it only reads, and prints the counts — keep them. Then set `APPLY` to `true` at the top and paste it again. It **refuses to turn approval off unless it has first counted every row**, including the hidden ones — a count it cannot take is reported as unknown and the write is refused. That refusal is deliberate; do not work around it.

Expected: it reports the hidden-row count for `Global Tracking Queue`, then turns approval off. Confirm afterwards that the previously hidden rows are visible to a non-approver account, and commit the printed read-back (`EnableModeration: false`) as the ITEM-41 evidence.

---

§11 · Close-out

Run all four, together, and record the output:

```
npm run keyimpl        # expect: 0 no flow, 0 not exported, 0 unreachable, 0 disclosure
errors
npm run wiring         # expect: 58/58 operations, 7/7 endpoints
npm run commission -- --posture pilot   # expect: 0 blockers
npm test               # expect: all green
```

Then update the brief's §6 list. Each item and what closes it:

Item	Closed by
1 · never executed end to end	§6 — and *only* §6
2 · only 8 of 18 endpoints configured	§4
3 · trigger posture unverified	§9
4 · signed URLs pending rotation	§3
5 · secret scanner scope gap	already closed 2026-09-03
6 · content approval	§10

Item	Closed by
7 · portal endpoint mapping	already closed 2026-09-03
8 · unimplemented contract keys	already closed 2026-09-04
9 · front end has no tests/build	already closed 2026-09-03
10 · deployed platform serves no config	§7.4
11 · the sanctioned crossing	already withdrawn 2026-09-04 — was an artefact

11.1 What this walkthrough does not make true

Completing every step above gets the estate **live, configured, verified against real responses, and rotated**. It does not make it *secure* in the sense §5 of the brief asks about:

- Authentication remains inert. Caller identity is client-asserted. Any flow called directly with its URL answers whoever calls it.
- `SCAN_INTAKE` does not verify the SHA-256 it is sent. Power Automate cannot compute one over a request body. The digest is a custody aid, not an integrity check.
- The `Flow Configuration` read on the internal site is a declared, accepted crossing of the portal trust boundary (D16), not an absence of one.

Those are the independent reviewer's subject, and they are why §5 of the brief is written the way it is. **Finishing this walkthrough is the precondition for that review, not a substitute for it.**

Part 4 · Notification tenant runbook — the steps for notification remediation

> **This document carries the steps for notification remediation, and it is the only one that does.** Every other document in this repository may name a step and say that it lives here; none restates it. There is no precedence rule to apply, because there is nothing to have a conflict with.
>
> The other two step-carrying runbooks are **Part 3 of this runbook** and **Part 5 of this runbook**.

What this closes, and why it is not stale

Three open items in `closure-disposition.json` are `SPECIFIED_NOT_APPLIED` — specified, and not applied to the tenant:

Item	What is wrong today
ITEM-48	The portal promises citizens four emails the estate does not send
ITEM-50	Verification codes, correspondence, reports and assignment notices go to a fixed mailbox
ITEM-51	Officers are never told of assignments, approvals, escalations, delegations or reminders

ITEM-50 is the one to read twice. A verification code sent to a fixed mailbox rather than to the person who asked for it is a delivered credential in the wrong hands, and the portal reports it as sent either way.

Every one of the 19 `RN-` identifiers below is present in the live 39-identifier matrix in `config/notification-matrix.config.js`. This work is current.

Where these steps came from

They were delivered as four separate files under `notification-instrument/tenant-execution/runbooks/`, imported wholesale on 2026-09-05 from a retiring repository and kept in the shape they arrived in. That shape is why no commissioning document ever referenced them and why, measured on 2026-09-10, they were reachable from nothing: four runbooks carrying roughly forty live steps, outside every path anyone reads.

The four originals stay where they are, marked superseded by this consolidation, as the record of what was delivered. **They are not the instruction.** This is.

The position in the sequence

Notification remediation runs **after** the portal runbook's §4 (both surfaces configured) and is independent of the governance runbook. Release 0 containment — §2 below — is the part that stops the ITEM-50 misdelivery and should not wait on the rest.

---

§1 · Baseline and connections

1. Export the current enabled-flow inventory from the target environment.
2. Record each flow ID, owner, trigger type, solution, connection references and enabled state.
3. Map every configured client endpoint to exactly one active workflow ID using a unique non-destructive correlation ID.
4. Record duplicate, orphaned and unknown active flows. Do not disable them yet.
5. Inspect the Office 365 Outlook connection.
6. Confirm the authenticated account equals the approved service mailbox.
7. Confirm send-as or send-on-behalf permissions through controlled internal and external messages.
8. Test a rejecting mailbox and record the message trace or non-delivery reference.
9. Complete `connection-verification.json` and `tenant-inventory.json`.
10. Run the tenant evidence validator.

---

§2 · Release 0 containment — do this first

1. Correct citizen verification `To` to the validated requesting address.
2. Correct officer OTP `To` to the authenticated requesting officer.
3. Remove all fixed personal-recipient dependencies from required notifications.

4. Remove the empty-recipient and empty-subject mail action.
5. Preserve the deployed single-assignment notification until its replacement passes RN-009 acceptance.
6. Preserve upload and writeback run visibility until structured telemetry passes RN-025 acceptance.
7. Identify one authoritative generate and one authoritative confirm flow for each verification domain.
8. Keep superseded flows disabled but rollback-ready until endpoint routing is proven.
9. Execute recipient, invalid-recipient, connector-failure and duplicate-event tests.

---

§3 · Event carriers and the scheduled processor

Immediate carriers

Implement RN-001, RN-008, RN-009, RN-010, RN-011, RN-021, RN-024, RN-037 and RN-038 first.

Lifecycle carriers

Implement RN-002, RN-003, RN-004, RN-005, RN-017, RN-018, RN-019, RN-020 and RN-036.

Scheduled processor

Use `deployment/notification-processor/processor-spec.json`. Do not build until recurrence, retry and claim decisions are approved.

The processor must claim an eligible row, re-read authoritative state, validate recipient and mandatory tokens, check idempotency, send through the approved Outlook connection, record the attempt, schedule retry or permanent failure, alert the administrator when required, and write an audit event.

---

§4 · Cutover, rollback and hypercare

Cutover

1. Freeze related configuration changes.
2. Export and hash old flow definitions.
3. Deploy in dependency order.
4. Bind approved connections and environment variables.
5. Execute smoke and recipient tests after each flow.
6. Reconcile in-flight queue rows and delivery receipts.
7. Route endpoints only after the new flow passes.
8. Disable superseded flows without deleting them.

Rollback

1. Restore the prior endpoint routing or enabled flow.
2. Disable the failed replacement.
3. Reconcile events processed during the cutover window.
4. Cancel or supersede duplicate queue rows.
5. Confirm no duplicate mail was delivered.
6. Record duration, operator and evidence.

Hypercare

Establish named business and technical owners, message-trace access, daily failure review, emergency recipient correction, rollback authority, and an agreed exit criterion.

What is NOT in this runbook

The scheduled processor in §3 says *"do not build until recurrence, retry and claim decisions are approved."* Those approvals are an agency decision, not an executor's. Until they exist, §3's processor half is specified and not buildable — §1, §2 and the carrier half of §3 are not blocked by it.

Part 5 · Governance tenant runbook — actions outside the repository

> **This document carries no commands.** The steps for the estate, end to end live in `RUNBOOK.md`, which is the only document that carries them. This document is a supporting record. If you are here to execute, go there.

GENERATED — do not edit by hand. Produced by `scripts/build-governance-runbook.mjs` from the governance list registry, the estate position record and the corrected flow definitions. Every GUID and count below is read from those files, so this document cannot quote a stale one. Change the source and re-run `npm run governance:runbook`.

Everything in this runbook needs a tenant. Nothing here can be done from the repository. What HAS been done in the repository is listed in `WHAT-WAS-DONE.md`.

Some of this has been done. 4 of 12 findings record tenant progress — GOV-08, GOV-01, GOV-03, GOV-09 — and the dated detail is in each finding's `tenantStatus` and in `GOVERNANCE-STATUS.md`. Read a step's status there before running it: several are idempotent and safe to repeat, and Step 3's delete phase is deliberately not.

Decision on record

GOV-01 is approved. `https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE` is the single authoritative site for all ten governance lists.

Rationale, from `docs/reference/governance-list-registry.json`:

> The internal flows already address DGO_UserDirectory, DGO_RoleCatalogue and DGO_AuditLog by GUID on this site, and the role catalogue is seeded there. Repointing the specification is therefore a one-sided change; repointing the flows would not be.

Prerequisites — confirm all four before starting

#	Prerequisite	How to confirm
1	**Manage Lists** on `DGO_ECM_GOVERNANCE`	Open https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE, then Settings ⚙ → Site permissions. Site Owner is sufficient. Tenant administrator is **not** required.
2	**Manage Lists** on `NITDADGO-EAAACTIVITYTRACKING`	Needed only for Step 3 (retiring duplicates). Same check on that site.
3	**Power Automate maker** access in environment `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1`	https://make.powerautomate.com → confirm the environment selector shows it and you can open a flow for editing. Needed for Step 4, and again for Step 5.4 — the trigger URLs the path tests call are read from each flow's HTTP trigger.
4	A browser signed in to `https://nitdanigeria.sharepoint.com`	Any page on either site loads without a sign-in prompt.

Order matters, and applying a correction is not the same step as validating it.

Step 2 before Step 3: provisioning writes only to the GUIDs it is given, so doing it first means the duplicates are already irrelevant when you delete them.

Step 4 before Step 5: the corrected definitions must be in place before the lists they write to exist, so an unguarded flow 05 never meets a populated registry. That is the one ordering mistake with an irreversible outcome, and it is why Step 4 stops at the import.

Step 5 before the path tests. Every path test writes to the registry lists, and the blocked path needs `DGO_HTTPFlowConsumerRegistry` in particular — with no consumer list there is no consumer row, the active-consumer count comes back zero and the guard answers 200 instead of 409. So validation is **Step 5.4**, after the lists exist, not inside Step 4.

Step 1 — Back up, before anything

This estate has no PowerShell path, and this step is not an exception. Since PnP 2.x, `Connect-PnPOnline -Interactive` needs an Entra app registration, which this estate has ruled out — that is why every other script in this runbook runs in the browser console. A mandatory first gate whose

only procedure cannot be executed here is not a gate. The PowerShell below is kept for an operator who already holds that registration; it is the alternative, not the route.

1.1 The browser path

1. Sign in and open any page on `https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`.
2. **F12** → **Console**.
3. Open `scripts/backup-governance-lists.browser.js`, select all, copy, paste, **Enter**.

It **reads only**. It downloads one JSON file per list — every field of every item, plus the list's custom field definitions — and a `governance-backup-manifest.json` naming the GUID, title, item count, field count and capture time of each.

Expected: 11 downloads (10 lists and the manifest), and a final line reading `10 of 10 captured, 0 failed`.

Validation. An empty list still produces a file, holding `"items": []` — an empty backup and a missing backup must not look alike. The script re-reads each list's `ItemCount` after the capture and reports `MISMATCH` if it differs from the number of items it wrote; that means the list was written to mid-capture, so run it again.

What this is, and what it is not. It is an item-and-field-definition export: enough to reconstruct the rows and the custom columns. It does **not** capture views, content types, permissions, attachments, version history or list settings. It is a backup of the data, not of the list.

Restoring is not the inverse of exporting. The captured fields include read-only and system values (`Id`, `Created`, `Author`, `_UIVersionString`), person and lookup values as objects rather than ids, and — after Step 2 — at least one field whose name has changed. Re-importing a file as-is will fail or write the wrong shape. Treat it as the evidence, and as the source values for a controlled restore, not as a restore artefact.

1.2 If you do have PowerShell

```
# Requires PnP.PowerShell AND an Entra app registration for interactive sign-in:
#   Install-Module PnP.PowerShell -Scope CurrentUser -Force
$site = "https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE"
Connect-PnPOnline -Url $site -Interactive
New-Item -ItemType Directory -Force -Path .\governance-backup | Out-Null

$guids = @(
  "3d591f5b-3f2f-409c-983a-a77b5c174834"   # DGO_UserDirectory
  "f675598b-271d-4200-8d75-2597aad4057f"   # DGO_RoleCatalogue
  "9fe1872f-3ae0-4b5f-96fc-5d3331c1d07c"   # DGO_UserRoleHistory
  "be0c7af1-b21d-4efe-8c30-53fc55598d95"   # DGO_AuditLog
  "ad1df270-21c7-4b78-99e7-fb18efd02cd2"   # DGO_PendingWrites
  "eed0ba42-ece1-40e2-bf66-79827de02766"   # DGO_DepartmentDirectory
  "f2ffd2fa-901e-4f28-8f28-957da0fe05e4"   # DGO_AccessScopes
  "ecc4ac65-1804-4f67-ad3e-eacb1a3a4dc2"   # DGO_PilotCohorts
  "08c6e1c4-f2b1-4810-933d-69b4327fb6af"   # DGO_EndpointRegistry
  "a40d5f57-859c-426c-826c-bfee090137ad"   # DGO_AccessEvents
)
foreach ($g in $guids) {
  $list   = Get-PnPList -Identity $g
  $items  = Get-PnPListItem -List $g -PageSize 2000
  $values = @($items | Select-Object -ExpandProperty FieldValues)
  # -InputObject, not the pipeline. A one-item array piped into ConvertTo-Json unrolls and
  # serialises as a bare object, and an empty one emits nothing at all — so an empty list
  # would leave a zero-byte file that no reader can tell from a failed capture.
  ConvertTo-Json -InputObject $values -Depth 10 |
    Out-File ".\governance-backup\$($list.Title).json" -Encoding utf8
  Write-Host "$($list.Title): $($values.Count) item(s) saved"
}

# Prove every file parses before trusting any of them.
Get-ChildItem .\governance-backup\*.json | ForEach-Object {
  try   { Get-Content $_.FullName -Raw | ConvertFrom-Json | Out-Null; "VALID  $($_.Name)"
}
  catch { throw "INVALID JSON BACKUP: $($_.FullName)" }
}
```

Expected: 10 files under `.\governance-backup\`, one per list, each printing `VALID`. **If it fails:** `Get-PnPList` returning *List does not exist* means the GUID is not on this site — stop and re-read `docs/reference/governance-list-registry.json`; do not substitute a title.

Step 2 — Provision the ten governance lists on the authoritative site

This creates missing columns, indexes the columns the specification marks indexed, and seeds the ten configuration rows. **It creates no list** — all ten exist — and it addresses every list by GUID, so it cannot mint an eleventh copy.

2.1 Dry run

1. Sign in to `https://nitdanigeria.sharepoint.com` and open any page on `https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`.
2. Open devtools: **F12** → **Console** tab.
3. Open `scripts/provision-governance-lists.browser.js` from this repository, select all, copy.
4. Paste into the console and press **Enter**.

Expected: a table headed `DRY RUN — 97 columns and 10 seed rows across 10 lists`, then one row per column and per seed, each marked `present`, `WOULD CREATE`, `WOULD INDEX` or `WOULD SEED`. The final line reports `0 failed`.

Validation before proceeding: every row's list name must be one of the ten, and the console must show the site `https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`. If any row reads `SITE UNREACHABLE` or `LIST NOT READABLE`, you lack Manage Lists on that site — stop and resolve prerequisite 1.

2.2 Apply

1. In the copied text, change the line `const DRY_RUN = true;` to `const DRY_RUN = false;`.
2. Paste again and press **Enter**.

Expected: the same table with `CREATED`, `INDEXED` and `SEEDED` in place of the `WOULD` verbs, and `0 failed`.

2.3 Verify

Paste the file a third time, unmodified (`DRY_RUN` back to `true` or left false — both are safe).

Expected: every row reads `present`. `0 created, 0 seeded`. That is the idempotence check: a second apply that creates anything means the first did not do what it reported.

2.4 If a list fails

A count of failures at the end of a 107-row table is not a diagnosis. The script now groups failures by list and quotes the server's own message; read that group, not the count.

The distinction that matters is WHERE the failure lands:

Pattern	Meaning	Action
`SITE UNREACHABLE`	No Manage Lists on the site	Resolve prerequisite 1
`LIST NOT READABLE`	The list GUID does not resolve	Re-read the registry; do not substitute a title

Pattern	Meaning	Action
`FAILED` on a column	That column could not be created	Read the message; usually a schema conflict
`RESERVED NAME`	The specification asked for a name SharePoint owns	Rename it in the specification — see below
`ADOPTED`	The tenant already had that column under a different internal name	Nothing. This is the correct outcome
Every failure on ONE list, all touching items	See **2.5**	Do not conclude the list is damaged

2.5 The failure this runbook previously misdiagnosed

The 2026-09-09 run failed every operation on `DGO_AccessScopes` that reached its item store — 500 `List does not exist. It may have been deleted by another user.` on the seed check, 500 `Cannot complete this action.` on the index — while its field metadata resolved normally. This was recorded as **GOV-07**, a damaged list. **It was not.** The list opens in a browser and its view shows a column titled **Access Scope Id**.

The real cause is **GOV-08**. `ScopeId` is a SharePoint SYSTEM field: every list carries a hidden one holding the item's security scope. The specification declared a custom column of the same name, the presence check answered yes against SharePoint's own field, the specification's column was never created — and then indexing a system field returned 500, and filtering a Guid-typed system field against the string `'all'` returned 500 "List does not exist."

Three columns were affected, and all three are now renamed in the specification:

List	Was	Now	Display name
`DGO_AccessScopes`	`ScopeId`	`AccessScopeId`	Access Scope Id
`DGO_RoleCatalogue`	`Version`	`CatalogueVersion`	Catalogue Version
`DGO_EndpointRegistry`	`Version`	`EndpointVersion`	Endpoint Version

This is why Step 2 must be re-run. The previous run's report — 97 of 97 columns present — counted at least one column that did not exist. Re-running against the corrected specification is what replaces that report with one that means what it says.

Expect `ADOPTED` on `DGO_AccessScopes.AccessScopeId`. The tenant already carries a custom column displayed as *Access Scope Id*, created by hand by someone who hit the same collision. The script uses that column and rewrites the seed onto its real internal name rather than creating a second column for the same fact. `ADOPTED` is a success, not a warning.

If a `RESERVED NAME` row appears for any other column, the specification is asking for a name SharePoint will never release. Rename it in `docs/reference/sharepoint-provisioning-spec.json` via `scripts/correct-governance-fields.mjs` and re-run — never by hand, or the next extraction loses the fix.

A failing list does not block the other nine. The script writes per list; the nine that succeed are complete and usable.

2.6 Read back what was actually created — do this after any apply

`createfieldasxml` is called with `Options: 24`, and 8 of that is **AddFieldInternalNameHint**: SharePoint takes the column's internal name from the SchemaXml's `Name` attribute, not from `StaticName` and not from the display name. A column can therefore be created under one internal name while showing another in the view — and the run reports `CREATED` either way.

That is not hypothetical. The run of **2026-09-09** created three columns from a SchemaXml whose `Name` still carried the old name, because the rename's last regex matched the tail of `DisplayName` instead of the standalone `Name` attribute:

List	Column	`Name` it was built with
`DGO_RoleCatalogue`	`CatalogueVersion`	`Version` — reserved
`DGO_EndpointRegistry`	`EndpointRedacted`	`FlowUrl` — the name GOV-03 removes
`DGO_EndpointRegistry`	`EndpointVersion`	`Version` — reserved

The specification is fixed and guarded. What the tenant received is a separate question, and only the tenant can answer it.

1. Open `scripts/verify-governance-columns.browser.js`, copy the whole file.
2. Console on the governance site → paste → Enter. It **reads only**.
3. Every row should read `present`, and the last line should read `WP-0 PASSES: every specified column exists under its specified name.`

The script prints exactly five verdicts and no others. They are not interchangeable, and the two that stop the step are not the two that read most alarmingly:

Verdict	Meaning	Stops the step
`present`	the column exists as a custom column under the specified internal name	no
`MISSING`	the specification declares it and the list does not have it under that name	**yes**
`RESERVED NAME`	a field of that name exists, but it is SharePoint's own	**yes**
`UNREADABLE`	the list itself could not be read	**yes**
`OLD NAME PRESENT`	a superseded name is still on the list	no — record it, do not delete it

If a row reads `MISSING`: the specified internal name is not on the list. That has **two** causes and they need different remedies, so establish which before acting:

- the create silently did nothing — re-run Step 2; or

- the create succeeded under a **different** internal name, which is the 2026-09-09 outcome and leaves the column present but unfindable by its specified name.

Run `scripts/dump-governance-columns.browser.js` and look for the display name. If a column carries the right display name under the wrong internal name, go to 2.7. **Do NOT delete the column on sight** — on some lists it holds the only copy of the data.

2.7 Remediating a column created under the wrong internal name

This was the outcome on **2026-09-09**, and the tenant has been read back. SharePoint did not overwrite anything: it **auto-disambiguated**, appending `0` and creating each new column *beside* the original. **No data was lost.**

List	Original — untouched	Created beside it	Holds
`DGO_RoleCatalogue`	`Version` — 6/6 populated `R11.6-PILOT`	`Version0` (Title "CatalogueVersion")	empty
`DGO_EndpointRegistry`	`Version` — 6/6 populated `document-portal.1`	`Version0` (Title "EndpointVersion")	empty
`DGO_EndpointRegistry`	`FlowUrl` — 0/6 populated, no signature	`FlowUrl0` (Title "EndpointRedacted")	empty

Two of those renames have since been WITHDRAWN. Both `Version` columns are ordinary custom columns — `FromBaseType=false`, `CanBeDeleted=true` — populated and working. `Version` is not a reserved internal name on an ordinary list; SharePoint's own version field is `_UIVersionString`. It was added to the reserved list by generalising from the confirmed `ScopeId` case instead of checking it. **Only** `ScopeId → AccessScopeId` **was a real fix**, and the tenant has confirmed it.

So the work here is smaller than it first looked: **delete three empty columns.** Nothing is migrated, because nothing that holds data is being renamed.

Delete	Why it is safe
`DGO_RoleCatalogue.Version0`	empty, created 2026-09-09, nothing references it
`DGO_EndpointRegistry.Version0`	empty, created 2026-09-09, nothing references it
`DGO_EndpointRegistry.FlowUrl0`	empty; `EndpointRedacted` is recreated properly by Step 2

2.7.1 — Confirm they are still empty before deleting. Paste `scripts/inspect-governance-strays.browser.js`, then `scripts/dump-governance-columns.browser.js`. The first reports how many rows are populated and whether any value carries a `sig=`; the second lists every custom column, so a name neither script anticipated cannot hide. **A column that has acquired data since is no longer a stray — stop and say so.**

2.7.2 — Delete the three. No PowerShell, no PnP, no install.

1. Open `scripts/cleanup-disambiguated-columns.browser.js`, select all, copy.
2. Console on the governance site → paste → **Enter**. Dry run; nothing changes.

3. Read the table. Then set `DRY_RUN = false` at the top and paste again.

It does not carry a list of three names — a fixed list would be right once and wrong at the next collision. It finds them by their shape on the live list. A column is a candidate only when **all four** hold:

	Condition
1	it is custom — `FromBaseType` false, `CanBeDeleted` true
2	its internal name is `<base><digits>`
3	a **custom** column named `<base>` also exists on the same list
4	every row is empty in it

Condition 3 is what keeps a legitimately-named column safe: `Version0` qualifies only because `Version` is beside it. A column called `Phase2` on a list with no `Phase` is never a candidate. Condition 4 is re-read at the moment of deletion, not trusted from an earlier run — **a column that has acquired data since is reported `KEPT — HOLDS DATA` and left alone.**

2.7.2a — If a rename still needs migrating. Open and paste `scripts/remediate-governance-strays.browser.js`; it handles the case where a column must be renamed and its values carried across. It is not needed for the 2026-09-09 cleanup — nothing being removed holds data — but it is the tool if that changes. Per column, in this order, and the order is the safety:

	Step	
1	**READ**	is the correct column there? does the old one hold anything?
2	**CREATE**	the correctly-named column, from the corrected specification
3	**COPY**	every value old → new. Never over a value already in new
4	**VERIFY**	read back; every populated row must match
5	**DELETE**	the old column — only if 4 passed for every row

Create, copy and verify all complete before anything is removed, so a failure at any point leaves the data exactly where it was.

It refuses to delete a column whose copy did not verify, and a column holding a value with a `sig=` in it. It skips a column that is already correct, and one whose old name is a SharePoint system field rather than a custom column — so it will never attempt to delete SharePoint's own `ScopeId`.

2.7.3 — If it reports `REFUSED`: a live credential is in a list column. That is **GOV-03** happening for real. **Deleting the column revokes nothing** — possession of the URL is the authentication, and any copy taken from that list still works. Regenerate the trigger in Power Automate first (the flow → the HTTP trigger → regenerate the URL), update `config.local.js` from the new URL, and only then re-run the remediation.

2.7.4 — If it reports `NOT DELETED`: the copy did not verify and the old column is still there, with its data. Nothing was lost. Read the mismatched item ids it names before re-running.

2.7.5 — Re-run Step 2. Once the strays are gone, paste the provisioner again. A third paste should report everything `present`.

2.8 `FlowUrl` — GOV-03's actual remaining work

`DGO_EndpointRegistry.FlowUrl` is **empty on all six rows and carries no signature**, so there is no live credential in a list column and **nothing to revoke**. GOV-03 is a design defect here, not an incident.

Retiring it is a deliberate act, separate from the cleanup above:

1. Step 2 creates `EndpointRedacted` (and `EndpointFingerprint`, already correct in the tenant).
2. Confirm `FlowUrl` is still empty.
3. Delete `FlowUrl`.

It will come back unless the flow is corrected too. The deployed flow *Repair the DGO governance lists* creates `FlowUrl` from its own SchemaXml — deleting the column without correcting that flow means the next run of it restores the column. That correction belongs with the other flow work in **Step 4**.

If you do have PowerShell and prefer it, the equivalent is `Add-PnPFieldFromXml` with the SchemaXml from the specification, then `Set-PnPListItem` per row, then `Remove-PnPField` — in that order, never the reverse. The console script is the supported path because it takes its SchemaXml from the specification rather than from a copy-paste, which is the defect that created these strays.

Rollback

Not everything this step does is additive, and the rollback has to know the difference. Read the verb the run reported against each row:

What the run reported	What it did	Undo
`CREATED`	added a column that did not exist	remove it — it held no data
`SEEDED`	added a configuration row	delete the row
`INDEXED`	changed an **existing** column's settings	set `Indexed` back to false; the column itself pre-dates this run
`ADOPTED`	used a column the tenant already had	**nothing.** Never remove an adopted column — it pre-dates this run and may hold data
2.7.2	deleted three empty stray columns	not undoable; they were verified empty first
2.8	deleted `FlowUrl`	not undoable; it was verified empty first

To remove a column **this run created** — never an adopted one, and never one you have not confirmed empty:

```
Connect-PnPOnline -Url "https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE" -Interactive
Remove-PnPField -List "<list GUID from the table below>" -Identity "<InternalName>" -Force
```

`Remove-PnPField` destroys the column and every value in it, breaks any view or query naming it, and does not ask. Confirm the column is one this run created and is still empty — 2.7.1's two read-only scripts do exactly that — before running it.

Seed rows can be restored from `.\governance-backup\<list>.json` captured in Step 1, but read Step 1's note on restoring first: that file is the source values for a controlled restore, not an import artefact.

Step 3 — Retire the 17 duplicate list instances

Do this only after Step 2 reports `present` for everything. These are the copies recorded as **GOV-02**: The 2026-08-18 capture shows all ten governance lists on both DGO_ECM_GOVERNANCE and NITDADGO-EAAACTIVITYTRACKING, and seven of them additionally carrying a _2 copy on NITDADGO-EAAACTIVITYTRACKING. A _2 suffix is SharePoint disambiguating a title that already existed, which is the signature of a title-based provisioning flow run more than once or run against the wrong site. A flow writing DGO_AuditLog and one reading DGO_AuditLog_2 both succeed and neither sees the other.

3.1 The instances to KEEP — never touch these

All on `https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE`.

List GUID	Title	Server-relative URL
`3d591f5b-3f2f-409c-983a-a77b5c174834`	`DGO_UserDirectory`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_UserDirectory`
`f675598b-271d-4200-8d75-2597aad4057f`	`DGO_RoleCatalogue`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_RoleCatalogue`
`9fe1872f-3ae0-4b5f-96fc-5d3331c1d07c`	`DGO_UserRoleHistory`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_UserRoleHistory`
`be0c7af1-b21d-4efe-8c30-53fc55598d95`	`DGO_AuditLog`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_AuditLog`
`ad1df270-21c7-4b78-99e7-fb18efd02cd2`	`DGO_PendingWrites`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_PendingWrites`
`eed0ba42-ece1-40e2-bf66-79827de02766`	`DGO_DepartmentDirectory`	`/sites/DGO_ECM_GOVERNANCE/Lists/DGO_DepartmentDirectory`

List GUID	Title	Server-relative URL
f2ffd2fa-901e-4f28-8f28-957da0fe05e4	DGO_AccessScopes	/sites/DGO_ECM_GOVERNANCE/Lists/DGO_AccessScopes
ecc4ac65-1804-4f67-ad3e-eacb1a3a4dc2	DGO_PilotCohorts	/sites/DGO_ECM_GOVERNANCE/Lists/DGO_PilotCohorts
08c6e1c4-f2b1-4810-933d-69b4327fb6af	DGO_EndpointRegistry	/sites/DGO_ECM_GOVERNANCE/Lists/DGO_EndpointRegistry
a40d5f57-859c-426c-826c-bfee090137ad	DGO_AccessEvents	/sites/DGO_ECM_GOVERNANCE/Lists/DGO_AccessEvents

3.2 The instances to RETIRE

List GUID	Title	Site	Superseded by
b9b522c6-1219-4fb1-ad01-b54b40416bba	DGO_UserDirectory	NITDADGO-EAAACTIVITYTRACKING	3d591f5b-3f2f-409c-983a-a77b5c174834
55c0daae-8b46-4db7-8c03-be54e3fc536f	DGO_RoleCatalogue	NITDADGO-EAAACTIVITYTRACKING	f675598b-271d-4200-8d75-2597aad4057f
4fdb8379-9f48-466e-a503-a59912be5015	DGO_UserRoleHistory	NITDADGO-EAAACTIVITYTRACKING	9fe1872f-3ae0-4b5f-96fc-5d3331c1d07c
74153ee5-e6dd-4347-b1fe-a51d7fd47521	DGO_AuditLog	NITDADGO-EAAACTIVITYTRACKING	be0c7af1-b21d-4efe-8c30-53fc55598d95
7ac06ab8-9754-4578-be86-87c809c8deea	DGO_AuditLog_2	NITDADGO-EAAACTIVITYTRACKING	be0c7af1-b21d-4efe-8c30-53fc55598d95
66e19c2a-877b-4537-8247-2351d6374cf9	DGO_PendingWrites	NITDADGO-EAAACTIVITYTRACKING	ad1df270-21c7-4b78-99e7-fb18efd02cd2
673b687d-5e07-4cd2-89c0-411d10238752	DGO_PendingWrites_2	NITDADGO-EAAACTIVITYTRACKING	ad1df270-21c7-4b78-99e7-fb18efd02cd2
db2f8e1a-69e4-4c8a-a2ee-4f2a29d85f4b	DGO_DepartmentDirectory	NITDADGO-EAAACTIVITYTRACKING	eed0ba42-ece1-40e2-bf66-79827de02766
ed850c51-79e6-4f7e-a725-f8ca9f42ff01	DGO_DepartmentDirectory_2	NITDADGO-EAAACTIVITYTRACKING	eed0ba42-ece1-40e2-bf66-79827de02766
3bd611eb-0391-47ee-9dfa-e4520a18787b	DGO_AccessScopes	NITDADGO-EAAACTIVITYTRACKING	f2ffd2fa-901e-4f28-8f28-957da0fe05e4
cccc57f9-1091-4e9c-9826-e46918b057f0	DGO_AccessScopes_2	NITDADGO-EAAACTIVITYTRACKING	f2ffd2fa-901e-4f28-8f28-957da0fe05e4
146c6162-7bea-4ecb-9b1d-254c1e979e12	DGO_PilotCohorts_2	NITDADGO-EAAACTIVITYTRACKING	ecc4ac65-1804-4f67-ad3e-eacb1a3a4dc2
22925b7d-9ea7-497f-9c01-70a32bcb6f2f	DGO_PilotCohorts	NITDADGO-EAAACTIVITYTRACKING	ecc4ac65-1804-4f67-ad3e-eacb1a3a4dc2

List GUID	Title	Site	Superseded by
`89ac07db-f7d0-43b4-b508-2479580fae18`	`DGO_EndpointRegistry_2`	NITDADGO-EAAACTIVITYTRACKING	`08c6e1c4-f2b1-4810-933d-69b4327fb6af`
`dbc0abbd-51c5-4002-8786-3ebc866a8c35`	`DGO_EndpointRegistry`	NITDADGO-EAAACTIVITYTRACKING	`08c6e1c4-f2b1-4810-933d-69b4327fb6af`
`4411f921-2a76-4dea-823d-03f1daf1df6d`	`DGO_AccessEvents`	NITDADGO-EAAACTIVITYTRACKING	`a40d5f57-859c-426c-826c-bfee090137ad`
`c107a54f-3f38-4f71-8a47-38c75f047186`	`DGO_AccessEvents_2`	NITDADGO-EAAACTIVITYTRACKING	`a40d5f57-859c-426c-826c-bfee090137ad`

3.2a STOP — the delete phase is gated. Read this before running anything.

The survey of 2026-09-09 found something that re-orders this runbook. Thirteen of the seventeen duplicates are empty. Four are not:

Duplicate	Items	Last written
`DGO_RoleCatalogue`	6	2026-08-31T01:58:48Z
`DGO_AuditLog`	5	2026-08-31T01:58:49Z
`DGO_UserDirectory`	1	2026-08-31T01:58:48Z
`DGO_PilotCohorts`	1	2026-08-31T01:58:48Z

All four within one second — that is **one run, not four accidents**. The list capture this runbook reasons from is dated **2026-08-18**. Something wrote seed rows into the **non-authoritative** site **thirteen days after the audit**, and the counts match the specification's seed set exactly: 6 role seeds, 1 bootstrap user, 1 pilot cohort.

A live producer is still pointed at the site these lists are being retired from. Recorded as **GOV-09**.

Why that gates the delete and nothing else:

- Deleting a list nothing writes to is housekeeping. Deleting a list something still writes to breaks that producer.
- The deployed provisioning flows **create lists by title** — that is the root cause **GOV-02** records. This repository's provisioner was changed to address by GUID; **the deployed flows were not.** Delete before correcting the producer and its next run recreates the duplicates, restoring exactly the estate the deletion was meant to end.

So:

Phase	Status
`survey`	run it
`export`	run it

Phase	Status
`rename`	**run it** — this is what makes the producer fail loudly while the fix is one `restore` away
`delete`	**blocked** until GOV-09 is resolved — that is Step 4

3.2b Confirm nothing unique would be lost — do this before any rename or delete

Open `scripts/compare-duplicate-governance-lists.browser.js`, copy the whole file, and paste it into the console on either site. It **reads only**, and it asks both.

It answers the one question that decides whether the data is at risk: does any row exist **only** in a duplicate? If one does, it is the sole copy.

Verdict	Action
No row exists only in a duplicate	Continue to 3.3
Any row exists only in a duplicate	**Stop.** Deleting would destroy the only copy. Report it

It does **not** identify the producer — nothing readable from a list says what wrote to it. That is GOV-09, and it is Step 4.

3.3 The browser path — no PowerShell required

Step 3 runs entirely from the console on the duplicate site. **Sign in to `https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING` — the DUPLICATE site, not the authoritative one — then open `scripts/retire-duplicate-governance-lists.browser.js`, copy the whole file, and paste it there.**

It is **MODE-driven, not `DRY_RUN`-driven**, because one boolean is not enough protection when the mistake is unrecoverable. Edit `MODE` at the top and paste again for each step:

MODE	What it does	Reversible
`survey`	read-only: what exists, item counts, current state	—
`export`	downloads every item of every list as JSON	—
`rename`	Title → `ZZ_RETIRED_<title>`	yes, via `restore`
wait	**days, not minutes**	—
`delete`	removes the list — **only if already renamed** AND `I_HAVE_THE_EXPORT = true`	**NO**
`restore`	strips the prefix	—

`delete` cannot touch a list `rename` has not already renamed. A rename you did not do is a rename that did not happen, so the reversible step cannot be skipped — the irreversible action is reachable only through it.

Why wait between rename and delete. The rename is what surfaces a consumer nobody documented: a flow, a view, a Power BI query silently reading one of these. Give it long enough to break loudly while the fix is still one `restore` away.

Refusals built in. It will not run at all if any target GUID appears in the keep set. It stops immediately if pasted on the authoritative site. It never addresses a list by title — the duplicates share their titles with the lists being kept, which is what makes them duplicates, so a title-addressed retirement would eventually delete the estate.

These guards are executed against a fake tenant by `tests/governance-retirement.test.mjs`, not asserted by pattern-matching the file — the acknowledgement check originally sat *after* the delete loop, where a text search still found it and it printed "NOTHING WAS DELETED" having just deleted everything.

3.4 If you do have PowerShell — the survey

For each GUID in 3.2, run:

```
Connect-PnPOnline -Url "https://nitdanigeria.sharepoint.com/sites/NITDADGO-
EAAACTIVITYTRACKING" -Interactive
$g = "<GUID from the table above>"
$l = Get-PnPList -Identity $g
"{0}: {1} item(s), last modified {2}" -f $l.Title, $l.ItemCount,
$l.LastItemUserModifiedDate
```

Decision rule, applied per list. `ItemCount` counts rows. It does not count consumers, and the two are not the same question: a list with no items can still be read by a flow, bound by GUID in a view, query or report, or be the list a producer writes to next.

- `ItemCount` is **0** → it is a candidate for `rename`. It is **not** cleared for `delete`.
- `ItemCount` is **greater than 0** → export it first (`Get-PnPListItem -List $g -PageSize 2000 | Select -ExpandProperty FieldValues | ConvertTo-Json -Depth 10 | Out-File .\duplicate-$($l.Title)-$g.json`), then compare against the authoritative instance in 3.1. Rows present only in the duplicate are the sole copy of those rows.

Neither answer clears the delete, and 3.2a is why: GOV-09 established that something wrote seed rows to four of these lists thirteen days after the audit this runbook reasons from. Until that producer is found and repointed, an empty duplicate is an empty list that something may still be addressing. `ItemCount = 0` says nothing was there when you looked.

3.5 If you do have PowerShell — rename, then delete

Renaming is reversible. Deleting is not, beyond the recycle bin's retention.

What a rename does and does not surface. `Set-PnPList -Title` changes the display title and nothing else: the list keeps its GUID and its server-relative URL, and Power Automate's SharePoint actions bind by GUID. A flow reading one of these will not notice, and neither will a weekly or

monthly job that does not happen to run in the window. What a rename surfaces is a consumer that addresses the list **by title** — which is the class that created these duplicates in the first place, so it is worth doing. Read the quiet period as evidence about title-bound consumers, not as proof that nothing uses the list.

```
Connect-PnPOnline -Url "https://nitdanigeria.sharepoint.com/sites/NITDADGO-
EAAACTIVITYTRACKING" -Interactive
$retire = @(
  "b9b522c6-1219-4fb1-ad01-b54b40416bba"   # DGO_UserDirectory
  "55c0daae-8b46-4db7-8c03-be54e3fc536f"   # DGO_RoleCatalogue
  "4fdb8379-9f48-466e-a503-a59912be5015"   # DGO_UserRoleHistory
  "74153ee5-e6dd-4347-b1fe-a51d7fd47521"   # DGO_AuditLog
  "7ac06ab8-9754-4578-be86-87c809c8deea"   # DGO_AuditLog_2
  "66e19c2a-877b-4537-8247-2351d6374cf9"   # DGO_PendingWrites
  "673b687d-5e07-4cd2-89c0-411d10238752"   # DGO_PendingWrites_2
  "db2f8e1a-69e4-4c8a-a2ee-4f2a29d85f4b"   # DGO_DepartmentDirectory
  "ed850c51-79e6-4f7e-a725-f8ca9f42ff01"   # DGO_DepartmentDirectory_2
  "3bd611eb-0391-47ee-9dfa-e4520a18787b"   # DGO_AccessScopes
  "cccc57f9-1091-4e9c-9826-e46918b057f0"   # DGO_AccessScopes_2
  "146c6162-7bea-4ecb-9b1d-254c1e979e12"   # DGO_PilotCohorts_2
  "22925b7d-9ea7-497f-9c01-70a32bcb6f2f"   # DGO_PilotCohorts
  "89ac07db-f7d0-43b4-b508-2479580fae18"   # DGO_EndpointRegistry_2
  "dbc0abbd-51c5-4002-8786-3ebc866a8c35"   # DGO_EndpointRegistry
  "4411f921-2a76-4dea-823d-03f1daf1df6d"   # DGO_AccessEvents
  "c107a54f-3f38-4f71-8a47-38c75f047186"   # DGO_AccessEvents_2
)
foreach ($g in $retire) {
  $l = Get-PnPList -Identity $g
  if ($l.Title -like "ZZ_RETIRED_*") { Write-Host "$($l.Title) already retired"; continue
}
  Set-PnPList -Identity $g -Title ("ZZ_RETIRED_" + $l.Title)
  Write-Host "renamed $($l.Title) -> ZZ_RETIRED_$($l.Title)"
}
```

Expected: 17 rename confirmations.

Then wait 14 days — and re-read 3.2a before going further. **The delete is gated on GOV-09, not on the quiet period.** A silent fortnight tells you no title-bound consumer broke; it does not tell you the producer that wrote to four of these lists on 2026-08-31 has been repointed. If it has not, the deployed provisioning flows create lists **by title** and the next run restores exactly the estate this deletion was meant to end.

Once GOV-09 is resolved and the quiet period has passed:

```
foreach ($g in $retire) { Remove-PnPList -Identity $g -Force }
```

Rollback

Within 14 days, rename back:

```
foreach ($g in $retire) {
  $l = Get-PnPList -Identity $g
  if ($l.Title -like "ZZ_RETIRED_*") { Set-PnPList -Identity $g -Title ($l.Title -replace "^ZZ_RETIRED_","") }
}
```

After deletion, restore from the site recycle bin (93 days by default): `Get-PnPRecycleBinItem | Where-Object { $_.Title -like "ZZ_RETIRED_*" } | Restore-PnPRecycleBinItem`.

Step 4 — Apply the flow corrections

Do this before Step 5, and stop at the import. Provisioning the registry lists while flow 05 can cascade unguarded is the one ordering mistake with an irreversible outcome, so the corrected definitions go in first. The path tests are **Step 5.4**, not part of this step: every one of them writes to a registry list that does not exist yet.

Flow	FlowId	Corrects	Changes	Registry lists it reads or writes
02 - GOV - Register HTTP Flow Truth	`651ae389-eeed-4bf0-9799-67bd964ebe71`	GOV-04, GOV-06	7	`DGO_HTTPFlowRegistry` `DGO_HTTPFlowContractVersions` `DGO_HTTPFlowDependencies`
03 - GOV - Record HTTP Flow Execution	`0545e19a-c34a-45db-bd11-27b87a34973e`	GOV-04	4	`DGO_HTTPFlowRegistry` `DGO_HTTPFlowExecutionLedger`
05 - GOV - Retire HTTP Flow	`493b4ab7-8f63-4fab-8121-746c2cac7f98`	GOV-04, GOV-05	11	`DGO_HTTPFlowRegistry` `DGO_HTTPFlowContractVersions` `DGO_HTTPFlowDependencies` `DGO_HTTPFlowRegistryExceptions` `DGO_HTTPFlowConsumerRegistry`

Flow	FlowId	Corrects	Changes	Registry lists it reads or writes
07 - GOV - Consumer HTTP Self-Registration Template	`c6a011c5-7b11-4241-82ae-a45b88ebf276`	GOV-04	4	`DGO_HTTPFlowRegistry` `DGO_HTTPFlowConsumerRegistry`

The GUID column is a `FlowId`: the 36-character Power Automate flow GUID, dashed. It is not a `WorkflowId`, which is 32 hex characters with no dashes and is what the tenant register keys on. Step 6 exists precisely because those two identifier spaces have never been recorded together, so nothing in this runbook may use one word for both.

The last column is read from the corrected definitions themselves, and it is what Step 5 provisions. Between them the four flows touch 6 of the 7 registry lists.

4.1 Export the current definition — the rollback

For each of the four flows:

1. https://make.powerautomate.com → environment `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1`.
2. **My flows** → find the flow by name → ⋯ → **Export** → **Package (.zip)**.
3. Save as `<flow name>-BEFORE-<yyyyMMdd>.zip`. **This is the only rollback.** Do not skip it.

4.2 Apply the corrected definition

The corrected definitions are in `docs/folio-Master-Flows-and-Endpoints/surfaces/governance/`, one file per flow, each carrying a complete `definition` object.

Power Automate's **Peek code** is read-only, so the definition cannot be pasted back through it. Use the package route:

1. Unzip the `-BEFORE-` package from 4.1.
2. Open `Microsoft.Flow/flows/<guid>/definition.json`.
3. Replace its `properties.definition` object with the `definition` object from the matching `*.corrected.json`.
4. Re-zip the package with the same internal structure.
5. Power Automate → **Import** → upload the edited package → **Update** the existing flow (not *Create as new*).

4.3 Do not test yet

The path tests belong to **Step 5.4**. Every one of them writes to a registry list, and the blocked path needs `DGO_HTTPFlowConsumerRegistry` — none of which exist until Step 5. Run them now and PATH 3 answers 200 instead of 409, which is the guard failing open and reporting nothing.

What you can confirm here, without any list:

```
# Invalid input must answer 400, not a silent acknowledgement. This path returns before
any
# list is touched, so it is the one test that is meaningful before Step 5.
curl -sS -o /dev/null -w '%{http_code}\n' -X POST '<flow 02 trigger URL>' \
  -H 'Content-Type: application/json' -d '{}'
# Expected: 400
```

The URL contains a `sig=` credential: paste it into a terminal, never into a ticket, chat message or document. Possession of that URL is the authentication — see 2.7.3.

Rollback

Power Automate → the flow → ⋯ → **Import** the `-BEFORE-` package from 4.1, choosing **Update** on the existing flow. The previous definition is restored exactly.

Step 5 — Provision the registry lists the corrected flows need, then validate

5.1 What is provisioned, and why that set

All 7 of them, and the set is not a judgement call: it is every list the four corrected definitions read or write, plus the configuration list they take their control parameters from. The membership is derived from the definitions at build time.

#	List	Fields	Unique key	Why now
1	`DGO_HTTPFlowRegistry`	36	`RegistryKey`	Read or written by flows 02, 03, 05, 07
2	`DGO_HTTPFlowContractVersions`	13	`ContractKey`	Read or written by flows 02, 05
3	`DGO_HTTPFlowExecutionLedger`	15	`ExecutionKey`	Read or written by flow 03
4	`DGO_HTTPFlowDependencies`	9	`DependencyKey`	Read or written by flows 02, 05
5	`DGO_HTTPFlowRegistryExceptions`	12	`ExceptionKey`	Read or written by flow 05
6	`DGO_HTTPFlowRegistryConfiguration`	6	`ConfigKey`	The control parameters the registry flows read
7	`DGO_HTTPFlowConsumerRegistry`	11	`ConsumerKey`	Read or written by flows 05, 07

Deferred:

Nothing is deferred. The four corrected flows touch every list but the configuration list, and they read that.

5.2 Why the earlier deferral could not stand

A previous version of this runbook provisioned lists 1, 2 and 6 and deferred 3, 4, 5 and 7 as "an observability layer with nothing yet to observe", then told the operator to exercise all four corrected flows. Those two instructions cannot both be followed. Flow 03 writes to the execution ledger; flow 07 writes to the consumer registry; flow 05 reads the consumer registry, the dependency list and the exceptions list; and flow 02 — which the deferral explicitly permitted running — writes to the dependency list. The deferral was of the lists the step's own tests require.

The note that accompanied it went further:

> With lists 4 and 7 deferred, both counts return zero and every retirement takes the safe path. That is correct — there are no dependants to protect

That is the guard failing open. A missing consumer registry is not a consumer registry holding no rows. The first is *unknown*; the second is *known to be zero*. A retirement guard that reads the first as the second will authorise exactly the cascade it exists to prevent, and will report a clean 200 while doing it.

5.3 Fail closed, whatever is provisioned

Whatever the deferral set becomes later, flow 05 must treat an unreadable dependency source as a blocker, not as a zero:

Condition	Required outcome
Dependency or consumer list missing	retirement **blocked**
Query returns an error	retirement **blocked**
Query returns zero rows from a list that exists	retirement **permitted**
Any of the above with `force: true`	permitted, and recorded as forced

Only the third row is a safe zero. Until flow 05 distinguishes them, do not defer either list.

5.3a Provision them

Open `scripts/provision-flow-registry-lists.browser.js`, copy the whole file, and paste it into the console on the authoritative site. It is dry-run by default.

1. Paste as-is. Every list and column should read `WOULD CREATE`.
2. Set `DRY_RUN = false` at the top and paste again.
3. **Paste a third time, unmodified.** Every row should read `present`, `0 created`. This third run is the pass condition — an apply that reports work on a second pass did not do what it claimed.

Record every list GUID it prints; `docs/reference/governance-list-registry.json` needs them, and until it has them the repository still describes these lists as unprovisioned.

5.4 Validate every path

Now that the lists exist, test each corrected flow with its trigger URL. **The URL contains a `sig=` credential: paste it into a terminal, never into a ticket, chat message or document.**

Flow 02 — registration.

```
# Invalid input answers 400.
curl -sS -o /dev/null -w '%{http_code}\n' -X POST '<flow 02 trigger URL>' \
  -H 'Content-Type: application/json' -d '{}'
# Expected: 400

# A valid registration answers 200 with outcome "succeeded".
curl -sS -X POST '<flow 02 trigger URL>' -H 'Content-Type: application/json' \
  -d '{"flowId":"00000000-0000-0000-0000-
000000000001","flowName":"__PATH_TEST__","environmentId":"Default-ca6a4b3f-9123-49bc-bcb9-
27085ebbf1a1"}'
# Expected: 200, body "outcome":"succeeded"
```

Flow 03 — the execution ledger. It writes `DGO_HTTPFlowExecutionLedger` and updates the health of the registry row named in the request, so register `__PATH_TEST__` above first.

```
# Invalid input answers 400.
curl -sS -o /dev/null -w '%{http_code}\n' -X POST '<flow 03 trigger URL>' \
  -H 'Content-Type: application/json' -d '{}'
# Expected: 400

# A valid execution record answers 200 with outcome "succeeded".
curl -sS -X POST '<flow 03 trigger URL>' -H 'Content-Type: application/json' \
  -d '{"environmentId":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1","flowId":"00000000-
0000-0000-0000-000000000001","runId":"__PATH_TEST_RUN__","startedUtc":"2026-01-
01T00:00:00Z","completedUtc":"2026-01-01T00:00:01Z","outcome":"Succeeded"}'
# Expected: 200, body "outcome":"succeeded", and one new row in
DGO_HTTPFlowExecutionLedger
```

Flow 07 — consumer self-registration. Read its trigger schema, then read what it consumes; the two have nothing in common. Flows 02, 03 and 05 each declare a `required` array naming exactly the fields they read. Flow 07 declares `action`, `operation` and `source`, requires nothing, sets `additionalProperties: true` — and its definition reads none of those three. What it reads is `registryKey`, `consumerName`, `ownerEmail` and a consumer identifier, and its 400 names all four:

> registryKey, consumerName, ownerEmail, and a consumer identifier are required; values must be 255 characters or fewer; the consumer identifier must not contain a pipe; ownerEmail must contain @.

A payload written from that schema returns 400 and reads like a fault in the flow. It is not — it is the schema describing a contract the definition does not implement. **The schema is left as it stands deliberately:** adding a `required` array would have Power Automate reject the request before the flow runs, returning its own 400 body instead of the `outcome` contract GOV-04 exists to establish. Build payloads for this flow from what it reads, not from what it declares.

```
# Invalid input answers 400 — and the message names the fields it wanted.
curl -sS -X POST '<flow 07 trigger URL>' -H 'Content-Type: application/json' \
  -d '{"action":"register","operation":"consumer","source":"__PATH_TEST__"}'
# Expected: 400, "outcome":"invalid-input"

# A registryKey with no registry row answers 404.
curl -sS -o /dev/null -w '%{http_code}\n' -X POST '<flow 07 trigger URL>' \
  -H 'Content-Type: application/json' \
  -d '{"registryKey":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|00000000-0000-0000-
0000-
0000000000ff","consumerName":"__NO_SUCH__","consumerIdentifier":"__NO_SUCH__","ownerEmail"
:"<your address>"}'
# Expected: 404, "outcome":"not-found"
```

Flow 05 — all four paths. Run them in this order.

```
URL='<flow 05 trigger URL>'

# PATH 1 — invalid input. Expect 400, outcome "invalid-input".
curl -sS -w '\n%{http_code}\n' -X POST "$URL" -H 'Content-Type: application/json' -d '{}'

# PATH 2 — registry row does not exist. Expect 404, outcome "not-found".
curl -sS -w '\n%{http_code}\n' -X POST "$URL" -H 'Content-Type: application/json' \
  -d '{"environmentId":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1","flowId":"00000000-
0000-0000-0000-0000000000ff","retirementReason":"path test — no such flow"}'

# PATH 3 — BLOCKED. Register a flow, give it one ACTIVE CONSUMER, then try to retire it.
#          Expect 409, outcome "blocked", activeConsumers >= 1 in the body.
#
#          The join is RegistryKey, and RegistryKey is <environmentId>|<flowId>. Flow 05
#          composes it from the retirement request; flow 07 must be given the identical
string
#          or it writes a consumer row the guard cannot see, the count comes back zero and
the
#          retirement succeeds with a 200 that looks like a pass.
curl -sS -X POST '<flow 02 trigger URL>' -H 'Content-Type: application/json' \
  -d '{"flowId":"00000000-0000-0000-0000-
000000000002","flowName":"__BLOCK_TEST__","environmentId":"Default-ca6a4b3f-9123-49bc-
bcb9-27085ebbf1a1"}'

curl -sS -X POST '<flow 07 trigger URL>' -H 'Content-Type: application/json' \
  -d '{"registryKey":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|00000000-0000-0000-
0000-
000000000002","consumerName":"__BLOCK_TEST_CONSUMER__","consumerIdentifier":"__BLOCK_TEST_
CONSUMER__","consumerType":"PowerAutomate","ownerEmail":"<your address>"}'
# Expected: 200, "outcome":"succeeded". If this answers 400 or 404, STOP — no consumer row
was
# created, so the retirement below will answer 200 and the guard will look broken when it
was
# the payload.

curl -sS -w '\n%{http_code}\n' -X POST "$URL" -H 'Content-Type: application/json' \
  -d '{"environmentId":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1","flowId":"00000000-
0000-0000-0000-000000000002","retirementReason":"path test — must be blocked"}'
# Expected: 409, "outcome":"blocked", "activeConsumers":1

# PATH 4 — FORCED. The same request with force. Expect 200, outcome "succeeded".
curl -sS -w '\n%{http_code}\n' -X POST "$URL" -H 'Content-Type: application/json' \
```

```
  -d '{"environmentId":"Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1","flowId":"00000000-
0000-0000-0000-000000000002","retirementReason":"path test — forced","force":true}'
```

A 200 on PATH 3 is not a smaller failure than a 500. It is the guard reporting that a flow with a live consumer is safe to retire, and PATH 4 would then be indistinguishable from it. If PATH 3 answers 200, check the consumer row before anything else: open `DGO_HTTPFlowConsumerRegistry` and confirm a row exists with `RegistryKey = Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|00000000-0000-0000-0000-000000000002` and `LifecycleStatus` not `Retired`.

Then confirm the forced retirement was recorded, which is the point of the guard. Open `DGO_HTTPFlowRegistry`, find `__BLOCK_TEST__`, and check its run history shows `Record_Forced_Retirement` with `forced: true` and `activeConsumersAtRetirement: 1`. A forced cascade that is indistinguishable from a safe one is the audit failure this guard exists to prevent.

5.5 Clean up the test rows

The tests write to more than one list, and how many rows depends on what flow 02 created for each registration. Remove them by key rather than by name — the two registry keys are:

- `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|00000000-0000-0000-0000-000000000001` — the `__PATH_TEST__` flow
- `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|00000000-0000-0000-0000-000000000002` — the `__BLOCK_TEST__` flow

Then, per list:

- `DGO_HTTPFlowRegistry` — filter `RegistryKey` on either key. Expect **2 rows**.
- `DGO_HTTPFlowConsumerRegistry` — filter `RegistryKey` on the `__BLOCK_TEST__` key. Expect **1 row**, `__BLOCK_TEST_CONSUMER__`.
- `DGO_HTTPFlowExecutionLedger` — filter `FlowId` on `00000000-0000-0000-0000-000000000001`. Expect **1 row**, from the flow 03 test.
- `DGO_HTTPFlowContractVersions` — filter `RegistryKey` on either key. Expect **0 or more**: flow 02 writes a contract row only when the payload carries a contract, and these payloads do not.
- `DGO_HTTPFlowDependencies` — filter `RegistryKey` on either key. Expect **0**: these payloads declare no dependency.

Re-run every filter after deleting and confirm each returns nothing. **Do this before real traffic arrives** — a synthetic flow left in the registry is a flow the audit will report on, and `__BLOCK_TEST__` is a flow the registry believes was force-retired over a live consumer.

Step 6 — Adopt `WorkflowId` as the permanent join

This closes **GOV-06**: The repository holds 35 flow definitions, one per adopted flow outside the Mailer (registers/flow-adoption.json, 2026-10-05), each identified by a 36-character Power Automate flow GUID. The register holds 55 workflow records, each identified by a 32-hex Logic Apps workflow id. These are different identifier spaces for the same objects, and ZERO definitions

carry an id the register knows. Joining on normalised display name — the only field the two share — matches 21. That leaves 14 definitions the register does not name and 34 register workflows for which no definition is held. Until 2026-10-05 the count was 77 definitions, 13 matched: the 45 retired flows took most of the unmatched ones with them. On 2026-10-07 the register gained IP_Create__Task__for_received_Email_Endpoint (8fbf2896..., the EMAIL_RELATED_TASK flow Folio now calls), whose definition is held: 51 → 52 records, 17 → 18 matched. Later on 2026-10-07 the capture of IP_OTP_GENERATE (e5632336, ITEM-82) was added: 32 → 33 definitions, and it is unmatched until the register names its workflow (741465e4...). On 2026-10-08 the capture of Folio_Incoming_Correspondence_Scan_Processing_Flow (197527c8, ITEM-80) was added: 33 → 34, unmatched until the register names its workflow. On 2026-10-10 the register gained the workflows of IP_OTP_GENERATE (741465e4...) and IP_User_Admin_Endpoint (466cf7ea...), the flows OTP_GENERATE and USER_ADMIN are wired to (config-build-ledger.json): 52 → 54 records, 18 → 20 matched, 16 → 14 unmatched definitions. Later on 2026-10-10 IP_Activity_Lifecycle_Endpoint (b4db02d8, ACTIVITY_LIFECYCLE, ITEM-25) was created, captured and registered (35fa90bf...): 34 → 35 definitions, 54 → 55 records, 20 → 21 matched. On 2026-10-18 the capture of Folio_Failure_Watch (c38bfeaa, the owner-approved failed-notice watch) was added: 35 → 36 definitions, 14 → 15 unmatched until the register names its workflow.

6.0 Three identifiers, and which one is the join

Name	Shape	Whose id it is	Role
`FlowId`	36 characters, dashed GUID	Power Automate	Identifies the flow. Read from the flow URL or the exported package's folder name.
`WorkflowId`	32 hex characters, no dashes	Logic Apps	**The join.** What the tenant register keys on. Read from the trigger URL, between `/workflows/` and `/triggers/`.
`RegistryKey`	the environment id, a pipe, the flow id	This registry	The row key inside `DGO_HTTPFlowRegistry`. Computed by flow 02 — never supplied. Local to this estate; it joins nothing outside it.

`RegistryKey` is written `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1|<flowId>` — the environment id, a pipe character, then the flow id, with no spaces.

An earlier heading here read *Adopt* `FlowId` + `RegistryKey` *as the permanent join*, which names the two identifiers that cannot do the job: `FlowId` is the space the register does not use, and `RegistryKey` is derived from it. The join is `WorkflowId`, recorded beside `FlowId` on the same row.

6.1 Current state, from `docs/folio-Master-Flows-and-Endpoints/registers/flow-definition-map.json`

Measure	Value
Exported definitions	36
— of those, carrying an HTTP trigger	26

Measure	Value
Register workflows	55
Exported definitions matched by display name	21
Register rows those matches cover	21
— because this many names are reused across register rows	0
Exported, not in the register	15
In the register, not exported	34
Shared identifiers	**0**

Both sides reconcile, and the middle row is why. 36 − 21 = 15 on the export side. On the register side it is 55 − 21 = 34, not 55 − 21: 21 exported names match 21 register rows because 0 names are reused. A table that shows only "matched: 21" cannot be made to add up, and the previous version of this table could not.

6.2 What blocks Step 6 today

1. Only 26 of the 36 exported definitions carry an HTTP trigger, and the trigger URL is the documented source of a workflow id. The remaining 10 need another source — the package metadata — or they cannot be joined this way at all.
2. `idOverlap` counts exported definitions carrying an id the register knows, so it cannot exceed 26 — the smaller of the 55-row register and the 26 exports that can supply an id. An exit criterion of 36 is unreachable.

Until 1 and 2 are resolved there is nothing to register: the column the join lives in is not in the specification, and the flow that writes registry rows has no field to put it in. **Neither is a tenant action** — both are repository changes, and they belong before the next tenant window, not inside it.

6.3 Then, in this order

1. Add `WorkflowId` to the registry specification, indexed, and re-run Step 5's provisioning so the column exists on the list.
2. Extend flow 02 to accept `workflowId`, validate its shape — 32 hex characters, no dashes — and write it to the row. Re-apply it through Step 4.
3. Register the exported definitions through flow 02, supplying `flowId` and `workflowId` together. The 10 definitions with no HTTP trigger have no trigger URL to read a workflow id from; take theirs from the package metadata or record them as unjoinable, with the reason.
4. Export `DGO_HTTPFlowRegistry` to JSON and commit it to `docs/reference/http-flow-registry-export.json`.
5. Extend `scripts/reconcile-flow-definitions.mjs` to join on `WorkflowId` when that export is present, falling back to the name join only for rows it does not cover.

6.4 What closes GOV-06

Define the measure before reporting against it. `idOverlap` counts exported definitions that carry an identifier the tenant register also holds. It is bounded above by 26 — the smaller of the 55-row register and the 26 exports that can supply an id — so **36 is not a target it can reach**, and closing GOV-06 on it would record a resolution that has not happened.

GOV-06's finding is that the two corpora cannot be joined. What answers it is not a single number but three, reported together:

Measure	Closes at
Exported definitions carrying a `WorkflowId`	26 by trigger URL; the remaining 10 from package metadata or recorded as unjoinable
`idOverlap` — exports whose id the register knows	every register row that has a definition; 34 register rows have none and that is the finding, not a failure of the join
Register rows still unaccounted for	34 → 0, or each one explained

The third is the one GOV-06 actually asked about: 34 workflows are in the register with no definition held for them. An identifier join does not produce those definitions. Close GOV-06 in `docs/folio-Master-Flows-and-Endpoints/reference/governance-estate-position.json` when all three are answered — and `tests/governance-estate.test.mjs` will fail until the record matches the measurement, in either direction.

Doing this without disrupting active flows

Nothing in Step 6 changes what a running flow does. Registration is a write to a SharePoint list by a flow the running estate does not call; `WorkflowId` is an additive column; and extending flow 02's trigger schema with an optional field leaves every existing caller valid. The name-based map stays in place and keeps working until the identifier join covers everything — that is why 6.3 says *falling back*, not *replacing*.

Step 7 — Measure the out-of-band flow-truth provisioning (GOV-11)

Blocks nothing. Changes nothing. Read-only throughout. It is the only measurement of tenant state this estate did not author, and until it is taken the repository cannot say what is on either site.

7.1 The fact to explain

Two browser scripts were reported executed on **2026-09-10**, creating two lists and a document library — `NITDA Flow Truth Registry`, `NITDA Flow Truth History`, `NITDA Flow Truth Artefacts` — with 90 columns between them. They are held verbatim at `docs/folio-Master-Flows-and-Endpoints/reference/out-of-band/`. None of the three is in any specification this estate holds, and none is in the 2026-08-18 capture.

Three properties make the outcome unpredictable from reading them:

- **Neither pins a site.** Both take whichever site the console was open on, and neither records which. Both candidates must be asked.

- **Both call `createfieldasxml` with `Options: 0`.** Without bit 8, `AddFieldInternalNameHint`, SharePoint may derive each internal name from the display name — so `RegistryKey` may be on the tenant as `Registry_x0020_Key`: correct in every view, invisible to every flow. See 2.6 for what that costs.
- **The idempotence guard depends on the bit that is missing.** If it is absent, every re-run re-created every column and SharePoint disambiguated by appending a digit.

7.2 Procedure

1. Open `scripts/verify-flow-truth-provisioning.browser.js`, copy the whole file.
2. Console on **either** site — it asks about both regardless. Paste. **It reads only.**
3. Copy the single relay line and report it verbatim.

Verdict	Meaning	Action
`absent` on every resource, both sites	The provisioning reached neither candidate site	Record it. Nothing further
`PRESENT`	It exists — site, GUID, template, item count	Record. **Do not delete it**
`asked-name`	The column carries the internal name the script asked for	Record
`DERIVED NAME`	The column is there under a different internal name	Record the name found
`DUPLICATED`	More than one column answers to the same base name	Record. The script ran more than once
`REQUIRED CLEARED`	Created `Required`, not `Required` now	Record
`UNIQUENESS NOT ENFORCED`	The key column does not enforce uniqueness	Record

Do NOT re-run either out-of-band script. A re-run is the duplication case, not a repair.

Do NOT delete, rename or repair anything this reports. Whether these three resources are adopted into `http-flow-registry-spec.json` or retired in favour of the specified registry lists is an agency decision. Report and stop.

Completion checklist

Two of these rows used to contradict each other: Step 4 closed when flow 05 answered 409, and Step 5 closed when the list that makes a 409 possible was absent by decision. They are one step apart now, and the 409 is on the side that provisions the list.

#	Step	Done when
1	Back up	10 list files and a manifest downloaded; each file's item count matches the list's, and empty lists produced a file too

#	Step	Done when
2	Provision governance lists	Third paste reports every row `present`, `0 failed`, and 2.6's read-back reports every row `present` and ends on `WP-0 PASSES: every specified column exists under its specified name.`
3	Retire duplicates	17 lists renamed `ZZ_RETIRED_*`. **Not deleted** — the delete is gated on GOV-09, not on the quiet period
4	Apply flow corrections	All four import cleanly, and `{}` to flow 02 answers 400. No other path test belongs here
5	Provision registry lists 1, 2, 3, 4, 5, 6, 7 and validate	Flow 05 answers 400 / 404 / 409 / 200 on its four paths, PATH 3's 409 carries `activeConsumers` of 1, flow 03 writes a ledger row, and every filter in 5.5 comes back empty
6	Adopt the identifier join	**Blocked.** 2 blockers in 6.2 are open — none of them a tenant action. Nothing in Step 6 can be executed until they are closed
7	Measure the out-of-band provisioning (GOV-11)	`verify-flow-truth-provisioning.browser.js` reports whether the three resources exist, on which site, and under which internal names. **Read-only — blocks nothing, changes nothing.** It was in the procedure and in neither checklist, which is how the one finding whose tenant state is UNMEASURED stayed that way

After Steps 2 and 3, re-capture the tenant list index and run `npm run test:governanceestate`. It will fail — GOV-01 and GOV-02 will be repaired while the record still says they are open — and the failure message names exactly what to close.

Part 6 · SharePoint and Power Automate — the flow build walkthrough

> **What this part is.** The click-by-click SharePoint and Power Automate build: how every flow in this estate was constructed, from provisioning the lists to building the last intake flow. It stood as `FLOW-BUILD-WALKTHROUGH.md` until 2026-09-24 and is carried here unchanged, so that the estate has one runbook rather than several.
>
> It is the long form. **Part 7** is the same build cut down to the pilot's six flows, and **Part 3** is what to do once the flows exist.

Click-by-click steps for the two things this platform actually depends on: your SharePoint site and your Power Automate environment. Hosting the front end and gating who may load it are handled outside this repository — not covered here.

Follow this from top to bottom. Every step is numbered. Do not skip a step or change its order; several steps fail in confusing ways if an earlier one was not done.

How this platform reaches its backend — read this first. The browser calls each Power Automate flow **directly**. There is no proxy, Worker or broker in the request path: nothing to deploy, secret-set, run or keep alive between the page and the flows. The flow trigger URLs are configured client-side at deploy time and are therefore delivered to every browser that loads the page. Whatever gates

who is allowed to load the page, that gate has no server of its own once the page is open. It follows that **each flow is the only place authentication, authorisation, rate limiting, reference minting, upload ticketing, filename-policy normalisation and validation can actually be enforced.** A flow that trusts its caller trusts the whole internet. Everything in Part C depends on this sentence; do not build a flow until you have taken it in.

Time required: Part B 1 hour · Part C 3–4 hours.

What you need before starting: an account with permission to create and delete flows in your Power Platform environment, an account with permission to create SharePoint lists and libraries, and a terminal on a machine with Node.js.

Recording your values. The signed flow trigger URLs cannot be printed in this document. Create a plain text file now called `deployment-values.txt`, somewhere outside the repository, and record each value as you obtain it. You will need them again. Delete the file when you finish: a signed Power Automate URL is a bearer credential — anyone who reads it can invoke the flow.

```
V6 SharePoint site URL       =
Flow trigger URLs            = (24 of them, recorded in Part C)
```

Read this before doing anything in SharePoint

Do not create new lists. An earlier version of this document told you to create a `Correspondence` list with 24 columns. That was wrong, and following it would have left the platform reading one set of lists while the intake channel wrote to another.

Two schemas already exist:

1. Your operational lists. These hold live correspondence today and the platform already reads them. The proof is in `core/domain.js`, which normalises columns named `RefIDD`, `Reference_ID`, `RoutedToDSU`, `DSU_KEY`, `CC_x0027_dTo` and `_x0033_rdAssigned` — the last two being SharePoint internal names for `CC'dTo` and `3rdAssigned`. Nobody invents those. They came from your lists.

*2. The DGO_ platform lists.** Ten lists and 97 fields, fully specified — including the exact `SchemaXml` for every field — in `docs/reference/sharepoint-provisioning-spec.json`:

```
DGO_UserDirectory        DGO_RoleCatalogue        DGO_UserRoleHistory
DGO_AuditLog             DGO_PendingWrites        DGO_DepartmentDirectory
DGO_AccessScopes         DGO_PilotCohorts         DGO_EndpointRegistry
DGO_AccessEvents
```

B1 · Find out what you already have

B1.1 Open your SharePoint site. Copy the URL from the address bar — everything up to and including the site name. Record it as **V6**.

B1.2 Run the report. It changes nothing:

```
./scripts/setup-sharepoint.ps1 -SiteUrl "V6" -WhatIf
```

If PnP.PowerShell is not installed:

```
Install-Module PnP.PowerShell -Scope CurrentUser
```

B1.3 Read the output. It tells you three things:

- which of the ten DGO_* lists exist and which would be created
- which of your operational lists it found, and under what name
- for each, which columns the platform reads and **which are missing**

B1.4 Write down the real name of your correspondence list. The script looks for `Activities`, `Correspondence`, `Documents` or `Records`; if yours is called something else it will say it found nothing, which is not an error — you simply need its real name for Part C.

B2 · Provision the DGO_* platform lists

B2.1 If B1.3 showed DGO_* lists missing, create them:

```
./scripts/setup-sharepoint.ps1 -SiteUrl "V6"
```

Everything comes from the specification file. Nothing is hardcoded in the script, and it is add-only — an existing list or column is reported and left alone.

B2.2 If all ten already exist, skip this. Nothing to do.

B3 · Decide about missing operational columns

B3.1 The report marks with `--` any column the platform reads that your list does not have. The script does **not** add these. Your operational lists hold live records, and adding a column to a register is a change to a system of record that a person should approve.

B3.2 For each one, decide: add the column in SharePoint by hand, or accept that the platform will show a blank there.

B3.3 One is worth particular attention. The public channel needs somewhere to record the **submitter's email address**, because the tracking page matches on reference *and* email — that pairing is what stops somebody who guesses a reference from reading another person's correspondence. If your correspondence list has no sender-email column, add one before opening the public channel. Note its exact internal name; you will map to it in C7.

B4 · The attachment library

B4.1 Confirm you have a document library for correspondence attachments, and record its name.

B4.2 It needs two columns the upload flow writes. Check whether they exist:

Column	Type	What it holds
`ReferenceId`	Single line of text	ties the file to its registry reference
`Sha256`	Single line of text	the digest, so you can prove later that the stored file is byte-for-byte what the citizen sent

B4.3 Add either one that is missing. A library is not a register — adding a column here carries none of the risk of B3.

This is the longest part. Work through it in order.

C1 · Open Power Automate and find your environment

C1.1 Open https://make.powerautomate.com in a browser.

C1.2 Look at the environment selector in the top right corner. Confirm it names the environment this platform uses. If it does not, click it and choose the correct one. Every step below applies to the selected environment only.

C1.3 In the left sidebar click **My flows**.

C1.4 Click the **Cloud flows** tab.

C2 · How to find a flow by its workflow ID

You will do this repeatedly. The workflow ID is the 32-character value in the flow's URL.

C2.1 Click any flow in the list.

C2.2 Look at the browser address bar. The URL contains `/flows/` followed by the workflow ID.

C2.3 To find a specific ID, click each flow in turn and compare, or paste this directly into the address bar, replacing `WORKFLOWID` with the 32-character value you are looking for:

```
https://make.powerautomate.com/environments/ENVIRONMENTID/flows/WORKFLOWID/details
```

Take `ENVIRONMENTID` from the URL of any flow you already opened.

C2.4 If a workflow ID produces "not found", that flow has already been deleted. Tick it off and move on — that is a completed item, not a problem.

C3 · Delete the 9 flows that must be deleted

These have no consumer, or carried two signatures each, where regenerating the trigger could not have revoked both. Deleting the flow is the only action that invalidates every signature it has ever had.

> **The signatures are no longer live.** `ITEM-22` rotated the estate and reconciled it against `endpoint-register.json`: none of the disclosed set authenticates an endpoint this platform calls. The deletions below still stand — a flow with no consumer is still clutter, and a superseded signature in git history is still readable — but do them as housekeeping, not as an exposure response.

C3.1 For each of these nine workflow IDs, find the flow as described in C2:

```
37642ba3597f4cf58288cc71b5e6b519
3931e2ff995242b6b2c920c8b2209797
ff455c68e9ac493e858fb984bcfd01fb
1ff7714c11a74fa4a876f8f6a79b64d2
3fc71cc29d15481291fd341def327572
5b29edc84b5d4a8db3c885d8441aa977
7995c1eb50d94d5daa2780e71391d874
c43388639d14452faef4ca3042a95b23
ca0bafc172114e0bb4853c135246654c
```

C3.2 With the flow open, click ⋯ (More commands) in the top toolbar.

C3.3 Click **Delete**.

C3.4 In the confirmation dialog, click **Delete** again.

C3.5 Repeat C3.1 to C3.4 for all nine. Tick each off as you go.

C3.6 Also delete this one — a retired endpoint the platform no longer calls:

```
02a3a70f3dec4dcd9a85a244a60c65b9
```

C4 · Delete and rebuild the 3 flows that carry two signatures

`OTP_GENERATE`, `OTP_VERIFY` and `FETCH_ALL` each have two published signatures. Regenerating revokes only the newer one. You must delete and rebuild.

C4.1 Open the flow with workflow ID `4a250f97181b4a28abc1d0fb0f7d4c4d` (this is `FETCH_ALL`).

C4.2 Click **Edit**.

C4.3 Take a screenshot of every action in the flow, or click each action to expand it and photograph the configuration. You are about to delete it and will rebuild the same logic.

C4.4 Click ⋯ → **Save As**. Name it:

```
FETCH_ALL v2
```

Click **Save**. This creates a copy with a brand new trigger URL and no history of the old signatures.

C4.5 Open the copy. Click **Edit**. Click the **When an HTTP request is received** trigger to expand it.

C4.6 Copy the **HTTP POST URL** shown. Record it in `deployment-values.txt` labelled `DGO_ENDPOINT_FETCH_ALL`.

C4.7 Click **Save**. Confirm the flow is turned **On** — if the toolbar shows **Turn on**, click it.

C4.8 Go back to the original flow `4a250f97181b4a28abc1d0fb0f7d4c4d` and delete it using C3.2 to C3.4.

C4.9 Repeat C4.1 to C4.8 for workflow ID `314aaf27593147089b38322e5ca25936`, naming the copy `OTP_GENERATE v2` and recording its URL as `DGO_ENDPOINT_OTP_GENERATE`.

C4.10 Repeat C4.1 to C4.8 for workflow ID `43879c5165de439680055ab4258b3f27`, naming the copy `OTP_VERIFY v2` and recording its URL as `DGO_ENDPOINT_OTP_VERIFY`.

C5 · Regenerate the remaining 12 flows

These have one signature each and can be regenerated in place.

C5.1 For each row in the table below, find the flow by its workflow ID as described in C2.

C5.2 Click **Edit**.

C5.3 Click the **When an HTTP request is received** trigger to expand it.

C5.4 Click ⋯ inside the trigger card, then click **Regenerate**. Confirm if asked.

C5.5 The **HTTP POST URL** now shows a new value. Copy the whole URL.

C5.6 Paste it into `deployment-values.txt` against the secret name in the third column.

C5.7 Click **Save**.

C5.8 Move to the next row and repeat.

Workflow ID	This flow is	Record the URL as
`1154b50e1d17420dadb3b012e7e2a02c`	Bulk assignment	`DGO_ENDPOINT_BULK_ASSIGNMENT`
`6b3bad3005b44bf6bced0f8074d3f2ed`	Single assignment	`DGO_ENDPOINT_SINGLE_ASSIGNMENT`
`818ec4053f1e4f0b87845114241d8b74`	Get documents	`DGO_ENDPOINT_GET_DOCS`
`20e3b003a57f47febae8a24ad5b9acd4`	AI document analysis	`DGO_ENDPOINT_AI_DOC_ANALYSIS`
`20e6340941ce4b1bbb87b43c9102a777`	Fetch email attachments	`DGO_ENDPOINT_FETCH_EMAIL_ATTACHMENTS`

Workflow ID	This flow is	Record the URL as
`85c556f10b8244ba9d839a2ebe240b91`	Fetch activities	`DGO_ENDPOINT_FETCH_ACTIVITIES` **and** `DGO_ENDPOINT_SUBSIDIARY_ACTIONS` — record the same URL twice, under both names
`a13c8b577bd44f8787c50d095ea3faf9`	AI chat	`DGO_ENDPOINT_AI_CHAT`
`a942d230337c4ddfa9a386e92bbd048b`	Email related task	`DGO_ENDPOINT_EMAIL_RELATED_TASK`
`bc83d98acf474a088832d78f50085388`	Dynamic actions	`DGO_ENDPOINT_DYNAMIC_ACTIONS` **and** `DGO_ENDPOINT_EMAIL` — record the same URL twice, under both names
`d67f2acb3708449490eed561ee56efbe`	Reference data	`DGO_ENDPOINT_REFERENCE_DATA`
`fe794e0139784ac694768e5a716e0be7`	AI email analysis	`DGO_ENDPOINT_AI_EMAIL_ANALYSIS`

C5.9 Count the entries in `deployment-values.txt`. You should now have 17 URLs recorded (12 flows, two of which are recorded under two names each, plus the three rebuilt in C4).

C6 · Understand what the browser sends to these flows

The browser calls each flow above **directly**. There is no proxy in front of them to verify a token, inject a trustworthy identity, or restrict who may connect. Every flow receives a POST with `Content-Type: application/json`, an `X-Correlation-Id` header, and this body:

```
{
  "action": "the contract action name",
  "payload": { },
  "userEmail": "officer@nitda.gov.ng",
  "requestId": "a UUID",
  "timestamp": "2026-08-04T09:15:22.431Z"
}
```

`userEmail` is asserted by the browser from the signed-in profile **only while platform authentication is inert**, which is the pilot posture (see `docs/architecture/AUTHENTICATION_CONTRACT.md`). Anyone who holds the flow URL can send this same request with any address in `userEmail` and any values in `payload`, because nothing stands between them and the flow. **Treat `userEmail` and everything in `payload` as attacker-controlled.**

When you later activate authentication (`AuthConfig.enabled = true` in `config/auth.config.js`), the browser stops sending `userEmail` and instead attaches `Authorization: ******` At that point the flow must validate the token against the identity provider's JWKS and derive both identity and role from its claims — never from the body. That obligation is set out in

docs/architecture/AUTHENTICATION_CONTRACT.md §2`, and with the proxy gone the flow is the only thing that can honour it.

A flow you can no longer restrict to a proxy. In the proxy design these flows could be reached only from the proxy's egress. That is no longer achievable, and pretending otherwise would hide the risk: the trigger URL is delivered to the browser, so the flow is reachable by anyone who has loaded the page or read the URL. Whatever gates who can load the internal page cannot limit who can POST to a Power Automate URL. Rotate these signatures on a schedule — regenerating the signature is the only way to revoke a leaked one — keep them out of Git, and assume every governed flow is directly reachable from the public internet.

The public intake flows — read before building C7 to C11

The five flows in C7 to C11 receive traffic from the **public** portal: submission, status, upload, email verification and the help desk. In the proxy design the proxy did the security work in front of them — it minted the registry reference, issued and redeemed single-use upload tickets, verified uploaded bytes against the declared size and SHA-256, applied the Universal Filename Policy, generated and checked email verification codes, and rate-limited by source. **That proxy is gone.** Each of these flows is now invoked directly by a stranger's browser, so each must do all of that work itself.

The click-by-click steps below build a working **skeleton** for each flow — trigger, storage, response — so traffic runs end to end. They are deliberately not the finished control. The full request and response contract, and every guarantee each flow must enforce, is the table under **"The contract each flow must satisfy"** in `document-portal/README.md`. Build to that table, not only to the skeleton. In particular:

- the `SUBMISSION` flow (C7) mints the `NITDA-YYYY-<sequence>` reference (unpadded — the live flow issues `NITDA-2026-217`) and must **never** restart the sequence within a year; issues one short-lived, single-use upload ticket per declared attachment; applies the Universal Filename Policy (`config/filename-policy.config.js`, `docs/policies/universal-filename-policy/`) to every attachment name while keeping what the submitter sent as `originalName`; and rate-limits by source;
- the `STATUS` flow (C8) returns a **byte-identical** `404` for an unknown reference and for a wrong email, so it cannot be used to discover whether a reference exists;
- the `UPLOAD` flow (C9) redeems each ticket once and only once and checks the bytes against the size and SHA-256 the submission declared;
- the email-verification flows (C10) generate a one-time code, rate-limit it per address and per source, and compare it as a single-use value. The comparison is not constant-time — WDL has no bitwise operators, so `equals()` is what compares them; the protection is the lifecycle (short expiry, single use, attempt cap), per SC-003 / INT-005.

Nothing above can be delegated to an intermediary any more, because there is no intermediary.

C7 · Build the intake submission flow

This flow does not exist yet. It receives public submissions from the portal, and it is the **single most important flow to get right**: with no proxy in front of it, it is the only place the reference can be minted, the filename policy applied, the rate limit enforced, verification required and the upload tickets issued. Build it to the `SUBMISSION` row of the contract in `document-portal/README.md`; the SharePoint steps below only cover storing the record.

C7.1 In the left sidebar click **Create**.

C7.2 Click **Instant cloud flow**.

C7.3 In **Flow name**, type exactly:

```
DGO Intake Submission
```

C7.4 In the trigger list, scroll to and click **When an HTTP request is received**.

C7.5 Click **Create**.

C7.6 The flow designer opens with the trigger. Click the trigger card to expand it.

C7.7 Click **Use sample payload to generate schema**.

C7.8 Paste this exactly into the box, then click **Done**. This is what the browser actually sends now (see `document-portal/js/submit.js`): note there is **no** `referenceId`, `declaredBytes`, `source` or renamed-filename fields — the proxy used to add those, and the flow must produce them itself. Each attachment carries only its name, size and SHA-256, and a `verification` proof is present only when the submitter has completed the email challenge:

```
{
  "localId": "draft-000000",
  "channel": "Portal",
  "correspondenceType": "Incoming",
  "subject": "Request for policy clarification",
  "category": "General Correspondence",
  "senderEmail": "citizen@example.org",
  "sender": {
    "name": "A Citizen",
    "organisation": "Example Organisation",
    "organisationType": "Private"
  },
  "senderPhone": "+2348000000000",
  "eventDate": "",
  "description": "The body of the correspondence.",
  "attachments": [
    {
      "name": "letter_to_the_dg.pdf",
      "size": 24576,
      "sha256": "0000000000000000000000000000000000000000000000000000000000000000"
    }
  ],
  "submittedAt": "2026-08-04T09:15:22.431Z",
  "verification": ""
}
```

C7.9 Set **Method** to `POST`. If there is no Method field visible, click **Show advanced options** first.

C7.9a Before you store anything, this flow must do the work the proxy used to do. None of it is optional now, because nothing else runs between the anonymous browser and this trigger:

- **Rate-limit by source.** Refuse a caller who submits too often in a window. The trigger URL is public (Part E), so an unthrottled flow is an open relay into the register.
- **Enforce verification when you require it.** If your posture requires a verified email, reject a submission whose `verification` proof is missing or spent by returning HTTP `403` with body `{"error":"verification_required"}` — the portal recognises exactly that shape and asks the submitter for a code rather than queuing a doomed retry.
- **Mint the registry reference.** Generate `NITDA-YYYY-<sequence>` yourself, monotonic within the year, and **never restart the sequence inside a year** — a collision reuses a citizen's reference. Do not zero-pad: the register's own shape is `NITDA-2026-217`. This is the value you return and store, not anything the browser sent.
- **Apply the Universal Filename Policy** to each attachment name, keeping the submitter's original name as `originalName`. See `config/filename-policy.config.js` and `docs/policies/universal-filename-policy/` for the exact normalisation rules.

- **Issue one single-use upload ticket per declared attachment.** You return these in the response; the portal redeems each against the upload flow (C9). Bind each ticket to the reference, the declared size and the declared SHA-256 so C9 can verify the bytes.

C7.10 Click **+ New step**.

C7.11 In the search box type `Create item`. Click the **SharePoint** connector, then click the **Create item** action.

C7.12 Set **Site Address** to your SharePoint site (V6).

C7.13 Set **List Name** to **your existing correspondence list** — the real name you wrote down in B1.4. Do not create a new list for this. The platform loads the register from this list through `FETCH_ALL`; a submission written anywhere else is invisible to every officer.

C7.14 Map the fields. The left column below is what `core/domain.js` actually reads, so these are the names that matter. Your list may use slightly different ones — use the report from B1.3 to confirm each, and map to what you actually have.

Column the platform reads	Set it to
`Title`	Dynamic content: `subject`
`RefIDD` — or `Reference_ID` if that is what your list uses	The reference you minted in C7.9a, **not** an inbound field
`Description`	Dynamic content: `description`
`Category`	Dynamic content: `category`
`AssignmentStatus`	Type the literal text: `Not Assigned`
your sender-email column from B3.3	Dynamic content: `senderEmail`

Leave `RoutedToDSU`, `AssignedTo` and `AttachmentLink` empty. Routing and assignment are decided by a registry officer during triage — filling them here would pre-empt a decision that belongs to a person. `AttachmentLink` is written by the upload flow in C9.

C7.14a If your list has columns for the submitter's name, organisation or phone, map them from `sender/name` using an expression such as:

```
triggerBody()?['sender']?['name']
```

and from `senderPhone` for the phone. If it does not have them, skip this — the platform does not read them, and the full submission is preserved in the audit trail either way.

C7.14b `Created` is set by SharePoint automatically. Do not map `submittedAt` onto it.

C7.15 Click **+ New step**.

C7.16 Search for `Response`. Click the **Request** connector, then **Response**.

C7.17 Set **Status Code** to `200`.

C7.18 In **Body**, return the reference you minted and one ticket per declared attachment. The portal reads `referenceId` and `uploads` from exactly this shape (see `document-portal/js/submit.js`); returning `{"ok":true}` would leave the submitter with no reference and no way to upload their files:

```
{
  "referenceId": "@{outputs('Compose_reference')}",
  "uploads": "@{outputs('Compose_upload_tickets')}"
}
```

C7.19 Click **Save**.

C7.20 Click the trigger card to expand it. Copy the **HTTP POST URL**. Record it in `deployment-values.txt` as `DGO_ENDPOINT_INTAKE_SUBMISSION`.

C8 · Build the status read-back flow

C8.1 Create → **Instant cloud flow** → name it exactly:

```
DGO Intake Status
```

C8.2 Trigger: **When an HTTP request is received**. Click **Create**.

C8.3 Expand the trigger. Click **Use sample payload to generate schema**. Paste exactly, then click **Done**:

```
{
  "referenceId": "NITDA-2026-000001",
  "email": "citizen@example.org"
}
```

C8.4 Set **Method** to `POST`.

C8.5 Click **+ New step**. Search `Get items`. Choose **SharePoint** → **Get items**.

C8.6 Set **Site Address** to V6. Set **List Name** to **your existing correspondence list** — the same one you used in C7.13, not a new one.

C8.7 Click **Show advanced options**. In **Filter Query**, paste this, replacing `RefIDD` with whichever reference column your list uses and `SenderEmail` with the sender-email column you identified in B3.3:

```
RefIDD eq '@{triggerBody()?['referenceId']}' and SenderEmail eq '@{toLower(triggerBody()?['email'])}'
```

Both must match. The reference alone is not enough — requiring the pair is what stops somebody who guesses a reference from reading another person's correspondence.

C8.8 Set **Top Count** to `1`.

C8.9 Click **+ New step**. Search `Condition`. Choose **Control** → **Condition**.

C8.10 On the left side of the condition, click the box and use **Expression**:

```
length(body('Get_items')?['value'])
```

C8.11 Set the operator to **is greater than** and the right side to `0`.

C8.12 In the **If yes** branch, click **Add an action**. Search `Response`. Choose **Request** → **Response**.

C8.13 Set **Status Code** to `200`. In **Body**, paste the following, then correct each column name on the right to match your list — `RefIDD`, `Status`, `Category` and the rest are the names `core/domain.js` reads, but your list is the authority. Any name that does not exist simply returns blank, which is why a mistake here is invisible until a citizen sees an empty tracking page:

```
{
  "referenceId": "@{first(body('Get_items')?['value'])?['ReferenceId']}",
  "status": "@{first(body('Get_items')?['value'])?['Status']?['Value']}",
  "statusLabel": "@{first(body('Get_items')?['value'])?['StatusLabel']}",
  "category": "@{first(body('Get_items')?['value'])?['Category']?['Value']}",
  "subject": "@{first(body('Get_items')?['value'])?['Subject']}",
  "receivedAt": "@{first(body('Get_items')?['value'])?['ReceivedAt']}",
  "acknowledgedAt": "@{first(body('Get_items')?['value'])?['AcknowledgedAt']}",
  "updatedAt": "@{first(body('Get_items')?['value'])?['UpdatedAt']}",
  "closedAt": "@{first(body('Get_items')?['value'])?['ClosedAt']}",
  "actionRequired": "@{first(body('Get_items')?['value'])?['ActionRequired']}",
  "timeline": @{if(empty(first(body('Get_items')?['value'])?['Timeline']), '[]',
first(body('Get_items')?['value'])?['Timeline'])}
}
```

C8.14 In the **If no** branch, click **Add an action**. Search `Response`. Choose **Request** → **Response**.

C8.15 Set **Status Code** to `404`. In **Body**, paste exactly:

```
{
  "ok": false
}
```

This same 404 must answer **both** an unknown reference and a correct reference with the wrong email — byte for byte, with no hint of which was wrong. The single `and` filter in C8.7 gives you that for free: both cases produce an empty result and fall into this one branch. Do not add a distinct "no

such reference" message, or you hand an enumerator a way to discover which references exist. The proxy used to guarantee this uniformity; now the flow is the only thing that can (see the `STATUS` row of `document-portal/README.md`).

C8.16 Click **Save**. Expand the trigger, copy the **HTTP POST URL**, record it as `DGO_ENDPOINT_INTAKE_STATUS`.

About the timeline. Each entry the `STATUS` flow returns to a citizen has an `at`, `status`, `label` and `note`. Return the `note` only when the entry also carries `"public": true`. Registry officers can therefore keep internal minutes on the same timeline; anything not explicitly marked public must be withheld. There is no proxy to strip internal notes on the way out any more, so this filter has to live in the flow — when you add timeline entries later, set `public` deliberately, and confirm the projection never returns an unmarked `note`.

C9 · Build the attachment upload flow

This one receives raw file bytes, not JSON, and it now serves **two direct callers** that the proxy used to normalise into one:

- the public portal `UPLOAD` — a `PUT` of one attachment's raw bytes with a single-use `X-Upload-Ticket` header that the `SUBMISSION` flow issued (see the `UPLOAD` row in `document-portal/README.md`);
- the internal registry counter deposit `SCAN_INTAKE` — a `PUT` of raw bytes with `X-DGO-Filename`, `X-DGO-Size` and `X-DGO-Sha256` headers, driven by `core/scan-intake-service.js` (see `config/config.example.js`).

With the proxy gone, **nothing verifies the bytes before this flow runs.** The flow must redeem the ticket exactly once, recover the reference, filename, declared size and SHA-256 it stands for, compute the SHA-256 of what actually arrived, and refuse anything oversize or mismatched. The steps below build the storage skeleton only; the ticket redemption and byte verification are yours to add, to the contract in `document-portal/README.md`.

C9.1 Create → **Instant cloud flow** → name it exactly:

```
DGO Intake Upload
```

C9.2 Trigger: **When an HTTP request is received**. Click **Create**.

C9.3 Expand the trigger. **Leave the schema box empty.** Do not click "Use sample payload to generate schema". The body is binary, and a JSON schema would corrupt every non-text file.

C9.4 Set **Method** to `PUT`. Both callers use `PUT`; the browser's `fetch` in `document-portal/js/core.js` sends the bytes with `PUT` and `Content-Type: application/octet-stream`.

C9.5 Click **Show advanced options**. If a **Content-Type** field appears, set it to:

```
application/octet-stream
```

C9.6 Click **+ New step**. Search `Create file`. Choose **SharePoint** → **Create file**.

C9.7 Set **Site Address** to V6.

C9.8 Set **Folder Path** to your correspondence attachment library, the one you confirmed in B4.1. For example:

```
/Shared Documents/Correspondence
```

C9.9 In **File Name**, click **Expression**. The name must combine the registry reference with the original filename so two citizens who both send `letter.pdf` do not overwrite each other. For the registry counter path that is:

```
concat(triggerOutputs()['headers']?['X-DGO-Reference'], '_',
decodeUriComponent(triggerOutputs()['headers']?['X-DGO-Filename']))
```

For the public path the reference and filename come from the redeemed `X-Upload-Ticket`, not from a header — substitute the values you recover when you redeem it. Decode any name that was percent-encoded in transit before you store it.

C9.10 In **File Content**, click **Expression** and paste exactly:

```
triggerBody()
```

C9.11 Click **+ New step**. Search `Update file properties`. Choose **SharePoint** → **Update file properties**.

C9.12 Set **Site Address** to V6. Set **Library Name** to the same library as C9.8.

C9.13 Set **Id** to the dynamic content **ItemId** from the Create file step.

C9.14 Set `ReferenceId` to the reference you recovered — from the redeemed ticket on the public path, or from the registry counter deposit on the scan path.

C9.15 Set `Sha256` to the digest you computed and verified against what the caller declared. Storing the digest lets you prove later that the stored file is byte-for-byte what the citizen sent.

C9.16 Click **+ New step**. Search `Response`. Choose **Request** → **Response**.

C9.17 Set **Status Code** to `200`. In **Body**, paste exactly:

```
{
  "stored": true,
  "attachmentLink": "@{body('Create_file')?['{Link}']}"
}
```

The portal reads `attachmentLink` (and `stored`) from your response, per the `UPLOAD` row in `document-portal/README.md`; the registry counter path reads the same `attachmentLink`. If you return neither, the file is stored but nothing links to it from the register.

C9.18 Click **Save**. Expand the trigger, copy the **HTTP POST URL**, and record it under **both** of these names in `deployment-values.txt`:

```
DGO_ENDPOINT_INTAKE_UPLOAD
DGO_ENDPOINT_SCAN_UPLOAD
```

One library serves both the public channel and registry counter deposits. Set them to different flows later only if counter deposits must be filed separately.

Headers this flow receives:

Caller	Header	Contents
Public portal	`X-Upload-Ticket`	The single-use ticket the `SUBMISSION` flow issued; redeem once
Registry counter	`X-DGO-Filename`	The filename, percent-encoded
Registry counter	`X-DGO-Size`	Declared byte length, to check against what arrived
Registry counter	`X-DGO-Sha256`	SHA-256 the caller declared, 64 lowercase hex characters

Nothing has verified that the bytes match the declared SHA-256 and size before this flow runs. The proxy used to; it is gone. Hash the received bytes here and refuse any mismatch, or you will file whatever anyone chooses to send.

C10 · Build the email verification flows

The portal verifies an email address in two calls, and the proxy used to sit between them: it generated the one-time code, held it, rate-limited the requests and checked the answer. That logic is gone. You now need **two** flows — one to issue a code (`VERIFY`) and one to check it (`VERIFY_CONFIRM`) — and the code generation, storage, expiry, single-use semantics and rate limiting all have to live inside them. Build them to the `VERIFY` and `VERIFY_CONFIRM` rows of the contract in `document-portal/README.md`.

C10.1 Create → **Instant cloud flow** → name it exactly:

```
DGO Intake Verify Email
```

C10.2 Trigger: **When an HTTP request is received**. Click **Create**.

C10.3 Expand the trigger, click **Use sample payload to generate schema**, paste exactly, click **Done**. The browser sends only the address; it does **not** send the code, because the flow is what invents it now:

```
{
  "email": "citizen@example.org"
}
```

C10.4 Set **Method** to `POST`.

C10.5 Generate a one-time code and an expiry inside the flow — the browser no longer supplies them. Add a **Compose** (or your preferred store) that produces a six-digit code and a short-lived `expiresAt`, and persist the pair against the address (a SharePoint list keyed by email, or Dataverse) so the confirm flow in C10.11 can read it back. Rate-limit here: refuse to mint more than a few codes per address and per source in a window, or the flow becomes a free mail relay for anyone who knows the URL.

C10.6 Click **+ New step**. Search `Send an email`. Choose **Office 365 Outlook** → **Send an email (V2)**.

C10.7 Set **To** to the dynamic content `email`.

C10.8 Set **Subject** to exactly:

```
Your NITDA correspondence verification code
```

C10.9 Click **</>** (Code view) in the Body toolbar, then paste exactly, substituting the code and expiry you generated in C10.5:

```
<p>Your verification code is <strong>@{outputs('Compose_code')}</strong></p>
<p>Enter this code on the NITDA document portal to complete your submission. The code
expires at @{outputs('Compose_expiresAt')}.</p>
<p>If you did not request this code, ignore this message. No submission has been made.</p>
<p>Directorate of Digital Operations<br>National Information Technology Development
Agency</p>
```

C10.10 Click **+ New step**. Search `Response`. Choose **Request** → **Response**. Set **Status Code** to `200`. In **Body**, return whether the mail was sent and when the code expires — the portal shows the countdown from `expiresAt`:

```
{
  "sent": true,
  "expiresAt": "@{outputs('Compose_expiresAt')}"
}
```

Click **Save**. Expand the trigger, copy the **HTTP POST URL**, record it as `DGO_ENDPOINT_INTAKE_VERIFY_EMAIL`.

C10.11 Now build the companion **confirm** flow. Create → **Instant cloud flow** → name it exactly `DGO Intake Verify Confirm`. Trigger **When an HTTP request is received**, method `POST`, with this sample payload:

```
{
  "email": "citizen@example.org",
  "code": "000000"
}
```

Read back the code you stored in C10.5 and compare it as a **single-use** value — the comparison is not constant-time — delete or mark it spent on first use so a code cannot be replayed — honour the expiry, and return the verification proof that the `SUBMISSION` flow will later accept:

```
{
  "verification": "@{outputs('Compose_proof')}",
  "expiresAt": "@{outputs('Compose_expiresAt')}"
}
```

Click **Save**, copy the **HTTP POST URL**, and record it as `DGO_ENDPOINT_INTAKE_VERIFY_CONFIRM`. This value has no row in the recorded set the proxy left behind — the proxy performed the check itself — so it is a genuine gap you are filling now. Note it in your value register (Part E) so nobody assumes the portal's `VERIFY_CONFIRM` endpoint can be left blank.

C11 · Build the support case flow

C11.1 Create → **Instant cloud flow** → name it exactly:

```
DGO Intake Support
```

C11.2 Trigger: **When an HTTP request is received**. Click **Create**.

C11.3 Expand the trigger, click **Use sample payload to generate schema**, paste exactly, click **Done**:

```
{
  "name": "A Citizen",
  "email": "citizen@example.org",
  "topic": "Tracking a submission",
  "message": "I cannot find my reference.",
  "aboutReference": "NITDA-2026-000001",
  "caseRef": "CASE-2026-000001",
  "receivedAt": "2026-08-04T09:15:22.431Z",
  "source": "document-portal"
}
```

C11.4 Set **Method** to `POST`.

C11.5 Click **+ New step**. Search `Create item`. Choose **SharePoint** → **Create item**. Set **Site Address** to V6, **List Name** to `SupportCases`.

Unlike the correspondence flows, this list genuinely is new. A support case is a question about the platform, not a piece of correspondence, and putting the two in one list would mean help-desk queries appearing in the register as though somebody had written to the agency. Create it with a `CaseRef` (unique), `Name`, `Email`, `Topic`, `Message`, `AboutReference` and `ReceivedAt` column — or skip this flow entirely, since the help desk is optional.

C11.6 Fill the fields with the matching dynamic content:

Field	Dynamic content
`Title`	`caseRef`
`CaseRef`	`caseRef`
`Name`	`name`
`Email`	`email`
`Topic`	`topic`
`Message`	`message`
`AboutReference`	`aboutReference`
`ReceivedAt`	`receivedAt`

C11.7 Click **+ New step**. Search `Response`. Choose **Request** → **Response**. Set **Status Code** to `200`, **Body** to exactly:

```
{
  "ok": true
}
```

C11.8 Click **Save**. Expand the trigger, copy the **HTTP POST URL**, record it as `DGO_ENDPOINT_INTAKE_SUPPORT`.

C12 · Check your list of URLs

C12.1 Open `deployment-values.txt`. Count the recorded URLs. You must have exactly 23 entries under these names:

```
DGO_ENDPOINT_FETCH_ALL
DGO_ENDPOINT_FETCH_ACTIVITIES
DGO_ENDPOINT_SUBSIDIARY_ACTIONS
DGO_ENDPOINT_REFERENCE_DATA
DGO_ENDPOINT_GET_DOCS
DGO_ENDPOINT_FETCH_EMAIL_ATTACHMENTS
DGO_ENDPOINT_SINGLE_ASSIGNMENT
DGO_ENDPOINT_BULK_ASSIGNMENT
DGO_ENDPOINT_DYNAMIC_ACTIONS
DGO_ENDPOINT_EMAIL
DGO_ENDPOINT_EMAIL_RELATED_TASK
DGO_ENDPOINT_AI_EMAIL_ANALYSIS
DGO_ENDPOINT_AI_DOC_ANALYSIS
DGO_ENDPOINT_AI_CHAT
DGO_ENDPOINT_OTP_GENERATE
DGO_ENDPOINT_OTP_VERIFY
DGO_ENDPOINT_INTAKE_SUBMISSION
DGO_ENDPOINT_INTAKE_UPLOAD
DGO_ENDPOINT_INTAKE_STATUS
DGO_ENDPOINT_INTAKE_SUPPORT
DGO_ENDPOINT_INTAKE_VERIFY_EMAIL
DGO_ENDPOINT_INTAKE_VERIFY_CONFIRM
DGO_ENDPOINT_SCAN_UPLOAD
```

`DGO_ENDPOINT_INTAKE_VERIFY_CONFIRM` is the one the proxy used to cover itself (C10.11); it now needs a real flow and a recorded URL like the rest.

C12.2 If any is missing, go back and complete that flow before continuing.

C12.3 Two endpoint names are deliberately absent: `DGO_ENDPOINT_DISPATCH_OUTBOUND` and `DGO_ENDPOINT_ARCHIVE_REFERENCE`. No flow exists for them and the platform is built to work without them. Do not create them for the pilot.

Part 7 · Minimal pilot — the short path

> **What this part is.** The shortest path that gets correspondence flowing end to end — six flows rather than twenty-four, about 55 minutes. It stood as `MINIMAL-PILOT.md` until 2026-09-24 and is carried here unchanged.
>
> **Part 6** is the same build with every feature at once. **Part 1** is the cutover in front of the estate today, and is what to read first if the estate is already built.

About 55 minutes, most of it building flows.

This gets correspondence flowing end to end: a citizen submits, the registry sees it, an officer triages, assigns, approves, dispatches and closes. Nothing else.

Hosting the front end and gating who may load it are handled outside this repository — not covered here. See **Part 6 of this runbook** instead if you want every SharePoint/Power Automate feature built at once (all 24 flows, not just the pilot's six). Come back to it later to add the ones you skip here — each is one line in your config file and a redeploy of the static site.

Why this is so much smaller

Every governed write in the platform — register, triage, treat, approve, dispatch, close, archive — goes through **one** endpoint, `DYNAMIC_ACTIONS`. The platform needs six endpoints, not twenty-four. The other eighteen are AI analysis, one-time passcodes, the email desk, the help desk, scan intake and email verification (which is now two flows). All can wait.

That also simplifies the security work. Rather than carefully regenerating twenty-odd triggers, you **delete everything you are not using** — 21 flows — and regenerate the three you are. Deleting is faster than regenerating and it is the stronger action: it invalidates every signature a flow has ever had, including the older ones that regeneration leaves live.

One thing you cannot skip, however small the pilot. The browser now calls every flow directly — there is no proxy, Worker or broker in between. The flow trigger URLs are shipped to every visitor's browser, so they are public. That means each flow you keep is the only place its own validation, rate limiting, reference minting and upload verification can happen. A minimal pilot is still a public endpoint on the open internet; build the flows accordingly.

---

1 · Prepare — 10 minutes

```
cd /path/to/ECM_DOCS_DEV
git checkout main && git pull
npm install
```

Then see where you stand before changing anything:

```
npm run commission
```

It lists every obligation between this repository and a live deployment, and exits non-zero until they are met. Re-run it after each step below.

Create a plain values file by hand, outside the repository, to hold the flow URLs as you collect them:

```
printf '# DGO pilot flow URLs — delete when deployed\n' > ~/dgo-values.txt
```

Leave it open. You will fill it in as you go. Every URL in it is a bearer credential, which is why it lives outside the repository and is deleted at the end.

2 · SharePoint — 10 minutes

Do not create new lists. Your correspondence lists already exist and the platform already reads them. First find out what you have — this changes nothing:

```
Install-Module PnP.PowerShell -Scope CurrentUser     # first time only
./scripts/setup-sharepoint.ps1 -SiteUrl "https://YOURTENANT.sharepoint.com/sites/YOURSITE"
-WhatIf
```

The report tells you three things:

1. Which of the ten `DGO_*` platform lists exist. If any are missing, run the same command without `-WhatIf` to create them from `docs/reference/sharepoint-provisioning-spec.json`.
2. The real name of your correspondence list. **Write it down** — you need it in step 3d.
3. Which columns the platform reads that your lists do not have. These are reported, never added: those lists hold live records.

One missing column matters before you open the public channel: somewhere to store the **submitter's email**. The tracking page matches reference *and* email, and that pairing is what stops somebody who guesses a reference from reading another person's correspondence. If your list has no such column, add it by hand and note its internal name.

Your attachment library also needs `ReferenceId` and `Sha256` columns. Add them if absent — a library is not a register, so there is no risk in doing so.

3 · Power Automate — 45 minutes

Open https://make.powerautomate.com and check the environment selector in the top right is the right environment.

3a · Delete the 21 flows you are not using — 20 minutes

To open a flow by ID, paste this in the address bar, replacing `WORKFLOWID` — take `ENVIRONMENTID` from the URL of any flow you already have open:

```
https://make.powerautomate.com/environments/ENVIRONMENTID/flows/WORKFLOWID/details
```

Then ⋯ → **Delete** → **Delete**.

```
02a3a70f3dec4dcd9a85a244a60c65b9    d67f2acb3708449490eed561ee56efbe
7e71fffe770a45ccb93bf216bb53786e    fe794e0139784ac694768e5a716e0be7
818ec4053f1e4f0b87845114241d8b74    20e3b003a57f47febae8a24ad5b9acd4
20e6340941ce4b1bbb87b43c9102a777    a13c8b577bd44f8787c50d095ea3faf9
314aaf27593147089b38322e5ca25936    a942d230337c4ddfa9a386e92bbd048b
43879c5165de439680055ab4258b3f27    85c556f10b8244ba9d839a2ebe240b91
37642ba3597f4cf58288cc71b5e6b519    3931e2ff995242b6b2c920c8b2209797
ff455c68e9ac493e858fb984bcfd01fb    1ff7714c11a74fa4a876f8f6a79b64d2
3fc71cc29d15481291fd341def327572    5b29edc84b5d4a8db3c885d8441aa977
7995c1eb50d94d5daa2780e71391d874    c43388639d14452faef4ca3042a95b23
ca0bafc172114e0bb4853c135246654c
```

"Not found" means it is already gone. Tick it off.

This is the security half of the whole deployment. Every one of these has a published trigger URL that works right now, and nothing in the platform calls it.

3b · Regenerate the 3 you keep — 10 minutes

For each: open it → **Edit** → click the **When an HTTP request is received** trigger → ⋯ inside the trigger → **Regenerate** → copy the new **HTTP POST URL** → **Save**.

Workflow ID	Paste the URL into `~/dgo-values.txt` as
`bc83d98acf474a088832d78f50085388`	`DGO_ENDPOINT_DYNAMIC_ACTIONS`
`6b3bad3005b44bf6bced0f8074d3f2ed`	`DGO_ENDPOINT_SINGLE_ASSIGNMENT`
`1154b50e1d17420dadb3b012e7e2a02c`	`DGO_ENDPOINT_BULK_ASSIGNMENT`

3c · Rebuild FETCH_ALL — 5 minutes

Workflow `4a250f97181b4a28abc1d0fb0f7d4c4d` carried **two** signatures, so regenerating would have revoked only the newer one — which is why this step rebuilds rather than regenerates.

> **Not an exposure step any more.** `ITEM-22` rotated the estate; neither signature is live. Rebuild here if you want the flow's history clean, and skip it if you do not.

1. Open it → ⋯ → **Save As** → name it `FETCH_ALL v2` → **Save**.
2. Open the copy → **Edit** → click the trigger → copy the **HTTP POST URL** → **Save**.
3. Make sure it is turned **On**.
4. Paste the URL as `DGO_ENDPOINT_FETCH_ALL`.
5. Go back and **delete the original**.

3d · Import the two intake flows — 5 minutes

These do not exist yet, but they no longer need to be built by hand: they ship pre-built in `docs/folio-Master-Flows-and-Endpoints/packages/` as flows **1** (`01-portal-submission`) and **2** (`02-portal-upload`) — complete, importable definitions, not a description of one.

1. Create the flow with an HTTP trigger (any method), select the trigger on the canvas, and paste the whole contents of `folio-Master-Flows-and-Endpoints/packages/paste/01-portal-submission.paste.json` → its URL is `DGO_ENDPOINT_INTAKE_SUBMISSION`.
2. Repeat with `02-portal-upload.paste.json` → its URL is `DGO_ENDPOINT_INTAKE_UPLOAD`.
3. Re-pick the `SharePoint` connection on each (Power Automate always drops connection authorisation on paste — this is normal, not a defect in the import).

If canvas paste isn't available in your designer, `folio-Master-Flows-and-Endpoints/packages/README.md` §"Alternative import paths" has the same two flows as `codeview/*.definition.json` (paste into a flow's Code view) or as reference `*.flow.json` packages. `FLOW-BUILD-WALKTHROUGH.md`'s C7/C9 steps are kept for reference — reading them tells you what each flow does, action by action — but building from them by hand is no longer the fastest correct path now that the flows themselves exist.

With no proxy in front of them these two flows carry the whole security burden of the public channel: the submission flow mints the `NITDA-YYYY-<sequence>` reference itself (unpadded, monotonic, never restarting within a year), rate-limits by source and issues one single-use upload ticket per attachment; the upload flow redeems that ticket once and verifies the received bytes against the declared size and SHA-256. That is already built into both — verify it holds by running the checks in `folio-Master-Flows-and-Endpoints/packages/README.md` before trusting the import, not by re-deriving it.

The other four flows in the same folder (`03` – `06`: support, verify-request, verify-confirm, status) cover the portal features the minimal pilot skips for now — import them the same way, later, if you add tracking, email verification, or the help desk (see "What you skipped" below).

4 · Gate who may load the internal page

Whatever keeps the internal interface off the open internet is set up and operated outside this repository. The platform has exactly six roles — `DGO-SystemAdmin`, `DGO-UserAdmin`, `DGO-Executive`, `DGO-Director`, `DGO-Operator`, `DGO-Viewer` — and every pilot officer needs to land in exactly one of them, however your access gate models that. Nothing here sits between the loaded page and the flows (step 3), so in this pilot an officer's role inside the app is advisory regardless of which gate you choose (see step 6). That gate is what you build real authorisation on later per `docs/architecture/AUTHENTICATION_CONTRACT.md`.

5 · Configure the endpoints — 5 minutes

There is no Worker to deploy and no secrets to set. The flow URLs you collected go into two git-ignored files that ship with the static site.

Create `config/config.local.js` with the four platform endpoints the minimal pilot uses, pasting each URL from `~/dgo-values.txt`:

```
window.DGO_CONFIG = {
  endpoints: {
    FETCH_ALL:         "DGO_ENDPOINT_FETCH_ALL",
    DYNAMIC_ACTIONS:   "DGO_ENDPOINT_DYNAMIC_ACTIONS",
    SINGLE_ASSIGNMENT: "DGO_ENDPOINT_SINGLE_ASSIGNMENT",
    BULK_ASSIGNMENT:   "DGO_ENDPOINT_BULK_ASSIGNMENT",
  },
};
```

Create `document-portal/config.local.js` — beside `index.html`, not in `js/` — with the two intake endpoints:

```
window.PF_CONFIG = {
  endpoints: {
    SUBMISSION: "DGO_ENDPOINT_INTAKE_SUBMISSION",
    UPLOAD:     "DGO_ENDPOINT_INTAKE_UPLOAD",
  }
};
```

Or let the setup script write both from the values file you have been filling in — it uses exactly the `DGO_ENDPOINT_*` / `PF_ENDPOINT_*` names recorded above:

```
npm run setup -- --values ~/dgo-values.txt --force
```

Both files are already git-ignored. Confirm before you deploy:

```
npm run commission
```

It must exit 0. Beyond checking that nothing is staged, it fails if any endpoint you just wired carries a signature that is still published in this repository — meaning that trigger was never regenerated, and going live on it means going live on a credential anyone with repository access already holds. A signed flow URL committed to Git is a leaked credential that outlives deleting the file. Every URL in these two files is delivered to every visitor's browser, so treat them as public and rotate them on a schedule.

6 · Deploy the static site — 5 minutes

Both config files must exist on disk first — whatever you use to deploy needs to publish the working directory as-is, config files included. How you host the front end and how you gate who may load it are decisions made outside this repository; make sure the deployed hostname matches whatever application domain your access gate is configured against, or the internal interface is reachable without a sign-in.

7 · Check it works — 10 minutes

Submit twice. Replace `SUBMISSION_URL` with the URL you recorded as `DGO_ENDPOINT_INTAKE_SUBMISSION`:

```
curl -s -X POST SUBMISSION_URL \
  -H 'Content-Type: application/json' \
  -d '{"subject":"Check one","category":"General
Correspondence","senderEmail":"registry@nitda.gov.ng","sender":
{"name":"Registry"},"description":"First check."}'
```

Run it again with `Check two`. Each response must contain a `"referenceId":"NITDA-` value, and the two must be **different, consecutive numbers**. If a number repeats, your `SUBMISSION` flow is restarting the sequence — go back to C7.9a and fix the minting. (If you built the flow to require a verified email it answers `403 verification_required` instead; test through the portal with a verified address.)

Look in SharePoint. The `Correspondence` list must hold both, with `Received` in Status.

Understand what the authenticated side does for a stranger. Replace `FETCH_ALL_URL` with the URL you recorded as `DGO_ENDPOINT_FETCH_ALL`:

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST FETCH_ALL_URL \
  -H 'Content-Type: application/json' -d '{}'
```

There is no Worker to refuse this. If your `FETCH_ALL` flow does not verify a token — the inert pilot default — it will answer `200` and hand the register to an anonymous caller. That is a known limitation of the pilot posture, not a misconfiguration: the URL is only shielded by being unadvertised and by whatever gates the interface, never the flow. If a `200` here is unacceptable for your data, build token verification into the flow before going live.

Have officers sign in. One from each group opens the deployed URL, signs in through whatever gates the interface, and adds `#/diagnostics` to the address bar. Each sees their email and a role — but in the inert pilot that role is **advisory**, read from the profile the interface holds, not verified against any group claim. Use it to confirm the interface renders per role, not as proof that privilege is enforced anywhere.

Confirm the register is shared. One officer registers a correspondence; a second, on a different machine, finds it under `#/lookup`. If they cannot, `FETCH_ALL` is not reaching its flow — check it is set in `config/config.local.js` and that the deployed site includes that file.

8 · Before real correspondence arrives

Delete the test records from the `Correspondence` list, and delete `~/dgo-values.txt` — it holds the signed flow URLs, and each one is a bearer credential.

Confirm the routing table below. It decides which desk each kind of correspondence lands on, and nobody has approved it yet.

H1 Open `config/correspondence-categories.config.js` in the repository.

H2 Read this table:

Document kind	Currently routes to
Ministerial Directive	Executive Correspondence
Policy Submission	Policy / Regulation
Compliance Filing	Policy / Regulation
Application	Operations
Proposal	Operations
Project Proposal	Operations
Report	Operations
Meeting Request	General Administration
Event Invitation	General Administration
Official Correspondence	General Administration
General Correspondence	General Administration

H3 Confirm each row with whoever owns registry policy, or write down the corrections.

H4 Confirm which kinds the public may choose on the portal. Currently seven of the eleven are offered: General Correspondence, Application, Proposal, Report, Compliance Filing, Policy Submission, Event Invitation.

H5 To change a routing destination, edit the `DocumentKindRouting` object in that file so the kind points at the correct destination, then run:

```
node tests/categories.test.mjs
```

It fails if any kind is left without a routing rule.

H6 If you change the list of kinds, update the `Category` choices in the SharePoint `Correspondence` list (step 2) to match exactly.

What you skipped, and how to add it

Each is one line added to your config file and a redeploy of the front end (step 6).

To add	Import from `folio-Master-Flows-and-Endpoints/packages/`	Then set the key
Citizens tracking their submission	`06-portal-status`	`STATUS` in `PF_CONFIG`
Outward correspondence email	new flow — not in this set	`EMAIL` in `DGO_CONFIG`
Email verification before a reference is issued	`04-portal-verify-request` and `05-portal-verify-confirm`	`VERIFY` and `VERIFY_CONFIRM` in `PF_CONFIG`, then have the `SUBMISSION` flow require the proof
The public help desk	`03-portal-support`	`SUPPORT` in `PF_CONFIG`
Registry counter scan deposits	reuse the upload flow	`SCAN_INTAKE` in `DGO_CONFIG`
AI analysis and one-time passcodes	new flows — not in this set	the matching key in `DGO_CONFIG`

Same import method as §3d for the four rows that ship pre-built. The two that don't (outward email, AI/OTP) are new flows to build from scratch, the same way `3a` – `3c` regenerated the internal ones — not by restoring anything you deleted in 3a, since those all had published credentials, which is why they were deleted. Whether verification is required is now a decision the `SUBMISSION` flow makes for itself — there is no server flag to flip.

The flow URLs are public — treat them so

There is no per-instance ticket or rate-counter caveat any more, because there is no Worker. What replaces it matters more: every signed flow URL in your two config files is delivered to every visitor's browser and can be read straight out of the page. Assume a hostile caller has each one from the moment you deploy. Two consequences you cannot pilot your way around:

- **Each flow must be safe when a stranger calls it** — it must validate its own input, rate-limit its own callers, mint its own reference and verify its own uploads. Whatever fronts the interface never sees a flow call, so it cannot help.
- **Rotate the URLs on a schedule** — regenerate the SAS signature in Power Automate, paste the new URL into the config file, and redeploy. That is the only way to revoke an exposed URL.

Part 8 · Commissioning from a phone — Termux and the harvest bookmark

> **What this part is.** The same commissioning path as **Part 1**, on Android, where there is no devtools console and no desktop terminal. It stood as `CLEAR-THE-LAST-BLOCKER-TERMUX.md` until 2026-09-24 and is carried here so the estate has one runbook rather than one per device.
>
> **This part owns** `scripts/harvest-trigger-urls.bookmarklet.txt`. §4a below is the only place in this repository that tells anyone to run it, which is what `docs/reference/document-`

> `ownership.json` records. It performs the step **Part 3 §3.2** owns — the same two Flow API calls per flow — by the one mechanism a phone has; §3.2 remains the source for the step itself, and this part restates none of it.

§0 · Read this before you start

0.1 One values file, generated for you

	What it is	Who creates it	Where
`~/dgo-values.txt`	**The values file.** 25 `KEY=URL` lines.	**§4a's bookmark** writes it complete, signatures and all. Otherwise `npm run values:template` (§3) writes it complete but for the signatures, and §4 fills those in.	Termux's own home folder, outside the repository. Deleted at the end.
`config/config.local.js`	Internal platform config	`npm run setup` writes it	Inside the repository. Git-ignored.
`document-portal/config.local.js`	Public portal config	`npm run setup` writes it	Inside the repository. Git-ignored.

You do not create the values file by hand and you do not edit the other two at all.

In Termux, `~` is `/data/data/com.termux/files/home` — Termux's own private folder, not your phone's Downloads or Documents. That is the right place: no other app can read it. Do **not** put it in shared storage.

0.2 The signature is the password

A trigger URL ends `sig=…`. That parameter **is** the password to the flow — anyone holding it can invoke it.

- **Never paste one into a chat, a ticket, an email, or a notes app**, including a Claude session. All keep it permanently, and deleting the message revokes nothing. Only regenerating the trigger in Power Automate revokes anything.
- **On a phone this matters more, not less.** The Android clipboard is readable by your keyboard app and, on some versions, by whatever is in the foreground. Copy, switch straight to Termux, pipe it in, clear the clipboard (§4.4).
- Do not let a screenshot or screen recorder catch the designer with the URL field expanded.

0.3 Two apps, and they must come from the same place

You need **Termux** and **Termux:API**, both from **F-Droid**:

- Termux — https://f-droid.org/packages/com.termux/
- Termux:API — https://f-droid.org/packages/com.termux.api/

Two traps that cost an hour each:

- **The Google Play build of Termux is deprecated** and its packages no longer update. Uninstall it and install from F-Droid.
- **Both apps must come from the same source.** F-Droid Termux plus Play Store Termux:API fails on a signature mismatch, and the symptom is not an error — the `termux-*` commands simply hang.

Termux:API is not strictly required for §4 — §4.3 gives a fallback — but it is what makes that a one-command-per-key job, and **§4a cannot work without it**: the clipboard is how the harvested file crosses from Chrome to Termux without ever being displayed. Install it.

---

§1 · Set Termux up

Open Termux. You get a prompt ending in `$`.

1.1 Update. First run only; a minute or two.

```
pkg update && pkg upgrade -y
```

1.2 Install.

```
pkg install -y git nodejs nano termux-api
```

`git` fetches the repository · `nodejs` runs everything here · `nano` is a text editor, since Termux ships with none · `termux-api` is the command-line half of the Termux:API app.

1.3 Check.

```
git --version
node --version
```

Expected: any `git version 2.x`, and Node **22 or higher**. Below 22 this repository fails with a syntax error, not a version message — try `pkg install -y nodejs-lts`, or `pkg update` then `pkg upgrade nodejs`.

1.4 Extra keys row. Long-press the terminal → **Keyboard** → enable it. That gives you `CTRL`, `TAB` and arrows. On most devices **Volume-Down also acts as Ctrl**.

1.5 Stop Android killing Termux while you are in Chrome.

```
termux-wake-lock
```

Release it at the end with `termux-wake-unlock`. Also set Android Settings → Apps → Termux → Battery to **Unrestricted**.

---

§2 · Get the repository

```
cd ~
git clone https://github.com/dgoeaa/ECM_DOCS_DEV.git ecm_docs_dev
cd ~/ecm_docs_dev
git checkout claude/system-remediation-gaps-ahpmsy
```

About **33 MiB**, unpacking to roughly 66 MB — use Wi-Fi if metered. With two-factor authentication on GitHub, `git` asks for a username and a **personal access token**, not your password.

Already cloned? `cd ~/ecm_docs_dev && git checkout claude/system-remediation-gaps-ahpmsy && git pull`

```
pwd
git branch --show-current
```

Expected: a path ending in `ecm_docs_dev`, and `claude/system-remediation-gaps-ahpmsy`.

Every remaining command is typed in Termux, in this folder. If Termux restarts, `cd ~/ecm_docs_dev` first.

You do not need `npm install` — which is why this works on a phone at all.

Confirm the ids are reconciled against the tenant:

```
npm run reconcile
```

Expected: `25 keys`, `20 distinct workflows`, `25/25 carry a complete URL template`. If it lists keys moving to different workflows, that is the register correcting the repository — expected.

See where you stand:

```
npm run commission
```

Scroll to the top for the ⛔ `BLOCKER` section. Unwired endpoints are the blocker; §4a, §5 and §6 clear it, or §3–§6 the long way.

---

§3 · Generate the values file

Read §4a before you start this. It fetches all 24 URLs from the API in one tap and writes this file complete, which makes §3 and §4 unnecessary. Come back here if the bookmark will not run on your phone, or if §4a reports a flow you do not co-own. This route always works.

```
umask 077
npm run values:template ~/dgo-values.txt
```

`umask 077` makes it readable by your account alone. Expected:

```
✅ wrote /data/data/com.termux/files/home/dgo-values.txt
   25 keys — 18 internal, 7 portal
   25 arrive with the URL already complete; add the signature after each 'sig='.
```

Read the flow headings without editing anything:

```
grep '^#   workflow' ~/dgo-values.txt
```

Twenty workflows serve the 25 keys — so twenty visits to Power Automate, not twenty-five.

---

§4 · Twenty signatures, one command each

4.1 Copy one in the browser

Open **Chrome** and sign in to Power Automate with an account that can **edit** flows in environment `Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1`.

Turn on desktop mode first: Chrome → ⋮ → tick "Desktop site", and rotate to landscape. The designer does not render usably in mobile layout.

For each workflow id in the values file:

1. Address bar: `https://make.powerautomate.com/environments/Default-ca6a4b3f-9123-49bc-bcb9-27085ebbf1a1/flows/<id>/details`
2. **Edit** → tap **When an HTTP request is received** to expand it.
3. Find **HTTP POST URL** (newer builds label it **HTTP URL**).
4. **Use the copy icon.** Dragging to select is how a copy gets truncated on a touch screen.
5. **Do not tap Regenerate.** For these 25 keys the rotation is behind you — regenerating now revokes the URL a deployed surface is using and means coming back for a new one.

You may copy the whole URL — §4.2 takes the signature out of it and discards the rest.

"Already rotated" has a scope, and it is exactly these 25 keys. `npm run rotation` reports ROTATED because all 25 resolve to workflows other than the ones the published signatures belong to, so none of those signatures authenticates anything the platform calls. The same report states what it cannot establish: that a flow *outside* the 25 was regenerated. That wider worklist is 39 flows and it belongs to §3 of **Part 3 of this runbook** — which is why that document reads as though the rotation is still ahead of you. Both are true of different sets of flows.

4.2 Put it on the right line, without an editor

Switch to Termux. One command per key, naming the key from the heading above its line:

```
termux-clipboard-get | npm run values:sign -- ~/dgo-values.txt FETCH_ALL
```

Expected: ✅ `FETCH_ALL signed — 1/25 keys complete`.

Where a heading says `2 keys, ONE url`, run it twice with the same clipboard:

```
termux-clipboard-get | npm run values:sign -- ~/dgo-values.txt FETCH_ACTIVITIES
termux-clipboard-get | npm run values:sign -- ~/dgo-values.txt GET_DOCS
```

The URL never appears on screen, in your scrollback, or in `~/.bash_history` — it goes from the clipboard into the file and nowhere else.

It refuses rather than corrupting the file:

`8 characters, expected 43`	The copy was cut short. Re-copy with the copy icon.
`FETCH_AL is not a key`	Typo. It lists the 25 real keys.
`already has a signature`	You have done this key. Add `--replace` if you mean to change it.

4.3 If `termux-clipboard-get` hangs or prints nothing

Termux:API is missing or came from a different source than Termux (§0.3). Fix that, or paste interactively — the command reads standard input either way:

```
npm run values:sign -- ~/dgo-values.txt FETCH_ALL
```

Long-press → **Paste**, press Enter, then **Ctrl+D**. Turn auto-correct and auto-capitalise off first: a capitalised first letter silently breaks a signature.

4.4 Clear the clipboard when you are done

```
termux-clipboard-set ""
```

---

§4a · All twenty in one tap

This replaces §3 and §4 entirely. One bookmark fetches every trigger URL from the same API the portal itself calls, using the session you are already signed into, and hands you the finished values file on the clipboard. Twenty designer visits become two taps.

Why a bookmark and not a console script. On a laptop this same harvest is a devtools console run, and §3.2 of **Part 3 of this runbook** is the document that carries that step. Chrome for Android has no devtools console, and remote debugging needs the laptop this runbook exists to do without — so the same harvest ships as a `javascript:` URL, which is the one way a phone browser will run it on a page you are signed into.

It needs the same permission as §4 and no more: you must own or co-own each flow. No tenant admin, no app registration, no PowerShell. It reads each flow and asks for its trigger URL; it changes nothing in the tenant — in particular it regenerates nothing, so nothing a deployed surface is using is revoked by running it.

What it collects is live, not about to be rotated. For these 25 keys the rotation is behind you: `npm run rotation` reports ROTATED, on the evidence that all 25 resolve to workflows other than the ones the published signatures belong to. The 39-flow worklist in §3 of **Part 3 of this runbook** is a wider set than these 25, and it is that document's to carry.

4a.1 Put the bookmark on the clipboard

```
cd ~/ecm_docs_dev
termux-clipboard-set < scripts/harvest-trigger-urls.bookmarklet.txt
```

That file is generated from the endpoint register and is committed. **It carries no credential** — only the 25 keys and their workflow ids, both already public in this repository — so `cat`-ing it, pasting it, or keeping the bookmark afterwards exposes nothing.

If it is ever out of step with the register, rebuild it: `npm run harvest:bookmarklet`.

4a.2 Save it as a bookmark in Chrome

Chrome deletes the `javascript:` prefix when you paste into the address bar — deliberately, and there is no setting for it. A saved bookmark is the way around that:

1. Open any page, tap ⋮ → ☆ to bookmark it.
2. ⋮ → **Bookmarks**, long-press the new one → **Edit**.
3. **Name**: `dgo harvest`. **URL**: clear it and long-press → **Paste**.
4. Save.

4a.3 Run it on a Power Automate tab

1. Open `https://make.powerautomate.com`.
2. In the address bar type `dgo harvest` and **tap the bookmark in the suggestions** — do not press Enter, which searches for the words instead.
3. The first tap says **Open a flow, and this will catch it**. That is expected: the bookmark reads the API address and the token off the portal's own traffic rather than guessing them, so it has to watch one call go past. Tap **Close**, open any flow you own from **My flows**, then tap the bookmark again. Do not reload the page in between.

4. Now a panel slides up listing the 20 flows, naming the API host it observed, and saying how many the portal has already handed over by itself. Nothing has been fetched yet. Tap **Fetch 20 trigger URLs**.
5. It ticks through them, then says `25 keys ready`. Tap **Copy the values file**.

Every URL is checked before it is kept, exactly as on the laptop route: the flow is addressed by its tenant flow id and the URL that comes back must name the workflow id the register holds for that key. A flow answering for a different workflow is discarded, not copied.

Nothing is ever shown on screen. The file is built in memory and goes straight to the clipboard, so no URL and no signature reaches the display, a screenshot, or a screen recording.

If one flow fails, nothing is copied. A half-complete values file wires some endpoints and leaves the rest answering 401 with no pattern to it — so the panel names the flow that failed and offers no copy button. The usual cause is a flow owned by a colleague: ask to be added as a co-owner and tap again, or take that one key through §4 by hand.

There is a second route inside the same bookmark, and it asks the API for nothing. Opening a flow makes the portal fetch that flow's HTTP POST URL so it can show it to you — and that answer goes past the same hook, so the bookmark keeps it without calling anything itself. Each URL is filed under the workflow id it names and matched to a contract key only by that id, so opening the wrong flow cannot produce a wrong credential.

This is the answer when every key answers `401` no matter how many times you re-observe the token: the session is authorised for the maker UI and refused at the management API, and only the portal's own requests get through. Open each flow the failure screen lists, tapping the bookmark whenever you like to check progress, and when the last one is in the button changes from **Fetch N trigger URLs** to **Build the values file**. On a phone this costs almost nothing — tapping through flows is what you would be doing under §4 anyway, and here each visit is one tap instead of a copy.

4a.4 Read the clipboard into the file

Back in Termux:

```
umask 077
termux-clipboard-get > ~/dgo-values.txt
termux-clipboard-set ""
```

The `>` is not optional. `termux-clipboard-get` on its own prints 25 credentials into your scrollback, where they stay. `umask 077` first makes the file readable by your account alone.

Then go to **§5** — `check:values` is the same gate for both routes, and it will tell you immediately if the clipboard arrived truncated.

If the panel says	What it means
`Wrong tab`	The bookmark ran somewhere other than Power Automate. Open a flow you own and tap it there.
`Open a flow, and this will catch it`	Not an error. Nothing has been watched on this tab yet. Close, open any flow you own, tap the bookmark again.
`serves workflow …, not …`	The flow it found is not the one that key belongs to, so its URL was discarded. Not a permissions problem: the register or the crosswalk has moved.
every key `401`	One expired or wrong token, not one problem per flow. Close, open a flow you own, tap the bookmark again. If it persists, reload the portal, wait for **My flows** to list, then tap again. If it *still* persists, stop trying to fix the token: open the flows the panel lists and let the bookmark read the portal's own answers instead.
`asks the API for nothing at all`, then a list of flows	The second route. Those flows have not been opened on this tab yet. Open each one, then tap the bookmark again.
Build the values file where **Fetch …** used to be	Every flow has already been seen. Nothing will be called; tap it.
nothing happens at all	Chrome did not run the bookmark. Check you tapped the suggestion rather than pressing Enter, and that the URL field still begins `javascript:` after saving. If it does not, this phone strips it — use §3 and §4.
`The browser refused the clipboard`	Tap **Copy the values file** again; the grant needs a fresh tap.

---

§5 · Check before writing anything

```
npm run check:values -- ~/dgo-values.txt
```

Prints **no URL, no host and no signature** — only shapes — so its output is safe to show anyone.

Expected: ✅ `25 value(s), all with a complete 43-character signature.`

43 is exact. A signature is base64url of an HMAC-SHA256 — 32 bytes, 43 characters unpadded. Any other number is a defect in the paste.

What you see	What it means
`the URL is complete but its signature is blank`	That key is not done yet. It names which.
`sig is N characters, not 43` — N smaller	Truncated on copy.

What you see	What it means
`sig is N characters, not 43` — N larger	Something extra came with it.
`no sig= parameter`	The line was overwritten. `npm run values:template ~/dgo-values.txt` and start that key again.

---

§6 · Write both config files

```
npm run setup -- --values ~/dgo-values.txt --force
```

The standalone `--` is not a typo — it tells npm the flags belong to the script.

`--force` **is required.** `setup` will not overwrite an existing `config.local.js` without it, and refuses outright (exit 2) rather than silently discarding your values.

`--recover` **is retired** and refuses to run. It read URLs from this repository's reference corpus; the rotation revoked those signatures and the workflows behind them are superseded. It produced a config that looked complete and answered 401 on every call. You do not need it: your values file carries all 24 keys.

Expected:

```
  Internal runtime  (config/config.local.js)
    18/18 endpoints wired
  Public portal     (document-portal/config.local.js)
    7/7 endpoints wired
  ✅ wrote config/config.local.js
  ✅ wrote document-portal/config.local.js
```

---

§7 · Confirm

```
npm run check:config
npm run check:config:portal
npm run commission
```

Both config checks must end `READY. Every key is present, well-formed, and points at the flow it should.` That last clause is the reconciliation working: each URL's workflow id is checked against the register. `points at the WRONG FLOW` means a signature went on the wrong line.

`commission` must end `No automated blocker for pilot usage.` Warnings and manuals remain and are meant to — the manuals need a person, not a script.

---

§7a · Is your terminal transcript safe to paste?

Every command in this runbook is safe. Measured: a canary signature was wired into a real config and every command's output searched for it.

Safe: `npm run reconcile`, `values:template`, `values:sign`, `check:values`, `setup`, `check:config`, `check:config:portal`, `commission`, `verify:endpoints`, `harvest:bookmarklet`, and `git status` / `diff` / `log` — the config files are git-ignored, so they appear in no diff.

Safe, and worth saying so: `cat scripts/harvest-trigger-urls.bookmarklet.txt` and `termux-clipboard-set < scripts/harvest-trigger-urls.bookmarklet.txt`. The bookmark carries workflow ids, which this repository already publishes, and no signature. It is the file it *fetches* that is the credential, and that never lands anywhere but your clipboard and `~/dgo-values.txt`.

Not safe: `cat ~/dgo-values.txt`, `termux-clipboard-get` **on its own** without a pipe or a `>` redirect — after §4a that clipboard holds all 24 credentials at once — `nano ~/dgo-values.txt`, and `history` if you ever typed a signature as part of a command.

Why `values:sign` is shaped as a pipe. The signature is never a shell argument and never reaches the screen, so it is absent from both your scrollback and `~/.bash_history`. Typing `npm run values:sign -- ~/dgo-values.txt FETCH_ALL <the signature>` would put it in both, permanently, in a file you would never think to check — which is why the command does not accept it that way.

`verify:endpoints` **and third-party text.** It prints the first 160 characters of any non-JSON response so a gateway 403 is diagnosable. That body is written by whatever answered — a proxy, a WAF — and echoing the request line back is what those do by default; the request line is the signed URL. Since 2026-09-04 that snippet is redacted at the point of capture, and `npm run test:verification` fails if the redaction is removed. On an older checkout, treat its output as unsafe to paste.

If you have already pasted a signature somewhere. Deleting the message does nothing — the credential is not in the message, it *is* the message. Regenerate that flow's trigger in Power Automate, then redo §4 and §6 for that key.

---

§8 · Afterwards

Keep the values file until §6 of Part 3 of this runbook confirms the endpoints answer. Then:

```
git status --porcelain | grep -iE 'config\.local|dgo-values'   # must print nothing
shred -u ~/dgo-values.txt || rm -f ~/dgo-values.txt
termux-clipboard-set ""
termux-wake-unlock
```

If that `git status` prints anything, **do not commit** — a credential is about to be published. The two `config.local.js` files are git-ignored; a values file is only safe because it lives in `~`, outside the repository, so the grep looks for both.

Keep the bookmark. It holds nothing secret and it is how you re-harvest in one tap after a rotation. What you delete is the values file and the clipboard.

What is left that a phone cannot do. §7 of the walkthrough — building the package and deploying both front ends — and §8's 150-test browser suite. Both need a laptop. Everything before them is done.

Provenance

Everything repository-side was measured on this codebase: the 33 MiB clone size, the 66 MB working tree, the `reconcile` / `values:template` / `values:sign` / `check:values` / `setup` / `check:config` / `commission` outputs (rehearsed end to end against the real tree with placeholder signatures, reaching **18/18 internal, 7/7 portal, READY on both surfaces and zero blockers**, then restored), and the fact that none of it needs `node_modules`.

§4a specifically. The bookmark is generated from the endpoint register, so its 25 keys and 20 workflow ids cannot drift from it — `npm run test:harvestbookmarklet` fails if they do. Its behaviour is not assumed either: `tests/harvest-bookmarklet.test.mjs` executes the payload against a hand-built DOM and asserts the four properties this section relies on — that no request goes out before the tap, that two calls per flow are made with the page's own bearer token, that one unresolvable flow suppresses the file entirely, and that no URL or signature is ever rendered. What that cannot cover is the tenant: the API shapes are the console harvester's, which has been run against this environment, but §4a's own fetches have not.

The Android-side steps — F-Droid packages, `pkg` names, the Termux:API signature-matching requirement, the extra-keys row, the battery setting, and §4a's bookmark mechanics (Chrome stripping `javascript:` on paste, and running a bookmark by name from the address bar) — follow Termux's and Chrome's own documented behaviour and have not been executed on a device from here. If one differs on your phone, §3 and §4 reach the same file, and the repository commands after it are unaffected either way.

Appendix B · The platform at a glance

> **This document carries no commands.** It describes the estate. What to do about it is in `deployment/PLAN.md`; how to do it is in `deployment/RUNBOOK.md`. Those three are the whole documentation set.

1 · What this is

A document and correspondence estate for a government agency, built as **static browser applications calling Power Automate flows directly**. There is no server tier and no API gateway. A page in a browser holds a signed trigger URL for each operation it needs and posts to it; the flow reads and writes SharePoint lists and answers. That one decision explains most of what follows — in particular why authorisation has to live inside each flow, and why a trigger URL is a credential rather than an address.

The surfaces

Surface	Port	What it is
Folio Workspace	8080	The operations platform — routing, RBAC, state, the Admin Suite
NITDA Document Portal	8081	Public submission and tracking; a PWA with its own origin
Folio Estate Console	8082	Standalone endpoint administration, openable from a file path
Folio Flow Workbench	8765	Flow inspection and alignment tooling
Folio Vault	8787	Holds signed flow URLs behind aliases; optional for the Workspace, required for the Portal before public launch

They are zero-build static sites. `npm run platforms` lists them; each starts on its own port so an origin-scoped service worker behaves as it will in production.

2 · How a request travels

```
  browser  ──POST signed trigger URL──▶  Power Automate flow
                                                 │
                                                 ├─ resolve the caller
                                                 ├─ resolve that caller's role from the
directory
                                                 ├─ decide: permitted?
                                                 ├─ read / write SharePoint lists
                                                 └─ answer a standard envelope
```

Every governed step in that chain happens **inside the flow**. There is nowhere else for it to happen: the browser is the client, and between the browser and SharePoint there is only the flow. A client-side route guard is user experience; it is not enforcement.

3 · Identity

There is no external identity provider. Identity is established by emailed one-time code:

1. `OTP_GENERATE` mails a code to an address that must already exist, and be active, in the user directory.
2. `OTP_VERIFY` exchanges a correct code for a **proof** — an opaque token written to a ledger with the address it was issued to and an expiry.
3. Every other endpoint reads the token from the `Authorization` header, looks it up, and takes the caller's address **from the ledger row**, never from the request body.

The proof is a row rather than a signed structure because Power Automate has no signing function. The ledger is what makes the token mean anything, and revoking one proof is a `Consumed` flag rather than a key rotation. Internal proofs live in their own ledger.

Because the trigger URL is itself the credential that gets you to the flow, and because the flow then decides for itself who you are, **a leaked trigger URL is not sufficient to read data — but a flow that trusts the request body makes it sufficient.** That is the single most important property of this design to preserve.

4 · The wired estate

26 contract keys are wired. 24 are named in the flow map. Of those, 15 are already served by the flow the map names, 5 were unresolved by any tenant reading until the map supplied them, and 4 are served by a different flow from the one intended.

Contract key	Flow	Surface
`AI_CHAT`	AI_Document_Processing	internal

Contract key	Flow	Surface
`AI_DOC_ANALYSIS`	AI_Document_Processing	internal
`AI_EMAIL_ANALYSIS`	AI_Document_Processing	internal
`BULK_ASSIGNMENT`	IP_Bulk_Assignment_Endpoint	internal
`BULK_ASSIGNMENT_DIRECT`	Deployed Bulk Task Assignment_Create Task	internal
`CG_Status_Check`	CG_Portal_Status_Check_Endpoint	portal
`CG_Submission`	CG_Submission_Endpoint	portal
`CG_Support`	CG_Portal_Support_Endpoint	portal
`CG_Upload`	CG_Upload_Endpoint	portal
`CG_Verification`	CG_Verification_Endpoint	portal
`CG_Verification_Confirmation`	CG_Portal_Verification_Confirmation_Endpoint	portal
`CG_Writeback`	CG_Portal_Writeback_Endpoint	portal
`DYNAMIC_GLOBAL_ACTIONS`	IP_Dynamic_Global_Actions_Endpoint	internal
`EMAIL`	IP_SEND_EMAIL	internal
`EMAIL_RELATED_TASK`	IP_Create_Email_Assignment_Endpoint	internal
`FETCH_ALL`	IP_FETCH_ALL__EXTENDED_ENDPOINT	internal
`FETCH_EMAIL_ATTACHMENTS`	IP_Retrieve_Email_Attachment_Endpoint	internal
`GET_DOCS`	IP_Get_Docs_Endpoint	internal
`OTP_GENERATE`	IP_OTP_Endpoint	internal
`OTP_VERIFY`	IP_OTP_VERIFY	internal
`REFERENCE_DATA`	IP_Reference_Data_Endpoint	internal
`SCAN_INTAKE`	IP_SCAN_INTAKE_Endpoint_v2.1	internal
`SINGLE_ASSIGNMENT`	IP_Single_Assignment_Endpoint	internal
`SUBSIDIARY_ACTIONS`	IP_Subsidiary_Actions_Endpoint	internal

Trigger posture

26 of 26 wired endpoints accept effectively any body: 17 carry no request schema and 9 carry one that rejects nothing. 0 demand at least one field.

An open trigger is not by itself a defect here: the platform authenticates in the flow, not at the trigger, and the one flow that historically required tenant sign-in did so because its caller arrives as a platform-supplied principal rather than in a body it could read.

5 · What it stores

10 SharePoint lists carry the estate, specified across 98 fields.

List	Fields	Purpose
`DGO_UserDirectory`	21	Authoritative pilot identity, enrollment, status, role and persona register.
`DGO_RoleCatalogue`	9	Canonical RBAC role, persona, permissions and allowed route catalogue.
`DGO_UserRoleHistory`	9	Immutable change history for role/persona assignment events.
`DGO_AuditLog`	14	Central audit trail for governed platform activity and provisioning events.
`DGO_PendingWrites`	10	Offline, failed, or retryable write queue for platform persistence operations.
`DGO_DepartmentDirectory`	6	Directorate, department, unit and DSU reference catalogue.
`DGO_AccessScopes`	6	Controlled access scope definitions for route/directorate scoping.
`DGO_PilotCohorts`	7	Pilot cohort register, ownership and lifecycle tracking.
`DGO_EndpointRegistry`	8	Power Automate endpoint inventory, URL, owner, environment and validation status.
`DGO_AccessEvents`	7	Access-denied, disabled-user, unregistered-user and session diagnostic event log.

`DGO_UserDirectory` and `DGO_RoleCatalogue` are the two that matter most to §3: the first says who exists and whether they are active, the second says what their role may do. Every authorisation decision in the estate resolves through that pair.

6 · How this documentation is kept true

The record of account is `deployment/PRODUCTION_READINESS_REGISTER.json` — version 1.12.87, 81 items, 0 of them open. Every document here is generated from it or from the reference data beside it, and each carries a check that fails on drift. The governing rule, applied throughout:

> **Where a document and a command disagree, the command governs.**

`npm run test:node` runs 182 such checks. They are not decoration: several of the findings in the register were caught by a gate failing on a change its author believed was safe, and the register pins itself to a commit so that evidence citations cannot quietly rot.

7 · Where the work stands

Status	Items	Means

The detail, with what closes each one, is in `deployment/PLAN.md`.

That table is the **readiness register alone**, which is one of 3 places this estate records outstanding work. Across all 3, declared in `deployment/OUTSTANDING_SOURCES.json` and totalled by `npm run outstanding`, **0 items are outstanding** and 122 are closed:

Source	Open	Closed
Production readiness register	0	81
Governance estate position	0	12
Flow-truth persistence review	0	29
Total	**0**	**122**

The count is federated rather than merged on purpose: each source owns its own items and its own closing conditions, and merging them would mean one register deciding when another register's finding is closed. The rollup totals them; it does not adjudicate between them.

The architecture

Everything above is read from the registers and regenerated on every change. What follows is hand-written: the shape of the thing, which no register holds.

1. Applications

Six independently bootable applications share the repository, of which five are the platforms `scripts/platforms.config.mjs` declares and *§1 · The surfaces* above lists with their ports. Only the first uses the full runtime.

Application	Entry	Scale	Role
DGO R11.6 Runtime	`index.html`	135 modules reachable · 29 routes	The platform shell
Document Portal	`document-portal/index.html`	34 files	Public submission & tracking (PWA)
Admin Console	`tools/admin-console.html`	1 file	Endpoint administration, openable from a file path
Flow Workbench	`tools/flow-workbench/`	built by `npm run workbench`	Flow inspection and alignment
Flow Capsule Registry	`tools/flow-capsule/`	Python, `npm run test:capsule`	The capsule registry
AckFlow	`newack/index.html`	5 files	Acknowledgement prototype — not a declared platform and served by no port

Application	Entry	Scale	Role
~~**ECM Activity Hub Portal**~~	—	—	**Retired at D6(b)**; briefs, meetings and projects are now root modules

This table said *four* applications until 2026-09-24 and listed neither the Admin Console, the Flow Workbench nor the Flow Capsule Registry, which had been running on their own ports and carrying their own test suites for some time. It was carried in from the root `PLATFORM_DOCUMENTATION.md`, which nothing regenerated; the surfaces table above is read from the platforms config, so the two disagreed in one document.

The Document Portal is a **progressive web app** — service worker, manifest, offline shell — and is deliberately self-contained so it can deploy standalone. That is why it keeps its own copy of the design tokens rather than sharing the runtime's.

---

2. Runtime architecture

Four layers. Data flows down; nothing in a lower layer knows about a higher one.

```
PRESENTATION   index.html · shared/shell.js · modules/*.js · styles/
     │           web-component shell, hash router, 25 lazy route modules
GOVERNANCE     core/action-authority.js · audit-log · idempotency
     │           · receipt-ledger · offline-action-queue · otp-service
INTEGRATION    core/auth.js · endpoint-registry · data-client · fetch-manager
     │           contract resolution, redaction, retry, dedupe, queueing
BACKEND        Power Automate (19 contracts) · SharePoint · one-time-code identity
```

3.1 Boot sequence

`core/boot.js`:

1. `PlatformProvisioner.ensure()` — normalise state shape against `config/state-schema.config.js`
2. Register all 25 route modules as lazy `import()` thunks
3. Apply theme and density to `<html>`
4. Run the welcome experience (skippable via `?skipWelcome=1` or `?embed=1`)
5. Mount `<dgo-shell>`, install relationship interceptors and the offline retry listener
6. Resolve any deep link, set `window.__DGO_BOOTED__ = true`
7. Load runtime data in the background — failure degrades to offline, never blocks boot

A boot watchdog in `index.html` is deliberately a classic (non-module) script. `boot.js` cannot report its own failure to *resolve*: module resolution happens before any of its code runs, so no `try/catch` fires and no error is logged. The watchdog surfaces failing URLs after 15 seconds. This exists because 12 config modules were once absent and the app simply hung forever with no diagnostic.

3.2 Routing and workspaces

`core/router.js` — hash-based, generation-token guarded so a slow module cannot overwrite a newer route.

Twenty-nine routes exist; **24 are visible workspaces**, 5 are a genuine sub-view of one primary workspace, reached from that parent screen's "Continue in" strip and the command palette rather than the sidebar. This split lives in `config/workflow-clarity.config.js` and follows the locked information architecture from the Figma "Application Shell" page, so navigation reflects operator intent rather than module inventory.

Group	Visible workspaces
START HERE	Command Center · ERP–ECM Charter
OPERATIONS	Activities · Intake & Assignment · My Work · Acknowledgment Queue · Registry · Comments · Lookup & Direct Action
CONTROL	Tracking & Monitoring · FastTrack SLA · Review & Approval · Briefs & Submissions · Meetings · Projects · Reports · Statistics · DGCEO Correspondence & Decision Hub
CLOSURE	Dispatch · Correspondence Email Desk
SYSTEM	Assistant · Operator HUD · Administration · System Health

Sub-views — `single-assignment`, `bulk-assignment` (both under Intake & Assignment), `scan-intake` (under Registry), `archive` (under Dispatch), `user-admin` (under Administration) — each declare which workspace surfaces them and why.

3.3 State

`core/state.js` — a single object, `structuredClone`-isolated, persisted to `localStorage` under `dgo.r11.viewport.runtime.state` (schema v4). Every non-runtime patch auto-records an audit entry (capped at 1,000). Endpoint overrides are sanitised on hydrate so no signed URL is ever persisted.

---

3. The governance spine

This is the platform's most substantial engineering, and the reason the codebase is worth protecting.

Action ownership — `core/action-authority.js`

Every mutating action must be **owned** by the invoking module, or explicitly list it as an allowed invoker. Otherwise it throws.

```
executeOwnedAction(module, action, runner, meta)
  ├── ensureCurrentUserActive()     → status check, audit on denial
  ├── assertModuleAction()          → ownership and boundary check
  ├── audit  stage: started
  ├── runner()
  └── audit  stage: completed | failed
                  └── on failure: toast the operator, then rethrow
```

Backed by `config/action-ownership.config.js` (per-action owner, label, service, backend, audit vocabulary) and `config/module-boundaries.config.js` (what each module owns and explicitly must not own).

Failure messaging — the contract

This layer is the only place that reliably sees **every** failed governed action, so it owns what the operator reads when one fails. Two audiences, two channels, and they never carry the same text:

Channel	Carries	Read by
`auditAction(… stage:'failed', error: error.message)`	The raw technical detail, verbatim	IT — also surfaced on System Health, the Operator HUD and the pending-write queue
The toast	`Could not <action label>. <what it means for the record>`	The operator

The operator-facing message is **never** derived from `error.message`. It is built from two things:

- the action's `label` in `config/action-ownership.config.js` — a plain noun phrase, presentation-only, mandatory on every action;
- the error's `errorClass` where the throw used `ObsidianError` (`core/errors.js`), which selects a plain consequence sentence. `ObsidianError` carries a typed class and `details`, but **no user-safe message field**, so its `message` is treated as developer text. An untyped error falls back to "Nothing was changed."

One failure, one toast. The layer speaks by default. A call site that can say something genuinely more useful — naming the reference, the recovery it just took, what is still outstanding — passes `meta.notify:false` and takes responsibility for telling the operator itself. Seven call sites across six modules do (archive, assistant, correspondence-email, dispatch, settings, bulk-assignment ×2); every other governed call site lets this layer speak. `ActionRuntime.run(module, action, payload, options)` forwards `options.notify` for callers routed through it.

The opt-out is explicit at the call site by design. A time-window de-duplication in the toast helper would hide two messages racing rather than stop them being sent.

> This exists because audit finding **I-07** — developer vocabulary on operator screens — was resolved in every module's copy and then reintroduced here: an unconfigured connection reached a registry officer as `"AI_CHAT: Endpoint AI_CHAT is not configured"`, and the module's own catch showed it a second time. `tests/governance.test.mjs` holds both halves: that the audit keeps the raw detail, and that the toast never does.

Audit — `core/audit-log.js`

Ring buffer of 5,000 events, indexed by reference, returned frozen. Records actor, event, phase, status transition, entity type and id.

Idempotency — `core/idempotency.js`

```
idem:<operation>:<ref>:<actor-email>:<5-min bucket>:<sha256 payload digest>
```

Sent with every write so a retry cannot double-apply.

Offline durability

`pending-queue` → `offline-action-queue` → `receipt-ledger`. Failed writes enqueue with retry metadata; an `online` listener drains them; each attempt writes a receipt (`queued` / `sent` / `failed`). Acknowledgements survive a network outage.

Step-up authentication — `core/otp-service.js`

Request, verify, and verify-and-execute — with the OTP **bound to a payload digest**, so a verified code cannot be replayed against different data.

Write discipline — `core/write-manager.js`

Three modes: `local` (state only), `backend` (remote with idempotency), `optimistic` (patch, then roll back on failure or enqueue).

> **The governance model was never the weakness. Its input was.** Every control above is enforced in the browser only. Section 6 addresses this.

---

4. Access control

Six roles, thirteen permissions, in `config/rbac.config.js`.

Role	Route access
`systemAdmin`	all
`userAdmin`	home, settings, user-admin, diagnostics, operator-hud

Role	Route access
`executive`	home, executive, response-tracking, approvals, reports, statistics, lookup, assistant, archive
`director`	operations + approvals, dispatch, reports, statistics
`operator`	operations + registry, comments, correspondence-email
`viewer`	home, response-tracking, reports, statistics, lookup

Permissions: `user:view/create/update/disable` · `role:assign/view` · `settings:manage` · `audit:view` · `dispatch:approve` · `bulk:assign` · `route:manage` · `executive:view/export`.

Route guards in `core/router.js` render an explicit denial rather than a blank page, distinguishing *disabled*, *not enrolled* and *role cannot open*.

---

5. Design system

`styles/index.css` declares a deterministic cascade:

```
@layer tokens, brand, base, layout, components, overrides;
```

Tokens layer in order: primitive → semantic → component → density → enhanced → theme(light, dark, hc) → legacy bridge.

Theme and density live on `<html>` only. Every themed rule is a bare `[data-theme="…"]` selector written for `:root`. Mirroring those attributes onto descendants makes the selector match directly and beat inheritance — which silently broke dark mode until it was removed. `CONTRIBUTING.md` records this as a rule; the smoke suite asserts the mirrors stay absent.

Three themes (light, dark, high-contrast) × two densities (comfortable, compact).

> **Known debt:** the `overrides` layer holds two authorities whose rule interaction is documented at length in `styles/index.css` and remains unresolved. There is currently no rendered-appearance regression coverage beyond the smoke suite's theme assertion.

---

6. Accessibility

Skip link to `#main`, one `<nav>` and one `<main>` landmark, `lang="en"`, three ARIA live regions, zero images without `alt`, zero buttons without an accessible name. Drawers trap Tab and restore focus to the opener. Tables receive `data-label` attributes for responsive stacking. High-contrast theme reaches pure black on white and includes a `forced-colors` block.

Open: a duplicate top-level heading on the shell, and a full contrast pass now that the themes actually apply.

---

7. Quality gate

```
npm test              # imports → secrets → auth → smoke
npm run test:imports  # static ES-module graph check, no browser, ~1s
npm run test:secrets  # SAS signature ratchet
npm run test:auth     # both authentication postures
npm run test:governance # the governance spine, no browser, ~1s
npm run test:smoke    # Playwright, 6 tests
npm run test:links    # linkinator crawl
```

Check	What it proves
`check-imports.mjs`	Every relative import resolves. **167 modules, 0 broken edges.** The check that would have caught the original boot failure in one second.
`check-secrets.mjs`	A ratchet, not a gate — fails on *new* signatures, reports known ones. Deleting a file revokes nothing, so failing on known exposure would only make CI permanently red.
`auth-posture.test.mjs`	Both postures, 25 assertions, one child process each.
`governance.test.mjs`	The spine itself, 108 assertions, no browser: action ownership and boundaries, RBAC, idempotency keys, the audit log, cross-config integrity, provisioning parity, endpoint contracts — and the execution path inside `executeOwnedAction()`: an unowned action refused, an allowed invoker permitted, a failure audited with the raw detail intact, exactly one operator toast carrying none of it, and the rethrow contract held. Includes a negative control so the vocabulary assertions cannot pass vacuously.
`smoke.spec.js`	Boot, accessibility entry points, all 29 routes, theme repaint, `?skipWelcome=1`, ECM portal. Gated on `__DGO_BOOTED__`, **not** HTTP 200 — a 200 proves the server served `index.html` and nothing more.

CI (`.github/workflows/ci.yml`): `imports` gates `smoke` and `links`; `secrets` runs independently.

> **Closed.** This section used to read: *"the governance spine itself has no tests. The smoke suite proves pages render; it proves nothing about whether action ownership still blocks an unowned action."* `governance.test.mjs` now does exactly that, and covers what `executeOwnedAction()` audits and says as well as what it refuses. The gap it closed was real and had a cost: with only the gate tested, the layer could put raw error text on an operator's screen (finding I-07) and CI stayed green.
>
> **Remaining gap:** enforcement is still browser-side only — see section 6. Tests prove the controls work; they cannot prove a client that skips them was ever asked to run them.

---

8. Configuration

```
cp config/config.example.js config/config.local.js   # git-ignored
```

`index.html` loads it with `onerror="void 0"` — a 404 is expected and allow-listed by both the smoke suite and the link checker.

File	Purpose
`app.config.js`	Version, storage key, schema version, themes, densities
`auth.config.js`	The authentication switch
`rbac.config.js`	Roles, permissions, route access
`routes.config.js`	25 route declarations
`workflow-clarity.config.js`	Visible workspaces vs guided internal routes
`endpoints.config.js`	19 contracts
`module-boundaries.config.js`	What each module owns and must not own
`action-ownership.config.js`	Per-action owner, plain-language label, service, audit vocabulary
`fetch-policy.config.js`	Timeout, retry, dedupe, cache TTL, payload budget

---

9. Repository layout

```
index.html                    Runtime entry + boot watchdog
config/            31 files   Declarative platform configuration
core/              57 files   Boot, router, state, governance, integration
modules/           25 files   Lazy-loaded route modules
shared/             8 files   Shell, adapters, welcome runtime
styles/            18 files   @layer cascade + design tokens
tests/              6 files   Import, secret, auth and smoke suites
modules/briefs.js …            briefs, meetings and projects (ported from the retired ECM
Activity Hub)
document-portal/              Public portal (PWA)
newack/                       Acknowledgement prototype
ECM_DOCS_DEV.zip              Archive of record — reference material
```

Documents: `PLATFORM_DOCUMENTATION.md` · `docs/STATUS_REPORT.md` · `docs/architecture/AUTHENTICATION_CONTRACT.md` · `docs/audits/CAPABILITY_ASSESSMENT_R11.6.md` · `docs/audits/REPOSITORY_AUDIT.md` · `docs/audits/INDEX.md` · `README.md` · `CONTRIBUTING.md`

---

10. Known limitations

Recorded plainly; each is tracked in `docs/STATUS_REPORT.md`.

1. **Nothing is enforced server-side.** All governance is browser-side. Until `docs/architecture/AUTHENTICATION_CONTRACT.md` §2 is implemented, controls are advisory.
2. ~~**The ECM Portal has no auth work at all**~~ — moot since D6(b): the tree is deleted, so there is one auth surface rather than two.
3. **Four signed URLs remain** in `document-portal/js/data.js` and `newack/config.js`; 22 pilot signatures still require rotation.
4. ~~**The governance spine is untested.**~~ — closed: `tests/governance.test.mjs` covers ownership, boundaries, RBAC, idempotency, the audit log, provisioning parity and the execution path inside `executeOwnedAction()` (108 assertions). Enforcement remains browser-side, which is limitation 1, not this one.
5. **No rendered-appearance regression coverage**; the `overrides` cascade debt is unmeasured.
6. **One duplicate** `<h1>` on the shell.